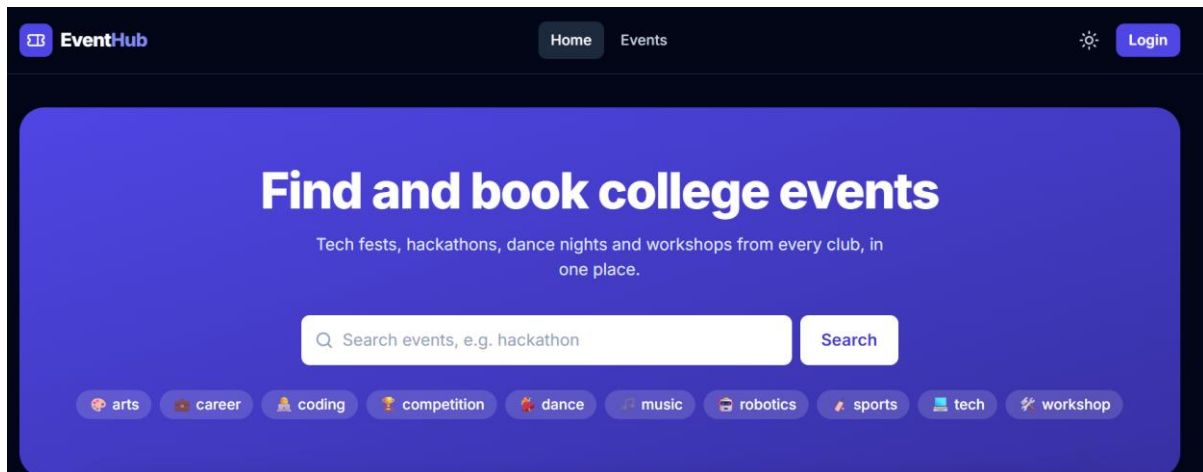


EventHub

College Event Management Platform

Complete Project Guide

Setup, architecture, code, database, API, testing, troubleshooting, user guide and viva questions - explained step by step with examples



Madhavan Suresh

Java Full Stack practice project · React + Spring Boot + MySQL

Version 1.0 · 29 September 2026 · github.com/madhavansuresh000-sys/eventhub

Contents

1. Introduction	5
1.1 What is EventHub?	5
1.2 Why this project? (the problem)	5
1.3 Objectives	6
1.4 Scope	6
1.5 Who should read this document	7
1.6 How this document is written	7
1.7 The demo accounts	7
2. Requirements	8
2.1 Functional requirements by role	8
2.2 Business rules (the most important ones)	9
2.3 Non-functional requirements	10
2.4 User stories with examples	10
3. Technologies used	12
3.1 The big picture	12
3.2 Frontend technologies	12
3.3 Backend technologies	14
3.4 Database and infrastructure	15
3.5 Quality tools	16
4. System architecture	17
4.1 The three tiers	17
4.2 Layers inside the backend	17
4.3 The journey of one request	18
4.4 Package structure (backend)	18
4.5 Folder structure of the repository	19
4.6 Important design decisions	19
5. Setup and installation	21
5.1 What you need to install	21
5.2 Step 1 - get the code	21
5.3 Step 2 - create your .env file	21
5.4 Step 3 - start MySQL and Mailpit	22
5.5 Step 4 - start the backend	23
5.6 Step 5 - start the frontend	24
5.7 Step 6 - run the tests	24
5.8 Step 7 - try the API with Postman	25
5.9 Useful addresses (development)	25

5.10 Stopping everything	25
5.11 The demo tour	25
6. Database design	27
6.1 Overview	27
6.2 Rules that protect the data	27
6.3 Optimistic locking columns	28
6.4 Every table, column by column.....	28
6.5 Useful SQL queries (with examples)	33
6.6 How Java classes map to tables (JPA)	34
6.7 Example: what the tables look like after one booking	35
7. Backend code explained	37
7.1 The application entry point.....	37
7.2 Errors that make sense: GlobalExceptionHandler	37
7.3 Events: search, rules and the approval workflow	38
7.4 Security: login, JWT cookie, CSRF and permissions	41
7.5 Bookings: holding seats without ever overselling	44
7.6 Payments: one interface, two gateways.....	47
7.7 The smart waitlist.....	49
7.8 Notifications: bell, email and reminders.....	50
7.9 The gate: QR codes and check-in	51
7.10 Certificates	52
7.11 Feedback	53
7.12 Analytics with caching.....	54
7.13 The audit log	55
8. REST API reference.....	56
8.1 What a REST API is	56
8.2 All endpoints	56
8.3 Examples with request and response	58
8.4 Swagger UI	59
8.5 The Postman collection.....	59
9. Frontend explained	60
9.1 How the React app starts.....	60
9.2 Folder by folder.....	60
9.3 Talking to the backend.....	60
9.4 Redux slices	61
9.5 Protecting pages: RequireAuth	62
9.6 Forms and validation.....	63
9.7 Timers, QR codes, charts and code splitting.....	64

9.8 Design system, dark mode and accessibility	65
10. User guide (with screenshots)	66
10.1 Visitor: finding events	66
10.2 Student: booking and paying	66
10.3 Student: my tickets, waitlist, notifications.....	68
10.4 Student: certificates and feedback	69
10.5 Volunteer: the gate scanner	70
10.6 Anyone: verifying a certificate	71
10.7 Organizer: creating and managing events	71
10.8 Admin: approvals, overview, analytics	73
10.9 What happens when someone is not allowed.....	75
11. Testing.....	76
11.1 Why test automatically?	76
11.2 The testing pyramid	76
11.3 The numbers	76
11.4 Unit tests with Mockito.....	77
11.5 Integration tests with Testcontainers	78
11.6 Concurrency tests	79
11.7 React component tests	80
11.8 Continuous integration (GitHub Actions)	81
11.9 Bugs the tests found	82
11.10 How to run the tests	82
12. Security in depth	83
12.1 The main threats and the answers	83
12.2 Authentication vs authorization	83
12.3 Why the JWT is in a cookie	84
12.4 The login lock in action	84
12.5 Security checklist (Phase 8).....	84
13. Troubleshooting	85
13.1 Setup and start-up	85
13.2 Login and security	86
13.3 Booking, payment and gate	86
13.4 Tests	86
13.5 How to investigate any problem.....	87
14. Lessons learned.....	88
14.1 Technical lessons.....	88
14.2 Working lessons	88
15. Deployment plan and future scope	89

15.1 Deployment plan (Phase 9).....	89
15.2 Future scope	89
16. Glossary.....	90
17. Viva and presentation questions with answers.....	92
17.1 About the project.....	92
17.2 Architecture	92
17.3 Backend and Spring.....	92
17.4 Concurrency and correctness	93
17.5 Security	94
17.6 Payments, waitlist, notifications, certificates	94
17.7 Frontend.....	95
17.8 Testing and quality.....	95
17.9 Database	96
17.10 Future and reflection	96
18. How to present EventHub (demo script)	97
18.1 Before you start (checklist)	97
18.2 The flow (about 11 minutes).....	97
18.3 Tips for presenting	98
19. How EventHub was built: the phase-by-phase story	99
20. How to change EventHub (developer how-to)	100
20.1 Add a new column (example: a "meeting link" for online events).....	100
20.2 Add a new endpoint (example: "my upcoming events count").....	100
20.3 Add a new page.....	100
20.4 Add a new business rule (example: max 2 active bookings per student per day).....	101
Appendix A. Commit history	102

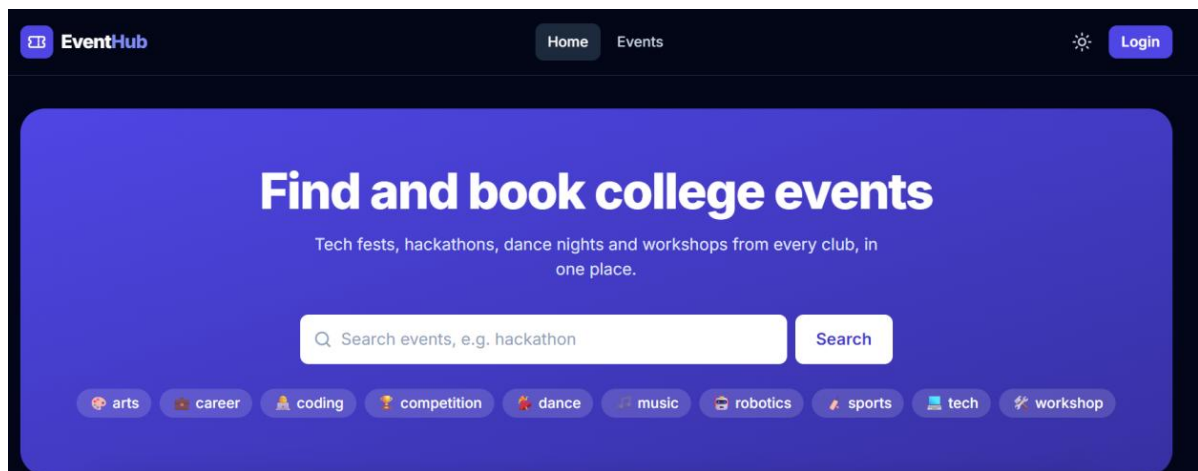
1. Introduction

1.1 What is EventHub?

EventHub is a website where a college runs all of its events in one place. A student can find an event, book a seat, pay, receive a QR ticket, enter through the gate by showing that QR code, download a certificate after the event and rate the event. Clubs create the events, the Student Affairs Office (the admin) approves them, and volunteers check tickets at the gate.

It is a **full stack** application: the part you see in the browser (the **frontend**) is written in React, the part that stores data and applies the rules (the **backend**) is written in Java with Spring Boot, and the data lives in a MySQL database.

Real-life example: Think of a cinema. The cinema website shows the films (frontend). Behind it, a ticket system makes sure two people never get the same seat and records every payment (backend). The list of seats and bookings is kept in a big register (database). At the door, a staff member scans your ticket once (gate check-in). EventHub is the same idea for college events.



The EventHub home page - students search events or pick a tag

1.2 Why this project? (the problem)

In many colleges, events are organised with WhatsApp groups, Google Forms and printed lists. This causes real problems:

- **Overbooking.** Two students fill the form for the last seat at the same time. Both believe they are in.
- **Fake or shared tickets.** A screenshot of a confirmation message is forwarded. At the door nobody can tell whether it was already used.
- **Slow, fake-able certificates.** Certificates are made by hand, weeks later, and a nicely edited PDF can be put on a resume even by someone who never attended.
- **No numbers for organizers.** Clubs do not know how many tickets they sold per day, how many people actually came, or what people thought of the event.
- **No control.** Anybody can announce an event; there is no approval step and no record of who changed what.

EventHub solves each of these problems with a clear rule in the software, and the rest of this document shows exactly how.

1.3 Objectives

#	Objective	How EventHub meets it
1	Easy discovery	Search by text, filter by tag, club and date; results are filtered and paged by the server
2	Fair booking	Seats are held for 10 minutes while paying; optimistic locking guarantees no overselling
3	Smart waitlist	When a seat frees up, it is kept 30 minutes for the next student on the waitlist
4	Fast, safe entry	Each ticket has a random code shown as a QR code; the gate accepts it only once
5	Proof of attendance	PDF certificates only for students who were scanned at the gate, verifiable by anyone
6	Control	Events need admin approval; every change is written to an audit log
7	Insight	Feedback stars, check-in rate and sales charts for organizers and the admin
8	Quality	146 automated tests, 91% service coverage, every push tested by GitHub Actions

1.4 Scope

Inside the scope of this project:

- Public catalogue of events, clubs and tags
- Registration and login with roles (student, organizer, volunteer, admin)
- Event creation, editing and the approval workflow
- Booking with seat holds, payment (Stripe test mode or a built-in test page), cancellation and refunds
- Waitlist with automatic offers, notifications (bell + email) and day-before reminders
- QR tickets and a gate scanner with a live counter
- PDF certificates and a public verification page
- Feedback and analytics dashboards
- Automated tests, continuous integration and full documentation

Outside the scope (see the Future Scope chapter): password reset by email, a mobile app, several colleges on one installation, and a real "volunteers" management page (volunteers exist and can scan; only the management screen uses sample names).

1.5 Who should read this document

Reader	Start with
A teacher or examiner	Chapters 1, 2, 4, 11 and 17 (viva questions)
A developer who wants to run it	Chapter 5 (setup), then chapter 13 (troubleshooting)
A developer who wants to change the code	Chapters 4, 6, 7, 8 and 9
A user of the website	Chapter 10 (user guide with screenshots)
An interviewer	Chapters 12 (security), 14 (lessons) and 17

1.6 How this document is written

- **Simple English.** Every technical word is explained the first time it appears, and the Glossary (chapter 16) repeats the definitions.
- **Real examples.** The examples use the real demo data: the student **Ravi Kumar**, the organizer **Madhavan** (Coding Club), the volunteer **Priya Raman**, the organizer **Kavya** (Cultural Club) and the **admin** of the Student Affairs Office.
- **Real code.** Every code excerpt is copied automatically from the project files when this document is built, so the code you read here is exactly the code on GitHub.
- **Real screenshots.** All screenshots were taken by a script (`tools/edge-tour.mjs`) that drives the real application in Microsoft Edge.

Note: Boxes like this one contain notes: extra detail, a warning or a tip.

Real-life example: Boxes like this one compare a technical idea with something from everyday life.

1.7 The demo accounts

In development the backend creates five demo accounts automatically (class `DevDataSeeder`). They all use the password stored in `DEMO_PASSWORD` in the `.env` file.

Email	Who	What they can do
<code>admin@eventhub.test</code>	Admin (Student Affairs Office)	Approve / send back events, see all analytics and the activity log
<code>madhavan@eventhub.test</code>	Organizer of the Coding Club	Create and submit Coding Club events, see its analytics and feedback, scan tickets
<code>kavya@eventhub.test</code>	Organizer of the Cultural Club	The same, but only for the Cultural Club
<code>priya@eventhub.test</code>	Volunteer of the Coding Club	Scan tickets at the gate for Coding Club events
<code>ravi@eventhub.test</code>	Student	Book, pay, tickets, waitlist, certificates, feedback

2. Requirements

Requirements describe **what** the system must do (functional) and **how well** it must do it (non-functional). They were written before the code, phase by phase, in the checklists of the **Phase_*** folders.

2.1 Functional requirements by role

Visitor (not logged in)

ID	Requirement
V1	See upcoming published events on the home page, with search and tag chips
V2	Search, filter (tag, club, from/to date) and sort (date, price, title) all events, with paging
V3	Open an event: description, venue, time, price and a live "seats left" bar
V4	Open a club page with its events
V5	Register a student account; log in
V6	Verify a certificate number without logging in

Student

ID	Requirement
S1	Book 1-10 tickets for a published future event; seats are held for 10 minutes
S2	Pay (Stripe Checkout, or the built-in test page when no Stripe key is set); free events are confirmed at once
S3	See "My tickets" (upcoming, past, cancelled) and open a QR ticket; print it
S4	Cancel a confirmed ticket before the event (seats go back, payment refunded)
S5	Join the waitlist of a sold-out event and see the position ("You are #2"); accept an offer within 30 minutes
S6	Receive notifications (bell) and emails: booking confirmed, seat offered, reminder the day before
S7	Download a PDF certificate for events attended (scanned at the gate) that are over
S8	Rate an attended event 1-5 stars with an optional comment; change the rating later

Organizer (per club)

ID	Requirement
O1	Create and edit events of their own club (title, description, venue, times, seats, price, tags)
O2	Submit a draft for approval; see the admin's note when an event is sent back
O3	Dashboard: published events, tickets sold, money collected, waiting approvals
O4	Analytics for the last 7 / 30 / 90 days: tickets and money per day, check-in rate, ratings
O5	Read the feedback of their events (anonymous)
O6	Use the gate scanner for their club's events

Volunteer (per club)

ID	Requirement
G1	Choose an event of their club and scan QR tickets (camera or typed code)
G2	See VALID (let in), ALREADY USED (when, by whom) or INVALID, in big colours
G3	See a live counter "checked in / booked" shared by all gates

Admin

ID	Requirement
A1	Approval queue (soonest first): approve, or send back with a reason
A2	Overview per club: published, waiting, tickets sold, money
A3	Activity (audit) log: who created, edited, submitted, approved or rejected each event
A4	Analytics for all clubs or one club
A5	Open the API documentation (Swagger) and technical pages (Actuator)

2.2 Business rules (the most important ones)

Business rules are the "laws" of the system. The server checks every one of them, even if the page already did.

Rule	Where it is checked
End time must be after start time	<code>EventService.checkTimes</code>
Seats cannot be reduced below the number already booked	<code>EventService.update</code>
An event waiting for approval cannot be edited	<code>EventService.update</code>
Only future events can be submitted	<code>EventService.submit</code>
Status moves only DRAFT → PENDING_APPROVAL → PUBLISHED (or back to DRAFT)	<code>EventStatus.canMoveTo</code>
Never sell more seats than exist	<code>@Version on Event + Retry.onConflict</code>
One active booking per student per event	<code>BookingService.holdOnce</code>
A ticket can be used at the gate only once	conditional <code>UPDATE</code> in <code>BookingRepository.markCheckedIn</code>
A used ticket cannot be cancelled	<code>BookingService.cancel</code>
Certificate only if checked in AND the event has ended	<code>BookingRepository.findIdsEarningCertificate</code>
Rating only by attendees, after the event, 1-5 stars	<code>FeedbackService.give</code> + database CHECK
5 wrong passwords (same email + address) → locked 15 minutes	<code>LoginAttemptService</code>

2.3 Non-functional requirements

Quality	Requirement	How it is met
Correctness under load	100 students clicking "book" at the same second never oversell	optimistic locking + retry; concurrency test with 100 threads
Security	Passwords never stored in plain text; tokens not readable by scripts; roles checked on the server	BCrypt, httpOnly JWT cookie, CSRF, <code>@PreAuthorize</code>
Performance	Search and lists are paged; dashboards do not hammer the database	server-side paging, <code>@EntityGraph</code> , <code>@BatchSize</code> , 60-second cache
Usability	Works on a phone (375 px wide), light and dark mode, readable contrast	Tailwind responsive layout; automated contrast and tap-size checks
Accessibility	Labels on every input, screen-reader announcements for results, table view for charts	<code>aria-*</code> attributes, <code>role="alert"</code> , "Show the numbers as a table"
Maintainability	Clear modules, one migration per database change, readable code with comments	package per feature, Flyway, code comments in simple English
Testability	Automatic tests for every rule; CI on every push	146 tests, Testcontainers, GitHub Actions
Portability	Runs on any laptop with Docker, Java and Node	<code>docker-compose.yml</code> , Maven wrapper, <code>package.json</code>

2.4 User stories with examples

A user story says who wants what and why. Each one below has an example with the demo data.

1. *As a student, I want to see how many seats are left, so that I know if I must hurry.* Example: Ravi opens "24-Hour Hackathon" and sees "110 booked / 120 · 92% full".
2. *As a student, I want my seats kept while I pay, so that nobody takes them in the meantime.* Example: Ravi presses "Book now"; the checkout shows a 10:00 timer; the two seats are his for ten minutes.
3. *As a student on a waitlist, I want to be told when a seat frees up.* Example: Asha cancels; Ravi (#1 on the waitlist) gets an email "A seat is waiting for you" and a bell notification, with 30 minutes to accept.
4. *As a volunteer, I want a clear answer at the gate, so that the queue moves fast.* Example: Priya scans Ravi's QR: a big green "LET IN - Ravi Kumar". Someone tries the same QR again: red "ALREADY USED at 8:41 pm by Priya Raman".
5. *As a company recruiter, I want to check a certificate, so that I know it is real.* Example: the recruiter types EH-2026-BEQR4Y79 on /verify and sees "Genuine EventHub certificate - Ravi Kumar - Linux Basics (demo)".
6. *As an organizer, I want to know if people liked my event.* Example: Madhavan opens the feedback page of Linux Basics: average 4.0 stars and the comment "Clear explanations of the terminal", without the student's name.
7. *As the admin, I want to review events before they are public.* Example: the admin opens the approval queue, reads "Drone Building Workshop", and sends it back with "Please add the venue capacity".

3. Technologies used

This chapter explains every important tool in simple words: what it is, why EventHub uses it, and a small example from the project.

3.1 The big picture

Layer	Technology	Version
Frontend language	JavaScript (JSX)	ES2024
Frontend library	React	19
Build tool / dev server	Vite	8
Styling	Tailwind CSS	4
Shared state	Redux Toolkit + React Redux	2 / 9
Routing	React Router	7
HTTP client	Axios	1
Charts / QR camera	Recharts / html5-qrcode	3 / 2
Backend language	Java	25
Backend framework	Spring Boot (Web MVC, Data JPA, Security, Validation, Mail, Cache, Actuator)	4.1
Database	MySQL	8.4
Database migrations	Flyway	via Spring Boot
Tokens	Nimbus JOSE (JWT, HS256) via spring-security-oauth2-jose	via Spring Boot
QR images / PDFs	ZXing / OpenPDF	3.5 / 2.2
Payments	Stripe Java SDK	29
Cache	Caffeine	via Spring Boot
API documentation	springdoc-openapi (Swagger UI)	3.1
Tests (backend)	JUnit 5, Mockito, AssertJ, MockMvc, Testcontainers, JaCoCo	
Tests (frontend)	Vitest, React Testing Library, jsdom	5 / 16
Containers	Docker Desktop, Docker Compose	
CI	GitHub Actions	

3.2 Frontend technologies

React

React builds a page out of small, reusable **components**. A component is a JavaScript function that returns what should appear on the screen (written in JSX, which looks like HTML). When data changes, React redraws only the parts that changed.

Real-life example: A component is like a LEGO brick. **EventCard** is one brick; the events page puts twelve of them in a grid; the home page reuses the same brick in the "Upcoming events" row.

Example from EventHub: the gate scanner shows its answer with the small component **ResultPanel**:

ResultPanel.jsx (frontend/src/components/scanner/ResultPanel.jsx)

```
/** The answer of one scan: VALID / ALREADY_USED / INVALID (from POST
/api/gate/events/{id}/check-in). */
export default function ResultPanel({ result }) {
  if (!result) {
    return (
      <div className="grid min-h-40 place-items-center rounded-2xl border-2 border-dashed
border-slate-300 p-6 text-center text-slate-500 dark:text-slate-400 dark:border-slate-700">
        Scan a ticket or type its code.
      </div>
    )
  }
  const l = look[result.result]
  return (
    <div role="alert" className={`rounded-2xl p-6 text-center text-white ${l.color}`}>
      <div className="text-6xl leading-none font-black" aria-hidden="true">{l.mark}</div>
      <p className="mt-3 text-3xl font-extrabold tracking-wide uppercase">{l.word}</p>
      {result.holder} && <p className="mt-1 text-lg font-semibold">{result.holder}</p>
      <p className="mt-1 text-white/90">{result.message}</p>
      <p className="mt-3 font-mono text-sm text-white/80">{result.code}</p>
    </div>
  )
}
```

Vite

Vite is the development server and build tool. **npm run dev** starts a server on port 5173 that reloads the page the moment a file is saved. **npm run build** produces small, optimised files for production. Vite also **forwards** every call that starts with **/api** to Spring Boot on port 8080, so the browser only ever talks to one address:

vite.config.js (frontend/vite.config.js, lines 5-20)

```
// https://vite.dev/config/
export default defineConfig({
  plugins: [react(), tailwindcss()],
  server: {
    // send /api/... requests to Spring Boot during development
    proxy: {
      '/api': 'http://localhost:8080',
    },
  },
  // Phase 8: `npm test` runs the component tests (Vitest + React Testing Library) in a fake
  browser (jsdom)
  test: {
    environment: 'jsdom',
    setupFiles: ['./src/test/setup.js'],
  },
})
```

Tailwind CSS

Tailwind is a way of styling with small ready-made class names written directly on the elements, such as **rounded-2xl p-6 text-white bg-green-700**. The EventHub colours (the indigo "brand" colours) are defined once as design tokens in **index.css**. Dark mode works by adding a **dark** class on the **<html>** element; classes such as **dark:bg-slate-900** then apply.

Redux Toolkit

Some data is needed by many pages at once: who is logged in, the seats being held in the cart, the organizer's events. Redux keeps this data in one **store**. Each area has a **slice** (auth, cart, notifications,

student, organizer, admin) with its own actions. Pages read with `useSelector` and change things with `useDispatch`.

Real-life example: The Redux store is like the notice board in a college office: everybody reads the same board, and only the office staff (the reducers) may change what is pinned on it.

React Router

React Router shows the right page for the address in the browser, without reloading the whole page. EventHub has 29 routes, for example `/events/:id`, `/checkout/:bookingId`, `/organizer/analytics` and `/verify/:number`. Protected routes are wrapped in `RequireAuth`, which sends visitors to the login page and remembers where they wanted to go (`?next=`).

Axios

Axios sends HTTP requests to the backend. EventHub creates one configured instance (`api/client.js`) with the base address `/api`. It automatically sends cookies and copies the CSRF token into the `X-XSRF-TOKEN` header.

Recharts and html5-qrcode

Recharts draws the bar charts of the analytics dashboard. html5-qrcode reads QR codes from the laptop or phone camera on the scanner page. Both are loaded only when those pages are opened (code splitting, see chapter 9).

3.3 Backend technologies

Java 25 and Spring Boot 4

Java is the programming language of the backend. Spring Boot is a framework: it gives ready-made building blocks (a web server, database access, security, email, scheduling) and connects them automatically, so the project only writes the business logic.

Important ideas of Spring used everywhere in EventHub:

- **Bean:** an object created and managed by Spring (for example `BookingService`).
- **Dependency injection:** a class says which beans it needs in its constructor, and Spring passes them in. With Lombok's `@RequiredArgsConstructor` this is one line.
- **Annotations:** labels such as `@RestController`, `@Service`, `@Transactional`, `@Scheduled` that tell Spring what to do with a class or method.

Real-life example: Dependency injection is like a hotel room: you do not bring your own bed and lamp; you ask for "a room" and everything you need is already there.

Spring Web MVC

Turns Java methods into web addresses. A method with `@GetMapping("/api/events/{id}")` answers `GET /api/events/5`. Java objects are converted to JSON automatically (Jackson 3).

Spring Data JPA and Hibernate

JPA maps Java classes (**entities**) to database tables. Hibernate is the library that writes the SQL. Spring Data creates **repositories**: an interface such as `BookingRepository` extends `JpaRepository<Booking, Long>` already has `save`, `findById`, `findAll`; methods named like `findByUserIdOrderByCreatedAtDesc` are turned into queries automatically; complex ones are written in JPQL with `@Query`.

Flyway

Flyway runs the SQL files in `db/migration` in order (V1, V2, ... V9) and remembers which ones already ran (table `flyway_schema_history`). So every laptop, test database and server always has exactly the same tables. Hibernate is set to `ddl-auto: validate`: it only checks that the entities match the tables and refuses to start if they do not.

Spring Security

Every request passes through a chain of filters before it reaches a controller. EventHub adds its own `JwtCookieFilter` (reads the login cookie) and configures CSRF protection, CORS and URL rules in `SecurityConfig`. Method-level rules use `@PreAuthorize`, for example `@PreAuthorize("@clubAccess.isOrganizer(#clubId)")`.

Other backend libraries

Library	Used for	Example in EventHub
Bean Validation	Checking request fields	<code>@NotBlank</code> , <code>@Size(max = 200)</code> , <code>@Min(1)</code> in <code>EventRequest</code>
Spring Mail	Sending emails	<code>EmailSender</code> sends "Seat offered" through Mailpit
Spring Scheduling	Jobs on a timer	<code>HoldExpiryJob</code> every 60 s; reminders at 18:00
Spring Cache + Caffeine	Remembering answers	analytics kept 60 s with <code>@Cacheable</code>
ZXing	Drawing QR codes	<code>GET /api/bookings/{id}/qr.png</code>
OpenPDF	Drawing PDFs	certificate PDF with name, event and QR
Stripe Java	Payments	Checkout session, webhook signature, refunds
Lombok	Less boilerplate	<code>@Getter</code> <code>@Setter</code> on entities
springdoc-openapi	API documentation	Swagger UI at <code>/swagger-ui.html</code> (admin only)

3.4 Database and infrastructure

MySQL 8.4

A relational database: data is stored in tables with rows and columns, and tables are linked with foreign keys. EventHub uses InnoDB transactions (all-or-nothing changes) with the default isolation level REPEATABLE READ.

Docker and Docker Compose

Docker runs programs in **containers**: small, isolated boxes that contain everything the program needs. `docker-compose.yml` starts two containers with one command: MySQL (port 3306) and Mailpit (a fake email server with a web inbox at port 8025).

Real-life example: A container is like a lunch box: the food (MySQL) comes with its own plate and spoon, so it works the same on every table (laptop).

Mailpit

In development, emails must not go to real people. Mailpit accepts every email on port 1025 and shows it in a web inbox at <http://localhost:8025>.

3.5 Quality tools

Tool	What it does
JUnit 5	Runs Java tests (<code>@Test</code>)
Mockito	Creates fake objects ("mocks") so one class can be tested alone
AssertJ	Readable checks: <code>assertThat(x).isEqualTo(5)</code>
MockMvc	Calls controllers like a browser would, without a real server
Testcontainers	Starts a real, fresh MySQL 8.4 in Docker for the tests
JaCoCo	Measures which lines the tests ran (coverage report)
Vitest + React Testing Library	Runs React component tests in a fake browser (jsdom)
oxlint	Finds mistakes in JavaScript, e.g. a variable that does not exist
GitHub Actions	Runs all of the above on every push
Postman	Manual and scripted API calls (collection with 43 checks)

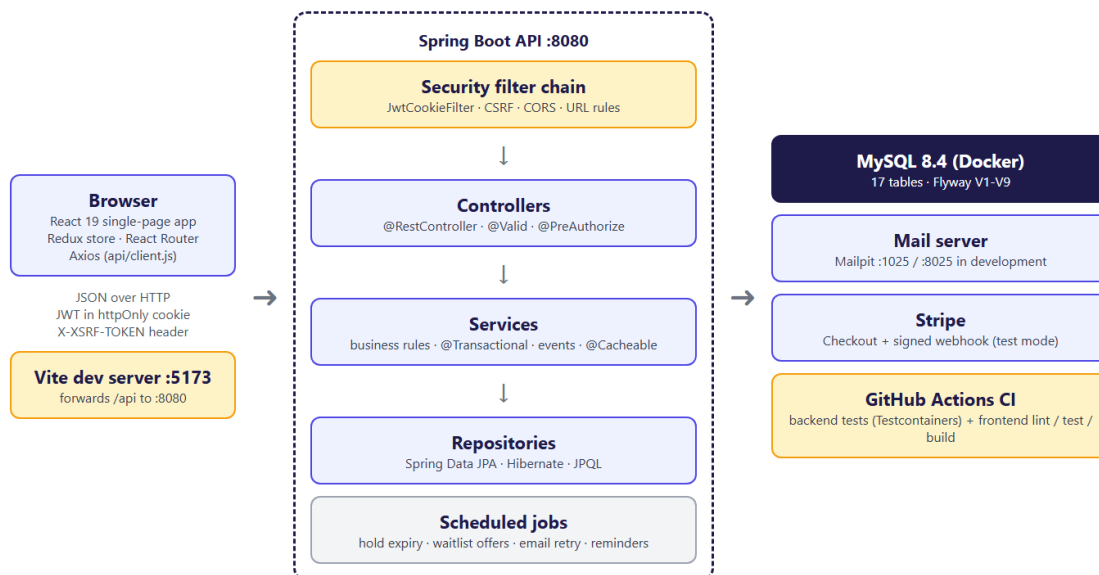
4. System architecture

4.1 The three tiers

EventHub follows the classic **three-tier architecture**:

1. **Presentation tier** - the React app in the browser. It only shows data and sends the user's actions to the server. It never decides important things (such as "is there a seat left?") on its own.
2. **Application tier** - the Spring Boot API. It owns every business rule, checks security, and talks to the database, the mail server and Stripe.
3. **Data tier** - MySQL. It stores everything and protects the data with keys and constraints.

EventHub architecture



EventHub architecture: browser, Spring Boot layers, MySQL and outside services

Real-life example: A restaurant: the waiter (React) takes your order and brings the food, but never cooks. The kitchen (Spring Boot) cooks by the recipes (business rules) and checks your table number (security). The store room (MySQL) keeps the ingredients safe and counted.

4.2 Layers inside the backend

The backend itself has four layers. Each layer only talks to the layer directly below it.

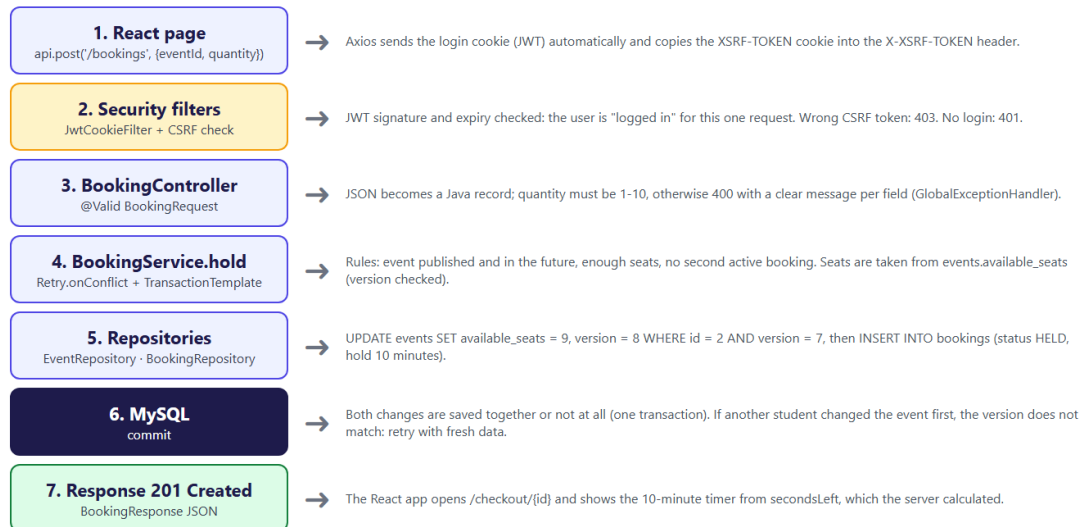
Layer	Job	Example class
Security filters	Who is calling? Are they allowed to call this URL?	<code>JwtCookieFilter</code> , <code>SecurityConfig</code>
Controller	Receive the HTTP request, validate the input, return JSON	<code>BookingController</code>
Service	Apply the business rules inside a transaction	<code>BookingService</code>
Repository	Read and write the database	<code>BookingRepository</code>

Why layers? Each piece can be understood and tested on its own. A rule such as "seats cannot go below zero" lives in exactly one place (the service), no matter whether the request came from the website, Postman or a test.

4.3 The journey of one request

The diagram below follows one real request from the moment Ravi presses "Book now" until the checkout page appears.

The journey of one request: Ravi presses "Book now"



One request from the React page to MySQL and back

This is the controller method that receives it. Notice how small it is: the controller only translates HTTP into a service call.

BookingController.java (backend/src/main/java/com/eventhub/booking/BookingController.java)

```

/** Hold seats: 201 with the booking (HELD with a timer, or CONFIRMED for a free event). */
@PostMapping
public ResponseEntity<BookingResponse> hold(@AuthenticationPrincipal CurrentUser user,
    @Valid @RequestBody BookingRequest request) {
    BookingResponse booking = service.hold(user.id(), request.eventId(), request.quantity());
    return ResponseEntity.created(URI.create("/api/bookings/" + booking.id())).body(booking);
}

```

4.4 Package structure (backend)

The backend code is organised **by feature**, not by technical type. Everything about bookings is in `com.eventhub.booking`: the entity, the repository, the service, the controller and the DTOs (data transfer objects, the shapes of the JSON).

Package	What it contains
<code>analytics</code>	Dashboard numbers (JPQL GROUP BY queries) and the 60-second cache
<code>audit</code>	The audit log: who did what to which event
<code>auth</code>	Registration, login, JWT, cookies, CSRF, club permissions, login lock, demo accounts
<code>booking</code>	Bookings, seat holds, confirmation, cancellation, expiry job, retry helper
<code>certificate</code>	Certificate numbers, PDF drawing, public verification

Package	What it contains
club	Clubs and club members (with their club role)
common	Shared exceptions and the global error handler; page responses
config	Security, CORS, cache and OpenAPI configuration
event	Events, search specifications, the status state machine, organizer and admin screens
feedback	Ratings and anonymous comments
gate	QR images, check-in and the live counter
health	/api/health
notification	In-app notifications, emails (after commit, with retry) and reminders
payment	The payment gateway interface, Stripe and the fake gateway, webhooks
tag	Event tags
user	Users, roles, profiles
waitlist	Waitlist entries and seat offers

4.5 Folder structure of the repository

eventhub/	
backend/	Spring Boot application (Maven)
pom.xml	dependencies and build plugins (JaCoCo, surefire)
mvnw, mvnw.cmd	Maven wrapper: no Maven installation needed
src/	
main/java/com/eventhub/<feature>/	code, one package per feature
main/resources/application.yml	settings (dev and prod profiles)
main/resources/db/migration/	Flyway SQL files V1..V9
test/java/com/eventhub/	133 tests
frontend/	React application (Vite)
package.json	dependencies and scripts (dev, build, lint, test)
vite.config.js	dev server, /api proxy, test settings
src/	
api/	one file per backend area (axios calls)
components/	reusable pieces: ui, layout, events, scanner ...
pages/	one file per screen (+ organizer/, admin/)
store/	Redux slices
hooks/	useAsync, useForm, useCountdown, useTheme
utils/	formatting and validation helpers
docker-compose.yml	MySQL 8.4 + Mailpit
.env.example	template for your secrets (.env is never committed)
postman/	Postman collection (25 requests, 43 checks)
tools/	Edge demo tour scripts
docs/sketches/	the original page sketches
Showcase/	presentation, this guide, screenshots
.github/workflows/ci.yml	GitHub Actions: tests on every push

4.6 Important design decisions

Decision	Why	Alternative considered
JWT in an httpOnly cookie (not localStorage)	JavaScript cannot read it, so an injected script cannot steal the login	Bearer token in localStorage (easier, but stealable)
Optimistic locking (@Version) for seats	No waiting locks, safe under heavy load, simple retry	Pessimistic SELECT ... FOR UPDATE (locks rows, slower)
One seat counter: events.available_seats	A single number is easy to keep correct; holds already take seats	Counting bookings on every request (slow, race-prone)

Decision	Why	Alternative considered
Server decides time (hold timers, secondsLeft)	A phone clock can be wrong or changed	Timer only in the browser
Payment behind an interface (PaymentGateway)	The same booking code works with Stripe and with the test page	Stripe calls inside BookingService
Emails sent after commit	Never email about a change that was rolled back	Sending inside the transaction
Club roles read from the database on every request	Removing an organizer works immediately	Club roles inside the JWT (valid until it expires)
Flyway migrations + ddl-auto: validate	Every database is identical; mistakes are caught at start-up	Hibernate creating tables itself
Package by feature	Everything for one feature is together	Package by layer (controllers/, services/ ...)

5. Setup and installation

This chapter takes you from an empty laptop to a running EventHub. The commands are for Windows (Command Prompt or Git Bash); on macOS or Linux use `./mvnw` instead of `mvnw` and `cp` instead of `copy`.

5.1 What you need to install

Tool	Version	Why	Check with
Git	any recent	download the code	<code>git --version</code>
Java JDK	25 (Temurin recommended)	run the backend	<code>java -version</code>
Node.js	24 LTS	run the frontend	<code>node -v</code>
Docker Desktop	recent	MySQL and Mailpit in containers; also needed by the tests	<code>docker --version</code>
IntelliJ IDEA (optional)	Community or Ultimate	edit and run the backend	-
VS Code (optional)	any	edit the frontend	-
Postman (optional)	any	try the API by hand	-

Note: Maven does **not** need to be installed: the project contains the Maven wrapper (`mvnw`), which downloads the right Maven version the first time.

5.2 Step 1 - get the code

```
git clone https://github.com/madhavansuresh000-sys/eventhub.git
cd eventhub
```

5.3 Step 2 - create your .env file

Secrets (database passwords, the JWT key, the demo password, Stripe keys) are **never** stored in the code. They live in a file called `.env` in the project folder, which is listed in `.gitignore` so it can never be pushed to GitHub. A template is provided:

.env.example (.env.example, lines 1-18)

```
# Copy this file to .env and choose your own passwords.
# .env is in .gitignore, so your real passwords never go to GitHub.
MYSQL_USER=eventhub
MYSQL_PASSWORD=change-me
MYSQL_ROOT_PASSWORD=change-me-root

# Phase 5 (security)
# Key that signs login tokens. Make one with: python -c "import secrets;
print(secrets.token_hex(32))"
JWT_SECRET=change-me-use-64-random-hex-characters
# Password for the demo accounts created in dev (leave empty to create none)
DEMO_PASSWORD=change-me

# Phase 6 (payments). Leave empty to use the built-in test payment page.
# Stripe dashboard (Test mode) -> Developers -> API keys -> Secret key (starts with sk_test_)
STRIPE_SECRET_KEY=
# From "stripe listen" (Stripe CLI) or Developers -> Webhooks; starts with whsec_
```

```
STRIPE_WEBHOOK_SECRET=
```

```
copy .env.example .env
```

Now open `.env` and replace every `change-me` with your own values:

1. `MYSQL_PASSWORD` and `MYSQL_ROOT_PASSWORD` - any passwords you like.
2. `JWT_SECRET` - at least 32 random characters. Generate one with `python -c "import secrets; print(secrets.token_hex(32))"`.
3. `DEMO_PASSWORD` - the password of the five demo accounts (leave empty to create no demo accounts).
4. `STRIPE_SECRET_KEY` and `STRIPE_WEBHOOK_SECRET` - leave empty. EventHub then uses its built-in test payment page.

Real-life example: The `.env` file is like the key of your house: you make copies for your own family (your laptop), but you never pin it on the notice board (GitHub). The `.env.example` file is the empty key shape that shows everyone what kind of key is needed.

5.4 Step 3 - start MySQL and Mailpit

Start Docker Desktop and wait until it says "Engine running". Then:

```
docker compose up -d
docker compose ps
```

You should see two containers, `eventhub-mysql` (healthy) and `eventhub-mailpit`. The file that describes them:

docker-compose.yml (docker-compose.yml, lines 1-34)

```
# EventHub local services – start with: docker compose up -d
# Passwords come from the .env file (never pushed to GitHub). See .env.example.
services:
  mysql:
    image: mysql:8.4
    container_name: eventhub-mysql
    restart: unless-stopped
    environment:
      MYSQL_DATABASE: eventhub
      MYSQL_USER: ${MYSQL_USER}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      TZ: Asia/Kolkata
    ports:
      - "3306:3306"
    volumes:
      - mysql-data:/var/lib/mysql # keeps your data when the container stops
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root", "-p${MYSQL_ROOT_PASSWORD}"]
      interval: 10s
      timeout: 5s
      retries: 10

  mailpit:
    image: axllent/mailpit:latest
    container_name: eventhub-mailpit
    restart: unless-stopped
    ports:
      - "1025:1025" # SMTP: Spring Boot sends emails here
```

```
- "8025:8025" # Web inbox: open http://localhost:8025

volumes:
  mysql-data:
```

- MySQL listens on `localhost:3306`. Its data is kept in the Docker volume `mysql-data`, so it survives restarts.
- Mailpit shows every email EventHub sends at `http://localhost:8025`.

5.5 Step 4 - start the backend

Option A, IntelliJ: open the `backend` folder, wait until Maven has downloaded the dependencies, then run the class `EventhubApplication` (green triangle).

Option B, command line:

```
cd backend
mvnw spring-boot:run
```

What happens on start-up:

1. Spring Boot reads `application.yml`. The default profile is `dev`, which also imports `../.env`.
2. Flyway creates or updates the tables (V1 to V9). The first run also inserts sample data (V2: 5 clubs, 10 tags, 20 events).
3. Hibernate checks that the Java entities match the tables (`ddl-auto: validate`).
4. `DevDataSeeder` creates the five demo accounts if they do not exist yet.
5. The server listens on port 8080.

Check it: open `http://localhost:8080/api/health` - it answers `{"app":"eventhub","status":"UP"}`.

The settings file, with a comment on every line:

application.yml (backend/src/main/resources/application.yml, lines 1-70)

```
# ----- Common settings (all profiles) -----
spring:
  application:
    name: eventhub
  profiles:
    default: dev # when no profile is chosen, use "dev"
  jpa:
    hibernate:
      ddl-auto: validate # Flyway creates tables; Hibernate only checks them
    open-in-view: false
  flyway:
    enabled: true
  cache:
    # Phase 7 step 7: dashboard numbers are kept in memory for 60 s (see
    analytics/AnalyticsService)
    cache-names: analytics
    caffeine:
      spec: maximumSize=500,expireAfterWrite=60s

management:
  endpoints:
    web:
      exposure:
        include: health,info
```

```

endpoint:
  health:
    # everyone sees only "UP"/"DOWN"; the admin also sees database, mail and disk details
    show-details: when-authorized
    roles: ADMIN

springdoc:
  swagger-ui:
    path: /swagger-ui.html

# Websites allowed to call the API from a browser (CorsConfig). Comma-separated.
app:
  cors:
    allowed-origins: ${CORS_ALLOWED_ORIGINS:http://localhost:5173,http://localhost:4173}
  jwt:
    expiry: 8h # how long a login lasts
  cookie:
    secure: false # true in prod (cookie only over HTTPS)
  login:
    max-failures: 5 # wrong passwords before the account+address is locked for a while
    lock-time: 15m
  booking:
    hold-time: 10m # how long seats are kept for a student who has not paid yet
    expiry-check: 60s # how often the job frees expired holds
  waitlist:
    offer-time: 30m # how long freed seats are kept for the next student on the
waitlist
  expiry-check: 60s # how often the job passes unanswered offers to the next student
  mail:
    from: ${MAIL_FROM:EventHub <no-reply@eventhub.test>}
    retry-every: 60s # emails that failed (mail server down) are tried again, up to 5
times
  reminders:
    cron: "0 0 18 * * *" # every day at 18:00: reminders for tomorrow's events
    zone: Asia/Kolkata
  scheduling:
    enabled: true # the every-minute job (tests switch it off)
# where the React app runs: Stripe sends the student back here after paying
frontend-url: ${FRONTEND_URL:http://localhost:5173}
stripe:
  # from .env; both empty = the built-in test payment page is used (dev only)
  secret-key: ${STRIPE_SECRET_KEY:}
  webhook-secret: ${STRIPE_WEBHOOK_SECRET:}

---
# ----- dev: your laptop (MySQL + Mailpit in Docker) -----
spring:
  config:
    activate:

```

5.6 Step 5 - start the frontend

```

cd frontend
npm install
npm run dev

```

Open <http://localhost:5173>. The footer shows "Backend: UP" when the React app can reach Spring Boot. Log in with one of the demo accounts from section 1.7.

5.7 Step 6 - run the tests

Docker Desktop must be running, because the backend tests start their own MySQL container.

```

cd backend
mvnw test

```

After about two minutes Maven prints **Tests run: 133, Failures: 0, Errors: 0**. The coverage report is written to **backend/target/site/jacoco/index.html**.

```
cd frontend
npm run lint
npm test
```

npm test prints **Tests 13 passed**.

Note: Stop the running backend before **mvnw test** on Windows: both use the **backend/target** folder, and Windows may lock files that are in use.

5.8 Step 7 - try the API with Postman

1. Open Postman → Import → choose **postman/EventHub.postman_collection.json**.
2. Start the backend.
3. Click the collection → Run. The 25 requests run in order and 43 checks turn green.

Or from the command line, without installing Postman:

```
npx -y newman@6 run postman/EventHub.postman_collection.json
```

5.9 Useful addresses (development)

Address	What
http://localhost:5173	The React app
http://localhost:8080/api/health	Backend health check (public)
http://localhost:8080/swagger-ui.html	API documentation (admin only: log in as the admin in the React app first)
http://localhost:8080/actuator/health	Health with details for the admin
http://localhost:8025	Mailpit inbox: every email EventHub sent
localhost:3306	MySQL (database eventhub)

5.10 Stopping everything

```
Ctrl + C in the backend and frontend windows
docker compose down      (stops the containers, keeps the data)
docker compose down -v   (also deletes the database data - be careful)
```

5.11 The demo tour

tools/edge-tour.mjs opens a separate Microsoft Edge window and walks through the whole application in 28 steps, with a yellow explanation box at each step: visitor, booking and payment, QR ticket, certificates, gate scanning, public verification, organizer, admin approval, analytics and security. It also saves a screenshot of each step. Run it with the backend and frontend running:

```
node tools/edge-tour.mjs tour-shots
```

Every run creates one new event ("Kotlin for Beginners HH:MM") and one new booking for Ravi in the development database.

6. Database design

6.1 Overview

EventHub stores its data in **17 MySQL tables**. They are never created or changed by hand: every change is a numbered SQL file (a **migration**) in `backend/src/main/resources/db/migration`, run automatically by Flyway when the backend starts.

Migration	Phase	What it adds
V1__create_core_tables.sql	2	roles, users, user_roles, profiles, clubs, club_members, events, tags, event_tags
V2__seed_sample_data.sql	2	5 clubs, 10 tags, 20 sample events
V3__create_audit_log.sql	5	audit_log
V4__create_bookings_and_payments.sql	6	bookings, payments, processed_payment_events
V5__create_waitlist.sql	7	waitlist_entries
V6__create_notifications.sql	7	notifications, bookings.reminder_sent_at
V7__add_check_in.sql	7	bookings.checked_in_at / checked_in_by
V8__create_certificates.sql	7	certificates
V9__create_feedback.sql	7	feedback

Main tables and how they are linked



Read it like this: a **club** has many **events**; a **user** has many **bookings**; each booking belongs to one event, may have **payments** and - after the gate scan and the end of the event - one **certificate** and one **feedback**.

The main tables and how they are linked

Real-life example: A migration is like a numbered page in a building's renovation log: "1 - build the walls, 2 - add furniture, 3 - add a security camera". Any new building (a laptop, the test database, the server) follows the same pages in the same order and ends up identical.

6.2 Rules that protect the data

The database itself refuses wrong data, even if a bug in Java tried to save it:

- **Primary keys** (`id BIGINT AUTO_INCREMENT`) identify every row.
- **Foreign keys** link rows: a booking must point to an existing user and event.

- **UNIQUE** keys stop duplicates: one email per user, one ticket code per booking, one certificate and one rating per booking, one processed webhook message per id.
- **CHECK** constraints limit values: booking status must be HELD, CONFIRMED, CANCELLED or EXPIRED; quantity 1-10; rating 1-5.
- **NOT NULL** columns must always have a value.
- **Indexes** make the frequent searches fast, for example `idx_bookings_hold (status, hold_expires_at)` for the job that frees old holds.

Note: Status columns are `VARCHAR + CHECK` instead of MySQL `ENUM`. Adding a new status then only needs a new CHECK, and the values read naturally in Java (`@Enumerated(EnumType.STRING)`).

6.3 Optimistic locking columns

`events.version` and `bookings.version` are **version numbers**. Hibernate adds `WHERE version = ?` to every UPDATE and increases the number. If two transactions read the same row and both try to save, the second UPDATE finds no row with the old version and fails - so the second one must read again and retry. Chapter 7 shows the code.

6.4 Every table, column by column

The tables below are generated automatically from the migration files, so they always match the real database. The "Meaning / rule" column shows the comment written next to each column in the SQL file, plus the rules (required, unique, default).

Table roles (V1)

Column	Type	Meaning / rule
id	BIGINT	primary key
name	VARCHAR(30)	STUDENT, ADMIN - required, unique

Table users (V1)

Column	Type	Meaning / rule
id	BIGINT	primary key
full_name	VARCHAR(100)	required
email	VARCHAR(150)	required, unique
password_hash	VARCHAR(100)	required
enabled	BOOLEAN	required, default TRUE
created_at	DATETIME(6)	required, default CURRENT_TIMESTAMP(6)

Table user_roles (V1)

Column	Type	Meaning / rule
user_id	BIGINT	required
role_id	BIGINT	required

Links to other tables: [user_id](#) → [users](#), [role_id](#) → [roles](#).

Table profiles (V1)

Column	Type	Meaning / rule
id	BIGINT	primary key
user_id	BIGINT	required, unique
department	VARCHAR(100)	
year_of_study	TINYINT	
phone	VARCHAR(20)	
bio	VARCHAR(500)	

Links to other tables: [user_id](#) → [users](#).

Table clubs (V1)

Column	Type	Meaning / rule
id	BIGINT	primary key
name	VARCHAR(100)	required, unique
slug	VARCHAR(100)	used in URLs: /clubs/coding-club - required, unique
description	VARCHAR(1000)	
created_at	DATETIME(6)	required, default CURRENT_TIMESTAMP(6)

Table club_members (V1)

Column	Type	Meaning / rule
id	BIGINT	primary key
club_id	BIGINT	required
user_id	BIGINT	required
club_role	VARCHAR(20)	required
joined_at	DATETIME(6)	required, default CURRENT_TIMESTAMP(6)

Links to other tables: [club_id](#) → [clubs](#), [user_id](#) → [users](#).

Table events (V1)

Column	Type	Meaning / rule
id	BIGINT	primary key
club_id	BIGINT	required
created_by	BIGINT	organizer (NULL for seed data)

Column	Type	Meaning / rule
title	VARCHAR(150)	required
description	TEXT	
venue	VARCHAR(150)	required
start_time	DATETIME(6)	required
end_time	DATETIME(6)	required
total_seats	INT	required
available_seats	INT	required
price	DECIMAL(10,2)	required, default 0.00
status	VARCHAR(20)	required, default 'DRAFT'
review_note	VARCHAR(500)	admin's reason when sending back to DRAFT
version	INT	optimistic locking (no overselling) - required, default 0
created_at	DATETIME(6)	required, default CURRENT_TIMESTAMP(6)
updated_at	DATETIME(6)	required, default CURRENT_TIMESTAMP(6)

Links to other tables: [club_id](#) → [clubs](#), [created_by](#) → [users](#).

Table tags (V1)

Column	Type	Meaning / rule
id	BIGINT	primary key
name	VARCHAR(50)	lower-case: tech, music, sports ... - required, unique

Table event_tags (V1)

Column	Type	Meaning / rule
event_id	BIGINT	required
tag_id	BIGINT	required

Links to other tables: [event_id](#) → [events](#), [tag_id](#) → [tags](#).

Table audit_log (V3)

Column	Type	Meaning / rule
id	BIGINT	primary key
user_id	BIGINT	
user_name	VARCHAR(100)	
user_email	VARCHAR(150)	
action	VARCHAR(20)	required
event_id	BIGINT	
event_title	VARCHAR(150)	required
details	VARCHAR(500)	
created_at	DATETIME(6)	required, default CURRENT_TIMESTAMP(6)

Links to other tables: [user_id](#) → [users](#), [event_id](#) → [events](#).

Table bookings (V4)

Column	Type	Meaning / rule
id	BIGINT	primary key
user_id	BIGINT	required
event_id	BIGINT	required
quantity	INT	required
status	VARCHAR(20)	required
amount	DECIMAL(10,2)	price x quantity when booked (the price may change later) - required
ticket_code	VARCHAR(40)	goes into the QR code (Phase 7 gate scan) - required, unique
hold_expires_at	DATETIME(6)	only for HELD bookings
created_at	DATETIME(6)	required, default CURRENT_TIMESTAMP(6)
confirmed_at	DATETIME(6)	
cancelled_at	DATETIME(6)	
version	INT	optimistic locking (webhook vs expiry job) - required, default 0
reminder_sent_at	DATETIME(6)	(added in V6)
checked_in_at	DATETIME(6)	(added in V7)
checked_in_by	BIGINT	(added in V7)

Links to other tables: [user_id](#) → [users](#), [event_id](#) → [events](#).

Table payments (V4)

Column	Type	Meaning / rule
id	BIGINT	primary key
booking_id	BIGINT	required
provider	VARCHAR(20)	STRIPE, or FAKE (dev test page) - required
session_id	VARCHAR(255)	the payment provider's checkout session id - required, unique
status	VARCHAR(20)	required
amount	DECIMAL(10,2)	required
currency	VARCHAR(3)	required
created_at	DATETIME(6)	required, default CURRENT_TIMESTAMP(6)
paid_at	DATETIME(6)	
refunded_at	DATETIME(6)	

Links to other tables: [booking_id](#) → [bookings](#).

Table processed_payment_events (V4)

Column	Type	Meaning / rule
event_id	VARCHAR(255)	primary key
received_at	DATETIME(6)	required, default CURRENT_TIMESTAMP(6)

Table waitlist_entries (V5)

Column	Type	Meaning / rule
id	BIGINT	primary key
user_id	BIGINT	required
event_id	BIGINT	required
quantity	INT	required
status	VARCHAR(20)	required
offer_expires_at	DATETIME(6)	only while OFFERED
booking_id	BIGINT	the booking made when the offer was accepted
created_at	DATETIME(6)	required, default CURRENT_TIMESTAMP(6)
offered_at	DATETIME(6)	
closed_at	DATETIME(6)	when it became BOOKED / EXPIRED / LEFT
version	INT	optimistic locking (accept vs expiry job) - required, default 0
open_flag	TINYINT	

Links to other tables: [user_id](#) → [users](#), [event_id](#) → [events](#), [booking_id](#) → [bookings](#).

Table notifications (V6)

Column	Type	Meaning / rule
id	BIGINT	primary key
user_id	BIGINT	required
kind	VARCHAR(30)	required
title	VARCHAR(200)	required
body	VARCHAR(1000)	required
link	VARCHAR(300)	page in the React app, e.g. /waitlist
created_at	DATETIME(6)	required, default CURRENT_TIMESTAMP(6)
read_at	DATETIME(6)	NULL = unread (counts on the bell)
email_status	VARCHAR(10)	NONE (bell only), PENDING, SENT, FAILED - required
email_attempts	INT	required, default 0
emailed_at	DATETIME(6)	

Links to other tables: [user_id](#) → [users](#).

Table certificates (V8)

Column	Type	Meaning / rule
id	BIGINT	primary key
number	VARCHAR(30)	required, unique
booking_id	BIGINT	required, unique
user_id	BIGINT	required
event_id	BIGINT	required
holder_name	VARCHAR(100)	required
issued_at	DATETIME(6)	required, default CURRENT_TIMESTAMP(6)

Links to other tables: [booking_id](#) → [bookings](#), [user_id](#) → [users](#), [event_id](#) → [events](#).

Table feedback (V9)

Column	Type	Meaning / rule
id	BIGINT	primary key
booking_id	BIGINT	required, unique
event_id	BIGINT	required
user_id	BIGINT	required
rating	TINYINT	required
comment	VARCHAR(1000)	
created_at	DATETIME(6)	required, default CURRENT_TIMESTAMP(6)
updated_at	DATETIME(6)	

Links to other tables: [booking_id](#) → [bookings](#), [event_id](#) → [events](#), [user_id](#) → [users](#).

6.5 Useful SQL queries (with examples)

You can run these in MySQL Workbench, IntelliJ's database tool, or `docker exec -it eventhub-mysql mysql -u eventhub -p eventhub`.

Which events are almost full?

```
SELECT id, title, total_seats, available_seats,
       ROUND(100 * (total_seats - available_seats) / total_seats) AS percent_full
FROM events
WHERE status = 'PUBLISHED' AND start_time > NOW()
ORDER BY percent_full DESC
LIMIT 5;
```

Ravi's tickets with the event name

```
SELECT b.id, e.title, b.quantity, b.status, b.ticket_code, b.checked_in_at
FROM bookings b
JOIN events e ON e.id = b.event_id
JOIN users u ON u.id = b.user_id
WHERE u.email = 'ravi@eventhub.test'
ORDER BY b.created_at DESC;
```

Check-in rate per event (the same idea as the analytics dashboard)

```
SELECT e.title,
       SUM(b.quantity) AS sold,
       SUM(CASE WHEN b.checked_in_at IS NOT NULL THEN b.quantity ELSE 0 END) AS came
```

```
FROM events e
JOIN bookings b ON b.event_id = e.id AND b.status = 'CONFIRMED'
GROUP BY e.id, e.title;
```

Who is on the waitlist of an event, in order

```
SELECT w.id, u.full_name, w.quantity, w.status, w.offer_expires_at
FROM waitlist_entries w
JOIN users u ON u.id = w.user_id
WHERE w.event_id = 16
ORDER BY w.id;
```

The audit trail of one event

```
SELECT created_at, action, user_email, event_title, details
FROM audit_log
WHERE event_id = 35
ORDER BY created_at;
```

Which Flyway migrations ran

```
SELECT version, description, installed_on, success FROM flyway_schema_history;
```

6.6 How Java classes map to tables (JPA)

An **entity** is a Java class that stands for one table; each field stands for one column. Here is the **Event** entity (shortened by the generator to its fields):

Event.java (backend/src/main/java/com/eventhub/event/Event.java)

```
/** A college event, e.g. "Tech Fest 2026". */
@Entity
@Table(name = "events")
@Getter
@Setter
@NoArgsConstructor
public class Event {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    /** Many events belong to one club. */
    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "club_id", nullable = false)
    private Club club;

    /** The organizer who created it (empty for sample data). */
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "created_by")
    private User createdBy;

    @Column(nullable = false, length = 150)
    private String title;

    @Column(columnDefinition = "TEXT")
    private String description;

    @Column(nullable = false, length = 150)
    private String venue;

    @Column(name = "start_time", nullable = false)
    private LocalDateTime startTime;

    @Column(name = "end_time", nullable = false)
    private LocalDateTime endTime;
```

```

@Column(name = "total_seats", nullable = false)
private int totalSeats;

@Column(name = "available_seats", nullable = false)
private int availableSeats;

@Column(nullable = false, precision = 10, scale = 2)
private BigDecimal price = BigDecimal.ZERO;

@Enumerated(EnumType.STRING)
@Column(nullable = false, length = 20)
private EventStatus status = EventStatus.DRAFT;

/** The admin's reason when an event is sent back to DRAFT. */
@Column(name = "review_note", length = 500)
private String reviewNote;

/** Optimistic locking: stops two people saving over each other (and overselling). */
@Version
@Column(nullable = false)
private int version;

/** Many-to-Many: tags via the event_tags table (loaded 50 events at a time). */
@ManyToMany
@BatchSize(size = 50)
@JoinTable(name = "event_tags",
    joinColumns = @JoinColumn(name = "event_id"),
    inverseJoinColumns = @JoinColumn(name = "tag_id"))
private Set<Tag> tags = new HashSet<>();

@CreationTimestamp
@Column(name = "created_at", nullable = false, updatable = false)
private LocalDateTime createdAt;

@UpdateTimestamp
@Column(name = "updated_at", nullable = false)
private LocalDateTime updatedAt;
}

```

Points to notice:

- `@ManyToOne(fetch = FetchType.LAZY)` - an event belongs to one club, but the club is only loaded from the database when the code actually uses it. This avoids loading data nobody needs.
- `@ManyToMany + @JoinTable(name = "event_tags")` - the link table between events and tags.
- `@Version` - the optimistic-locking column described in 6.3.
- `@Enumerated(EnumType.STRING)` - the status is stored as text ('PUBLISHED'), not as a number.
- `@BatchSize(size = 50)` - when a page shows 12 events, their tags are loaded in one extra query instead of 12 (this avoids the "N+1 queries" problem).

6.7 Example: what the tables look like after one booking

Ravi books 1 ticket for the 24-Hour Hackathon (event 2, ₹150) and pays.

Table	Row after the booking
events	id 2 · available_seats 10 → 9 · version +1
bookings	new row · status HELD → CONFIRMED · amount 150.00 · ticket_code EVH-97Z38-F9Y5Q · hold_expires_at now+10 min · confirmed_at set
payments	new row · provider FAKE (or STRIPE) · session_id fake_cs_... · status PENDING → PAID
processed_payment_events	new row with the payment notice id (so the same notice is never processed twice)
notifications	new row "Booking confirmed" for Ravi · email PENDING → SENT

After Priya scans the ticket at the gate: `bookings.checked_in_at` and `checked_in_by` are filled. After the event, when Ravi opens his certificates page, a row is added to `certificates` with a random number like EH-2026-BEQR4Y79.

7. Backend code explained

This chapter walks through the backend module by module. For each one you get: what it does, the most important real code, an explanation line by line, a worked example, and what could go wrong without it.

7.1 The application entry point

Every Spring Boot application starts from one class with a **main** method:

EventhubApplication.java (backend/src/main/java/com/eventhub/EventhubApplication.java)

```
@SpringBootApplication
public class EventhubApplication {

    public static void main(String[] args) {
        SpringApplication.run(EventhubApplication.class, args);
    }

}
```

@SpringBootApplication tells Spring to scan the package **com.eventhub** and everything below it, create a bean for every class marked **@Service**, **@RestController**, **@Component**, **@Configuration** or **@Repository**, and start the embedded web server.

7.2 Errors that make sense: GlobalExceptionHandler

When something goes wrong, the API always answers in the same shape: the standard **Problem Details** format (RFC 9457) with an extra **errors** map for field problems. The React app reads **detail** for the message and **errors** for the field messages.

Situation	Exception	HTTP status
A field is invalid (@Valid)	MethodArgumentNotValidException	400 Bad Request + errors map
Wrong input such as end before start	BadRequestException	400
Not logged in	(security)	401 Unauthorized
Logged in but not allowed	AccessDeniedException	403 Forbidden
The thing does not exist	ResourceNotFoundException	404 Not Found
A business rule says no (sold out, not your turn)	BusinessRuleException	409 Conflict
Two people changed the same row	ObjectOptimisticLockingFailureException	409
Too many wrong passwords	TooManyLoginAttemptsException	429 + Retry-After
Anything unexpected	Exception	500 (details only in the server log)

GlobalExceptionHandler.java (backend/src/main/java/com/eventhub/common/GlobalExceptionHandler.java)

```
@ExceptionHandler(BusinessRuleException.class)
ProblemDetail handleBusinessRule(BusinessRuleException ex) {
    return ProblemDetail.forStatusAndDetail(HttpStatus.CONFLICT, ex.getMessage());
}
```

Example answer when Ravi tries to book a sold-out event:

```

HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{
  "type": "about:blank",
  "title": "Conflict",
  "status": 409,
  "detail": "Dance Night is sold out. You can join the waitlist.",
  "instance": "/api/bookings"
}

```

Note: A 500 error never shows the internal message to the user (it could reveal table names or code). The real exception is written to the server log instead.

7.3 Events: search, rules and the approval workflow

The status state machine

The allowed moves are written once, in the enum itself. Every service method asks `canMoveTo` before changing the status, so an illegal move (for example DRAFT straight to PUBLISHED) is impossible.

EventStatus.java (backend/src/main/java/com/eventhub/event/EventStatus.java)

```

/**
 * Approval workflow (a small "state machine"):
 *
 * <pre>
 *   DRAFT --submit--> PENDING_APPROVAL --approve--> PUBLISHED
 *   ^
 *   +-----reject-----+ (with a reason)
 * </pre>
 */
public enum EventStatus {

    DRAFT, PENDING_APPROVAL, PUBLISHED;

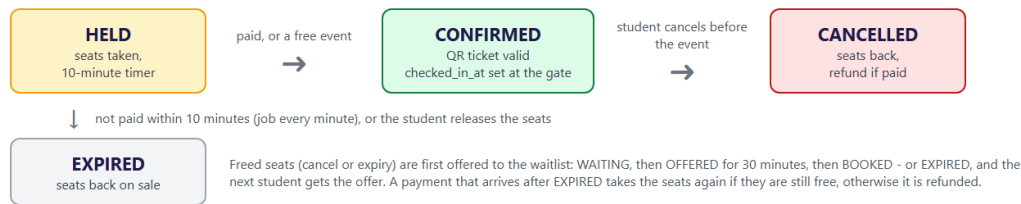
    /** The only moves allowed from this status. */
    public Set<EventStatus> nextAllowed() {
        return switch (this) {
            case DRAFT -> Set.of(PENDING_APPROVAL);
            case PENDING_APPROVAL -> Set.of(PUBLISHED, DRAFT);
            case PUBLISHED -> Set.of();
        };
    }

    public boolean canMoveTo(EventStatus target) {
        return nextAllowed().contains(target);
    }
}

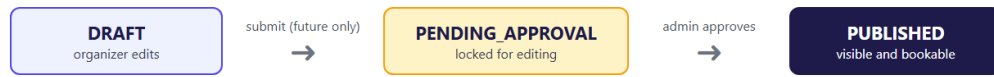
```

Real-life example: Like a job application: "draft" → "submitted" → "accepted". You cannot jump from "draft" to "accepted", and once it is submitted you cannot secretly edit it; you can only get it back (rejected) and submit again.

Booking states (bookings.status)



Event states (events.status)



Event and booking states

Searching with Specifications

The search page can combine up to five filters (text, tag, club, from, to). Instead of writing 32 different queries, EventHub builds the WHERE clause from small pieces called **Specifications** and joins only the ones that were filled in:

EventSpecifications.java (backend/src/main/java/com/eventhub/event/EventSpecifications.java)

```

/** Builds the full search from the filter; blank options are skipped. */
public static Specification<Event> publishedMatching(EventFilter f) {
    Specification<Event> spec = hasStatus(EventStatus.PUBLISHED);
    if (hasText(f.q())) {
        spec = spec.and(textContains(f.q().trim()));
    }
    if (hasText(f.tag())) {
        spec = spec.and(hasTag(f.tag().trim().toLowerCase()));
    }
    if (hasText(f.club())) {
        spec = spec.and(inClub(f.club().trim().toLowerCase()));
    }
    if (f.from() != null) {
        spec = spec.and(startsOnOrAfter(f.from()));
    }
    if (f.to() != null) {
        spec = spec.and(startsOnOrBefore(f.to()));
    }
    return spec;
}

```

Example: `GET /api/events?q=robot&tag=robotics&page=0&size=9&sort=date` becomes, roughly:

```

SELECT ... FROM events e
WHERE e.status = 'PUBLISHED'
  AND (lower(e.title) LIKE '%robot%' OR lower(e.description) LIKE '%robot%')
  AND EXISTS (SELECT 1 FROM event_tags et JOIN tags t ... WHERE t.name = 'robotics')
ORDER BY e.start_time ASC
LIMIT 9 OFFSET 0

```

Visitors only ever see **PUBLISHED** events: the status filter is always added by the server, whatever the browser sends.

Creating and updating an event

EventService.java (backend/src/main/java/com/eventhub/event/EventService.java)

```

/**
 * Rules: an event under review cannot be edited, and seats cannot be
 * reduced below the number already booked.
 */
@Transactional

```

```

public EventDetailResponse update(Long id, EventRequest request) {
    Event event = findEvent(id);
    checkTimes(request);

    if (event.getStatus() == EventStatus.PENDING_APPROVAL) {
        throw new BusinessException("Event " + id + " is waiting for approval and cannot
be edited");
    }

    int booked = event.getTotalSeats() - event.getAvailableSeats();
    if (request.totalSeats() < booked) {
        throw new BusinessException(
            "totalSeats cannot be less than the " + booked + " seats already booked");
    }
    event.setAvailableSeats(request.totalSeats() - booked);
    applyRequest(event, request);
    // more seats on a published event? the waitlist gets them first
    waitlistOffers.offerFreeSeats(event, LocalDateTime.now());

    audit.record(AuditAction.UPDATE, event, null);
    return EventMapper.toDetail(event);
}

```

Line by line:

1. `findEvent(id)` loads the event or throws 404.
2. `checkTimes` refuses an end time before the start time (400).
3. An event waiting for approval cannot be changed (409) - the admin must see exactly what was submitted.
4. Seats cannot be reduced below the seats already sold. `booked = totalSeats - availableSeats`.
5. `applyRequest` copies the new values; `availableSeats` moves by the same amount as `totalSeats`.
6. If seats were added, `waitlist.offerFreeSeats` immediately offers them to the waitlist.
7. `audit.record` writes who changed it into the audit log, in the same transaction.

Submit, approve, reject

EventService.java (backend/src/main/java/com/eventhub/event/EventService.java)

```

/** Admin: PENDING_APPROVAL -> PUBLISHED. Students can now see it. */
@Transactional
public EventDetailResponse approve(Long id) {
    Event event = findEvent(id);
    moveTo(event, EventStatus.PUBLISHED, "approve");
    event.setReviewNote(null);
    audit.record(AuditAction.APPROVE, event, null);
    publisher.publishEvent(new EventReviewed(event.getId(), true, null)); // tells the
organizers
    return EventMapper.toDetail(event);
}

```

EventService.java (backend/src/main/java/com/eventhub/event/EventService.java)

```

private void moveTo(Event event, EventStatus target, String action) {
    EventStatus current = event.getStatus();
    if (!current.canMoveTo(target)) {
        throw new BusinessException("Cannot " + action + " event " + event.getId()
+ ": it is " + current + " (allowed next: " + current.nextAllowed() + ")");
    }
    event.setStatus(target);
}

```

`eventsPublisher.publishEvent(new EventReviewed(...))` announces the decision inside the application. The notification module listens and creates "Your event was approved" (or "sent back") for the organizer - the event module does not need to know how notifications work.

7.4 Security: login, JWT cookie, CSRF and permissions

Passwords

Passwords are hashed with **BCrypt** before they are saved. A hash cannot be turned back into the password; at login, BCrypt hashes the typed password again and compares. Each hash contains a random "salt", so two users with the same password still get different hashes.

AuthService.java (backend/src/main/java/com/eventhub/auth/AuthService.java)

```
/**
 * New STUDENT account. The password is stored only as a BCrypt hash: a one-way "fingerprint"
 * with a random salt, so even we cannot see the real password, and two users with the same
 * password get different hashes.
 */
@Transactional
public User register(RegisterRequest request) {
    String email = normalizeEmail(request.email());
    if (users.existsByEmail(email)) {
        throw new BusinessException("An account with this email already exists. Try
logging in.");
    }
    User user = new User();
    user.setFullName(request.fullName().trim());
    user.setEmail(email);
    user.setPasswordHash(passwordEncoder.encode(request.password()));
    user.getRoles().add(role(STUDENT));
    users.save(user);

    Profile profile = new Profile();
    profile.setUser(user);
    profile.setDepartment(blankToNull(request.department()));
    profile.setYearOfStudy(request.yearOfStudy());
    profiles.save(profile);
    user.setProfile(profile);
    return user;
}
```

The login token (JWT)

After a successful login the server creates a **JSON Web Token**: a small signed text with the user id, email, name and global roles, valid for 8 hours. It is signed with HMAC-SHA256 using `JWT_SECRET`, so nobody without the secret can create or change one.

JwtService.java (backend/src/main/java/com/eventhub/auth/JwtService.java)

```
/** A new token for this user, valid for app.jwt.expiry (8 hours). */
public String issue(User user) {
    Instant now = Instant.now();
    JwtClaimsSet claims = JwtClaimsSet.builder()
        .issuer(ISSUER)
        .issuedAt(now)
        .expiresAt(now.plus(expiry))
        .subject(String.valueOf(user.getId()))
        .claim("email", user.getEmail())
        .claim("name", user.getFullName())
        .claim("roles", user.getRoles().stream().map(Role::getName).sorted().toList())
        .build();
    JwsHeader header = JwsHeader.with(MacAlgorithm.HS256).build();
    return encoder.encode(JwtEncoderParameters.from(header, claims)).getTokenValue();
}
```

The token travels in a cookie named **EVENTHUB_TOKEN** with three important flags: **HttpOnly** (JavaScript cannot read it), **SameSite=Lax** (other websites cannot make the browser send it with a form POST) and **Secure** in production (only over HTTPS).

On every request the **JwtCookieFilter** reads the cookie and, if the signature and expiry are fine, tells Spring Security who the user is:

JwtCookieFilter.java (backend/src/main/java/com/eventhub/auth/JwtCookieFilter.java)

```
@Override
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
FilterChain chain)
    throws ServletException, IOException {
    Cookie cookie = WebUtils.getCookie(request, AuthCookies.NAME);
    if (cookie != null && !cookie.getValue().isBlank()) {
        jwtService.read(cookie.getValue()).ifPresent(jwt -> {
            CurrentUser user = new CurrentUser(Long.valueOf(jwt.getSubject()),
                jwt.getClaimAsString("email"), jwt.getClaimAsString("name"));
            List<SimpleGrantedAuthority> authorities = JwtService.roles(jwt).stream()
                .map(role -> new SimpleGrantedAuthority("ROLE_" + role))
                .toList();
            SecurityContext context = SecurityContextHolder.createEmptyContext();
            context.setAuthentication(UsernamePasswordAuthenticationToken.authenticated(user,
null, authorities));
            SecurityContextHolder.setContext(context);
        });
    }
    chain.doFilter(request, response);
}
```

Real-life example: The JWT is like a college ID card with a hologram. The gatekeeper does not phone the office for every student; he checks the hologram (signature) and the expiry date. A card with a changed name has a broken hologram and is refused.

The security rules

SecurityConfig.java (backend/src/main/java/com/eventhub/config/SecurityConfig.java)

```
@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http, JwtService jwtService) throws
Exception {
    http
        // no server-side session: every request carries its own JWT cookie
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        // CSRF for a single-page app: the server sets an XSRF-TOKEN cookie that JavaScript
CAN read;
        // Axios copies it into the X-XSRF-TOKEN header. Another website cannot read our
cookie,
        // so it cannot send a valid header, and its forged POST gets 403.
        // sessionAuthenticationStrategy: Spring normally makes a NEW CSRF token whenever
someone "logs in".
        // Without server sessions it thinks every request with our JWT cookie is a new login,
so the token
        // changed on every request and a second request sent at the same moment could fail.
Keep one token.
        .csrf(csrf -> csrf.spa().sessionAuthenticationStrategy((authentication, request,
response) -> { }))
        // Stripe's server has no CSRF cookie; the Stripe-Signature header proves the
message instead
        .ignoringRequestMatchers("/api/payments/stripe/webhook"))
        .addFilterAfter(new CsrfCookieFilter(), CsrfFilter.class)
        // use the rules from CorsConfig (browser calls from other addresses)
        .cors(Customizer.withDefaults())
        .addFilterBefore(new JwtCookieFilter(jwtService),
UsernamePasswordAuthenticationFilter.class)
        .authorizeHttpRequests(auth -> auth
            // health checks for the hosting platform (Phase 9): only "UP" / "DOWN", no
details

```

```

        .requestMatchers("/api/health", "/actuator/health").permitAll()
        // Phase 8 step 4: the API map (Swagger) and the other Actuator pages tell an
        attacker a lot
        // (every URL, versions, settings), so only the admin may open them. The admin's
        login cookie
        // also works on :8080 (cookies are per host name, not per port): log in on the
        React app first.
        .requestMatchers("/swagger-ui.html", "/swagger-ui/**", "/v3/api-docs/**",
        "/actuator/**").hasRole("ADMIN")
        .requestMatchers("/api/auth/register", "/api/auth/login", "/api/auth/logout",
        "/api/auth/me").permitAll()
        .requestMatchers(HttpMethod.POST, "/api/payments/stripe/webhook").permitAll() //
        checked by its signature
        // public catalogue: anyone can browse events, clubs and tags
        .requestMatchers(HttpMethod.GET, "/api/events/**", "/api/clubs/**",
        "/api/tags").permitAll()
        // anyone (a company checking a resume) can verify a certificate number
        .requestMatchers(HttpMethod.GET, "/api/certificates/verify/").permitAll()
        // admin work
        .requestMatchers("/api/admin/**").hasRole("ADMIN")
        .requestMatchers(HttpMethod.POST, "/api/events/*/approve",
        "/api/events/*/reject").hasRole("ADMIN")
        // everything else (create / edit / submit events, organizer screens ...) needs
        login;
        // the controllers then check "organizer of THIS club" with @PreAuthorize
        .anyRequest().authenticated()
        .exceptionHandling(ex -> ex
        .authenticationEntryPoint(new SecurityProblems().notLoggedIn())
        .accessDeniedHandler(new SecurityProblems().forbidden()));
        return http.build();
    }

```

- **STATELESS:** the server keeps no session; every request carries its own proof (the cookie).
- **CSRF:** the server sets a cookie **XSRF-TOKEN** that JavaScript *can* read. Axios copies it into the header **X-XSRF-TOKEN**. Another website cannot read our cookie, so its forged POST has no valid header and gets 403.
- **URL rules:** public catalogue and login pages; **/api/admin/****, Swagger and Actuator for ADMIN only; everything else needs login.

Club-level permissions

Global roles (STUDENT, ADMIN) are in the token. Club roles (ORGANIZER, VOLUNTEER of a club) are **read from the database on every check**, so removing an organizer takes effect immediately.

ClubAccess.java (backend/src/main/java/com/eventhub/auth/ClubAccess.java)

```

/** May the logged-in user edit / submit this event? Only organizers of the event's own club.
 */
@Transactional(readOnly = true)
public boolean canManageEvent(Long eventId) {
    return events.findClubIdById(eventId).map(this::isOrganizer).orElse(false);
}

```

Used on controller methods like this: **@PreAuthorize("@clubAccess.canManageEvent(#id)")**.
 Example: Kavya (Cultural Club organizer) may edit Dance Night (event 6, Cultural Club). If she calls **PUT /api/events/5** (Spring Boot Bootcamp, a Coding Club event), the answer is 403 Forbidden.

Stopping password guessing

LoginAttemptService.java (backend/src/main/java/com/eventhub/auth/LoginAttemptService.java)

```

/** Wrong password: count it; the 5th one locks. */
public void failed(String email, String address) {
    if (attempts.size() > CLEANUP_ABOVE) {

```

```

        removeOld();
    }
    attempts.compute(key(email, address), (k, a) -> {
        int failures = (a == null ? 0 : a.failures()) + 1;
        return failures >= maxFailures
            ? new Attempts(0, clock.instant().plus(lockTime))
            : new Attempts(failures, null);
    });
}

```

After the 5th wrong password for the same email **from the same address**, login is locked for 15 minutes (HTTP 429 with a **Retry-After** header), even with the right password. Locking by email alone would let anybody lock your account on purpose; locking by address alone would block a whole college Wi-Fi.

7.5 Bookings: holding seats without ever overselling

The problem

Imagine one seat left and two students pressing "Book" at the same moment. Both requests read "1 seat left". Both subtract one and save "0 seats left". Two bookings, one seat. This is called a **race condition** or **lost update**.

The solution: optimistic locking + retry

The **Event** entity has a **@Version** field. When Hibernate saves the event it runs:

```
UPDATE events SET available_seats = 0, version = 8 WHERE id = 2 AND version = 7
```

The first request changes one row. The second finds no row with version 7 (it is now 8), changes nothing, and Hibernate throws an optimistic-locking exception. The **Retry** helper catches it and runs the whole booking again **in a new transaction**, which reads the fresh value "0 seats left" and answers "sold out".

Retry.java (backend/src/main/java/com/eventhub/booking/Retry.java)

```

public static <T> T onConflict(Supplier<T> work) {
    for (int attempt = 1; ; attempt++) {
        try {
            return work.get();
        }
        catch (OptimisticLockingFailureException | PessimisticLockingFailureException
            | jakarta.persistence.OptimisticLockException ex) {
            // Pessimistic... = MySQL found a deadlock and cancelled one of the two: also safe
            to retry
            if (attempt == MAX_ATTEMPTS) {
                try again.");
            }
            sleep(ThreadLocalRandom.current().nextLong(5, 10 + attempt * 10L));
        }
    }
}

```

BookingService.java (backend/src/main/java/com/eventhub/booking/BookingService.java)

```

private Long holdOnce(Long userId, Long eventId, int quantity) {
    if (quantity < 1 || quantity > MAX_PER_BOOKING) {
        throw new BusinessException("You can book 1 to " + MAX_PER_BOOKING + " seats at a
        time.");
    }
    LocalDateTime now = LocalDateTime.now();
    Event event = events.findById(eventId)

```

```

        .filter(e -> e.getStatus() == EventStatus.PUBLISHED) // drafts are invisible to
students
        .orElseThrow(() -> new ResourceNotFoundException("Event", eventId));
    if (!event.getStartTime().isAfter(now)) {
        throw new BusinessException(event.getTitle() + " has already started.");
    }
    if (bookings.existsByUserIdAndEventIdAndStatusIn(userId, eventId, ACTIVE)) {
        throw new BusinessException("You already have seats for " + event.getTitle() + ".
See My tickets.");
    }
    WaitlistEntry myPlace = waitlist
        .findFirstByUserIdAndEventIdAndStatusIn(userId, eventId,
List.of(WaitlistStatus.WAITING, WaitlistStatus.OFFERED))
        .orElse(null);
    if (myPlace != null && myPlace.getStatus() == WaitlistStatus.OFFERED) {
        throw new BusinessException("Seats of " + event.getTitle() + " are kept for you.
Accept the offer on the Waitlist page.");
    }
    if (event.getAvailableSeats() < quantity) {
        throw new BusinessException(event.getAvailableSeats() == 0
            ? event.getTitle() + " is sold out. You can join the waitlist."
            : "Only " + event.getAvailableSeats() + " seats are left for " +
event.getTitle() + ".");
    }

    // 1) take the seats FIRST: "UPDATE events ... WHERE id = ? AND version = ?".
    // If someone else changed the event since we read it, 0 rows match -> exception ->
Retry.
    // (Doing this before inserting the booking also avoids a MySQL deadlock.)
    event.setAvailableSeats(event.getAvailableSeats() - quantity);
    events.saveAndFlush(event);

    // 2) then write the booking
    Booking booking = saveNew(userId, event, quantity, now);
    if (myPlace != null) {
        // booked directly while waiting: the waitlist place is no longer needed
        myPlace.setBooking(booking);
        myPlace.close(WaitlistStatus.BOOKED, now);
    }
    return booking.getId();
}

```

Details worth noticing:

- `transactions.execute(...)` (TransactionTemplate) gives each attempt its own transaction.
- `events.saveAndFlush(event)` writes the seat change **before** inserting the booking. Always touching the event row first gives every booking the same lock order, which prevents a MySQL **deadlock** that appeared during testing.
- Free events are confirmed immediately; paid ones stay HELD for 10 minutes (`app.booking.hold-time`).

Real-life example: Optimistic locking is like editing a shared Google Sheet cell that shows "last edited version 7". If someone else saved first, your save is refused with "this changed - please reload", instead of silently overwriting their work.

Proof: the concurrency test

`BookingConcurrencyTest` starts **100 threads** that all try to book one seat of an event with **10 seats**, at the same moment, three times in a row. The test checks that exactly 10 bookings succeed and the

event shows 0 seats left. When `@Version` was removed as an experiment, the same test sold 96 to 99 tickets.

Freeing unpaid holds

BookingService.java (backend/src/main/java/com/eventhub/booking/BookingService.java)

```
/**
 * Step 4: called every minute by HoldExpiryJob. Each booking is expired in its own small
 * transaction, so one problem does not stop the others. Returns how many were expired.
 */
public int expireOldHolds() {
    int expired = 0;
    for (Long id : bookings.findExpiredHoldIds(LocalDateTime.now())) {
        List<String> toClose = new ArrayList<>();
        Boolean done = Retry.onConflict(() -> tx.execute(status -> {
            toClose.clear();
            Booking booking = bookings.findById(id).orElse(null);
            LocalDateTime now = LocalDateTime.now();
            // re-check: the student may have paid a moment ago
            if (booking == null || booking.getStatus() != BookingStatus.HELD ||
                booking.getHoldExpiresAt().isAfter(now)) {
                return false;
            }
            release(booking, BookingStatus.EXPIRED, now, toClose, new ArrayList<>());
            return true;
        }));
        toClose.forEach(gateway::expireSession);
        if (Boolean.TRUE.equals(done)) {
            expired++;
        }
    }
    if (expired > 0) {
        log.info("Expired {} unpaid seat holds; their seats are free again", expired);
    }
    return expired;
}
```

`HoldExpiryJob` calls this method every 60 seconds (`@Scheduled`). Each expired hold is handled in its own transaction, so one failure does not stop the others; the freed seats are offered to the waitlist.

Cancelling

BookingService.java (backend/src/main/java/com/eventhub/booking/BookingService.java)

```
/** Step 9: the student cancels. Seats go back; a paid booking gets its money back. */
public BookingResponse cancel(Long userId, Long bookingId) {
    List<String> toClose = new ArrayList<>();
    List<String> toRefund = new ArrayList<>();
    Retry.onConflict(() -> tx.execute(status -> {
        toClose.clear();
        toRefund.clear();
        Booking booking = own(userId, bookingId);
        LocalDateTime now = LocalDateTime.now();
        if (!booking.getStatus().holdsSeats()) {
            throw new BusinessException("This booking is already " +
                booking.getStatus().name().toLowerCase() + ".");
        }
        if (booking.getCheckedInAt() != null) {
            throw new BusinessException("This ticket was already used at the gate, so it
cannot be cancelled.");
        }
        if (booking.getStatus() == BookingStatus.CONFIRMED &&
            !booking.getEvent().getStartTime().isAfter(now)) {
            throw new BusinessException("The event has already started, so this booking
cannot be cancelled.");
        }
        release(booking, BookingStatus.CANCELLED, now, toClose, toRefund);
        booking.setCancelledAt(now);
    }));
}
```

```

        return null;
    }));
    toClose.forEach(gateway::expireSession);
    toRefund.forEach(gateway::refund);
    return get(userId, bookingId);
}

```

A ticket that was already scanned at the gate cannot be cancelled (otherwise someone could enter and then ask for a refund).

7.6 Payments: one interface, two gateways

The booking code never talks to Stripe directly. It talks to an **interface**:

PaymentGateway.java (backend/src/main/java/com/eventhub/payment/PaymentGateway.java)

```

/**
 * The outside payment company, behind one small interface (like a plug socket):
 *   StripePaymentGateway - real Stripe Checkout in test mode (when STRIPE_SECRET_KEY is in
 *   .env)
 *   FakePaymentGateway   - a built-in test page for development (no keys needed)
 * The booking code only talks to this interface, so it works the same with both.
 */
public interface PaymentGateway {

    PaymentProvider provider();

    /** Starts a checkout for a HELD booking. The student is sent to returnUrl to pay. */
    CheckoutSession createCheckout(Booking booking);

    /** Has the student paid for this session? (asked when they come back, in case the webhook
    is late) */
    boolean isPaid(String sessionId);

    /** The hold ran out or was cancelled: close the payment page so nobody can pay any more.
    Best effort. */
    void expireSession(String sessionId);

    /** Gives the money back (cancelled booking, or a payment that arrived after the seat was
    gone). */
    void refund(String sessionId);

    record CheckoutSession(String sessionId, String returnUrl) {
    }

}

```

PaymentConfig chooses the implementation when the application starts: **StripePaymentGateway** if **STRIPE_SECRET_KEY** is set, otherwise **FakePaymentGateway**, which sends the student to EventHub's own test page. In production the fake gateway is refused.

Real-life example: A phone charger with a standard plug: the phone (booking code) works with any charger (Stripe, test page) that fits the socket (interface).

The Stripe webhook (and why it must be idempotent)

After a student pays on Stripe's page, **Stripe's server** calls EventHub (**POST** `/api/payments/stripe/webhook`). This endpoint has no login, so the message is proven by its **signature**:

PaymentController.java (backend/src/main/java/com/eventhub/payment/PaymentController.java)

```

/**

```

```

* Step 7: Stripe tells us a checkout was paid. Anyone on the internet can call this URL, so
we
* FIRST check the Stripe-Signature header: an HMAC made with the webhook secret that only
* Stripe and we know. Wrong signature -> 400, and nothing is changed.
* We answer 200 quickly for every genuine message (also duplicates), otherwise Stripe sends
it again.
*/
@PostMapping("/stripe/webhook")
public ResponseEntity<String> stripeWebhook(@RequestBody String payload,
    @RequestHeader(name = "Stripe-Signature", required = false) String signature) {
    if (webhookSecret.isBlank()) {
        return ResponseEntity.status(HttpStatus.SERVICE_UNAVAILABLE).body("Stripe webhook
secret is not configured");
    }
    com.stripe.model.Event event;
    try {
        event = Webhook.constructEvent(payload, signature, webhookSecret);
    }
    catch (SignatureVerificationException | RuntimeException ex) {
        log.warn("Rejected a webhook with a bad signature");
        return ResponseEntity.badRequest().body("Invalid signature");
    }

    String type = event.getType();
    if ("checkout.session.completed".equals(type) ||
"checkout.session.async_payment_succeeded".equals(type)) {
        Session session = (Session) dataObject(event);
        if ("paid".equals(session.getPaymentStatus())) {
            BookingService.PaidResult result = bookings.markPaid(event.getId(),
session.getId());
            log.info("Stripe {} for session {}: {}", type, session.getId(), result);
        }
    }
    return ResponseEntity.ok("received");
}

```

Stripe may send the same message more than once. **markPaid** first inserts the message id into **processed_payment_events** (primary key). A second insert of the same id fails with a duplicate key, and the message is ignored: **the same notice never confirms twice** (this is called idempotency).

BookingService.java (backend/src/main/java/com/eventhub/booking/BookingService.java)

```

private PaidResult markPaidOnce(String noticeId, String sessionId, List<String> toRefund) {
    processedEvents.insert(noticeId); // throws DataIntegrityViolationException for a
duplicate

    Payment payment = payments.findBySessionId(sessionId).orElse(null);
    if (payment == null) {
        log.warn("Payment notice {} for unknown session {}", noticeId, sessionId);
        return PaidResult.UNKNOWN_SESSION;
    }
    if (payment.getStatus() == PaymentStatus.PAID || payment.getStatus() ==
PaymentStatus.REFUNDED) {
        return PaidResult.ALREADY_DONE;
    }
    LocalDateTime now = LocalDateTime.now();
    payment.setStatus(PaymentStatus.PAID);
    payment.setPaidAt(now);

    Booking booking = payment.getBooking();
    switch (booking.getStatus()) {
        case HELD -> {
            confirm(booking, now);
            return PaidResult.CONFIRMED;
        }
        case EXPIRED -> {
            // paid just after the 10 minutes ran out: give the seats again if they are still
free
            Event event = booking.getEvent();

```

```

        if (event.getAvailableSeats() >= booking.getQuantity() &&
event.getStartTime().isAfter(now)) {
            event.setAvailableSeats(event.getAvailableSeats() - booking.getQuantity());
            events.saveAndFlush(event);
            confirm(booking, now);
            return PaidResult.CONFIRMED;
        }
        return refundLater(payment, now, toRefund);
    }
    default -> {
        // CANCELLED, or CONFIRMED already by another payment: never keep money twice
        return refundLater(payment, now, toRefund);
    }
}
}
}

```

Late payments: if the money arrives after the 10-minute hold expired, the code tries to take the seats again; if they are gone, the payment is **refunded** automatically.

Verify on return

If the webhook is slow (or the Stripe CLI is not running on a laptop), the success page calls **POST** `/api/payments/{session}/verify`, which asks Stripe directly "is this session paid?" and runs the same **markPaid** code.

7.7 The smart waitlist

When an event is sold out, students can join its waitlist. When seats become free (a cancellation, an expired hold, or the organizer adds seats), they are **offered** to the first students in the queue and kept for them for 30 minutes.

WaitlistOffers.java (backend/src/main/java/com/eventhub/waitlist/WaitlistOffers.java)

```

/** Offers the event's free seats to waiting students. Returns how many offers were made. */
public int offerFreeSeats(Event event, LocalDateTime now) {
    if (event.getStatus() != EventStatus.PUBLISHED || !event.getStartTime().isAfter(now)
        || event.getAvailableSeats() == 0) {
        return 0;
    }
    int offers = 0;
    for (WaitlistEntry entry : entries.findByEventIdAndStatusOrderByIdAsc(event.getId(),
WaitlistStatus.WAITING)) {
        if (event.getAvailableSeats() == 0) {
            break;
        }
        if (entry.getQuantity() > event.getAvailableSeats()) {
            continue; // does not fit yet; keeps its place
        }
        event.setAvailableSeats(event.getAvailableSeats() - entry.getQuantity()); // kept for
this student
        entry.setStatus(WaitlistStatus.OFFERED);
        entry.setOfferedAt(now);
        entry.setOfferExpiresAt(now.plus(offerTime));
        // sent to listeners (email, bell) only AFTER the transaction commits - see
notification package
        publisher.publishEvent(new SeatOffered(entry.getId()));
        offers++;
    }
    if (offers > 0) {
        events.saveAndFlush(event); // optimistic lock: a clash with another booking -> the
caller retries
        log.info("Offered free seats of event {} to {} waiting student(s)", event.getId(),
offers);
    }
    return offers;
}
}

```

Rules visible in the code:

- The queue is served in order of joining (**id** ascending).
- A group that does not fit (Ravi wants 3, only 2 are free) **keeps its place**; smaller requests behind it may be served meanwhile.
- **offerFreeSeats** runs **inside the same transaction** that freed the seats, so seats are never free and un-offered at the same time.
- It publishes **SeatOffered**, which triggers the email and the bell notification.

WaitlistOfferJob runs every 60 seconds: offers older than 30 minutes expire and the seats move to the next student.

7.8 Notifications: bell, email and reminders

The notification module **listens** to events from other modules (**SeatOffered**, **BookingConfirmed**, **EventReviewed**):

NotificationTriggers.java (backend/src/main/java/com/eventhub/notification/NotificationTriggers.java)

```
/** Done-when check of Phase 7: a cancel frees a seat -> the first waitlisted student is
    emailed. */
@EventListener
public void seatOffered(SeatOffered offered) {
    WaitlistEntry entry = waitlist.findById(offered.entryId()).orElseThrow();
    Event event = entry.getEvent();
    String seats = entry.getQuantity() == 1 ? "1 seat" : entry.getQuantity() + " seats";
    String them = entry.getQuantity() == 1 ? "it" : "them";
    notifications.create(entry.getUser().getId(), NotificationKind.SEAT_OFFERED,
        "Seats free for " + event.getTitle() + " - kept for you",
        "Good news! " + seats + " for " + event.getTitle() + " (" +
    WHEN.format(event.getStartTime())
        + ") came free and we are keeping " + them + " for you until " +
    TIME.format(entry.getOfferExpiresAt())
        + ". Accept " + them + " on your Waitlist page before then, or " +
    (entry.getQuantity() == 1 ? "it goes" : "they go")
        + " to the next student.",
        "/waitlist", true);
}
```

The email is sent **after the transaction commits**:

EmailSender.java (backend/src/main/java/com/eventhub/notification/EmailSender.java)

```
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
public void afterCommit(EmailQueued queued) {
    send(queued.notificationId());
}
```

Why after commit? If the email were sent inside the transaction and the transaction then rolled back (for example because of a conflict), the student would get "A seat is waiting for you" for a seat that does not exist. This pattern - save the message with the change, deliver it after the commit - is a simple form of the **outbox pattern**.

If the mail server is down, the notification stays PENDING and **NotificationJobs.emailRetry** tries again every minute, at most 5 times (then FAILED). The bell in the app shows the notification anyway.

Reminders: every day at 18:00 (India time) `ReminderService.remindForDayAfter` sends "See you tomorrow" to everyone with a confirmed ticket for tomorrow's events, once per booking (`reminder_sent_at`).

7.9 The gate: QR codes and check-in

Drawing the QR code

QrCodeService.java (backend/src/main/java/com/eventhub/gate/QrCodeService.java)

```
public byte[] png(String text, int size) {
    try {
        // M = about 15% of the squares may be dirty or scratched and it still reads; margin 2
        = white border
        BitMatrix matrix = new QRCodeWriter().encode(text, BarcodeFormat.QR_CODE, size, size,
            Map.of(EncodeHintType.ERROR_CORRECTION, ErrorCorrectionLevel.M,
            EncodeHintType.MARGIN, 2));
        ByteArrayOutputStream out = new ByteArrayOutputStream();
        MatrixToImageWriter.writeToStream(matrix, "PNG", out);
        return out.toByteArray();
    }
    catch (WriterException ex) {
        throw new IllegalArgumentException("Cannot make a QR code of: " + text, ex);
    }
    catch (IOException ex) {
        throw new UncheckedIOException(ex);
    }
}
```

The QR code contains **only** the random ticket code (for example EVH-97Z38-F9Y5Q), not the name or the event - nothing private is printed on the ticket.

One scan, one entry

The heart of the gate is one SQL statement that checks and marks in a single step:

BookingRepository.java (backend/src/main/java/com/eventhub/booking/BookingRepository.java, lines 40-48)

```
/**
 * Marks the ticket as used - ONLY if it is not used yet. Returns 1 (you let them in) or 0
 * (someone
 * else scanned it a moment earlier). The database does the "check and set" in one step, so
 * two
 * volunteers can never both get VALID. The version goes up too, like any other change of the
 * booking.
 */
@Modifying(flushAutomatically = true, clearAutomatically = true)
@Query("update Booking b set b.checkedInAt = :now, b.checkedInBy = :scanner, b.version =
b.version + 1"
    + " where b.id = :id and b.checkedInAt is null and b.status =
com.eventhub.booking.BookingStatus.CONFIRMED")
int markCheckedIn(@Param("id") Long id, @Param("now") LocalDateTime now, @Param("scanner")
User scanner);
```

CheckInService.java (backend/src/main/java/com/eventhub/gate/CheckInService.java)

```
@Transactional
public CheckInResponse checkIn(Long eventId, String rawCode, Long scannerId) {
    Event event = events.findById(eventId).orElseThrow(() -> new
    ResourceNotFoundException("Event", eventId));
    String code = rawCode.trim().toUpperCase(Locale.ROOT);
    LocalDateTime now = LocalDateTime.now();

    Booking booking = bookings.findByTicketCode(code).orElse(null);
    if (booking == null) {
        return invalid(event, code, "This code is not an EventHub ticket. Ask for the QR code
in the app.");
    }
}
```

```

        if (!booking.getEvent().getId().equals(eventId)) {
            return invalid(event, code, "This ticket is for " + booking.getEvent().getTitle() + ",
not " + event.getTitle() + ".");
        }
        String holder = booking.getUser().getFullName();
        if (booking.getStatus() != BookingStatus.CONFIRMED) {
            String why = switch (booking.getStatus()) {
                case CANCELLED -> "This booking was cancelled.";
                case HELD -> "This booking is not paid yet.";
                default -> "This booking was never paid (" +
booking.getStatus().name().toLowerCase() + ").";
            };
            return invalid(event, code, why);
        }
        if (!event.getEndTime().isAfter(now)) {
            return invalid(event, code, event.getTitle() + " is already over.");
        }
        if (booking.getCheckedInAt() != null) {
            return alreadyUsed(event, code, holder, booking);
        }

        // the one important line: "mark used, but only if not used yet" - done by the database in
one step
        int updated = bookings.markCheckedIn(booking.getId(), now,
users.getReferenceById(scannerId));
        if (updated == 0) {
            // Another gate scanned it a split second ago. We cannot simply read the booking again
to see when:
            // MySQL (REPEATABLE READ) keeps showing this transaction the snapshot from its first
read,
            // where the ticket was still unused.
            return new CheckInResponse(CheckInResult.ALREADY_USED, "Already used just now at
another gate.", code, holder,
                booking.getQuantity(), null, stats(eventId));
        }
        int people = booking.getQuantity();
        return new CheckInResponse(CheckInResult.VALID, "Let in " + people + (people == 1 ? "
person" : " people") + ". ",
            code, holder, people, now, stats(eventId));
    }
}

```

How to read the result:

- **markCheckedIn** returns **1** (one row changed): this scan won → VALID, "Let in".
- It returns **0**: the ticket was already used → ALREADY USED, with the time and the volunteer name.
- The code does not exist, is for another event, or was cancelled → INVALID.

Real-life example: Like a library book with one stamp box: the librarian stamps it only if the box is empty. Two librarians cannot both stamp it - the second sees the box is already filled.

A subtle bug found during testing: MySQL's default isolation (REPEATABLE READ) keeps showing a transaction the data as it was when it started. The gate that lost the race re-read the ticket and still saw "not used". The fix: decide from the UPDATE's result instead of reading again.

GateConcurrencyTest has 20 gates scan the same ticket at once and expects exactly one VALID.

7.10 Certificates

Certificates are created **lazily**: when a student opens "My certificates", the server looks for their bookings that were checked in and whose event has ended, and creates any missing certificates.

CertificateService.java (backend/src/main/java/com/eventhub/certificate/CertificateService.java)

```
/** My certificates: first issue any that were earned since last time, then list them all. */
public List<CertificateResponse> mine(Long userId) {
    for (Long bookingId : bookings.findIdsEarningCertificate(userId, LocalDateTime.now())) {
        if (!certificates.existsByBookingId(bookingId)) {
            issue(bookingId);
        }
    }
    return tx.execute(s -> certificates.findByUserIdOrderByIssuedAtDesc(userId).stream()
        .map(CertificateResponse::from).toList());
}
```

CertificateService.java (backend/src/main/java/com/eventhub/certificate/CertificateService.java)

```
/** EH-2026-7QK2MP4X : 8 random characters = about 1 million million possibilities. */
private static String newNumber(int year) {
    StringBuilder code = new StringBuilder("EH-").append(year).append('-');
    for (int i = 0; i < 8; i++) {
        code.append(CODE_CHARS[RANDOM.nextInt(CODE_CHARS.length)]);
    }
    return code.toString();
}
```

- The number is random (EH-2026 + 8 characters without look-alike letters), so nobody can walk through numbers 1, 2, 3 and collect names.
- `certificates.booking_id` is UNIQUE: even two clicks at the same moment create only one certificate.
- The PDF is drawn by `CertificatePdf` with OpenPDF: name, event, club, date, venue, number and a QR code that opens the public verify page.
- `GET /api/certificates/verify/{number}` is public: it answers only with the name, event, club and date - no email or other private data.

7.11 Feedback

FeedbackService.java (backend/src/main/java/com/eventhub/feedback/FeedbackService.java)

```
/** Give or change my rating for one attended ticket. */
@Transactional
public MyFeedbackResponse give(Long userId, Long bookingId, FeedbackRequest request) {
    Booking b = bookings.findWithEventById(bookingId)
        .filter(x -> x.getUser().getId().equals(userId)) // someone else's booking = 404
        .orElseThrow(() -> new ResourceNotFoundException("Booking", bookingId));
    Event event = b.getEvent();
    if (b.getCheckedInAt() == null) {
        throw new BusinessException("Only students who attended " + event.getTitle() + " (ticket scanned at the gate) can rate it.");
    }
    if (event.getEndTime().isAfter(LocalDateTime.now())) {
        throw new BusinessException("You can rate " + event.getTitle() + " after it ends.");
    }
    String comment = request.comment() == null || request.comment().isBlank() ? null : request.comment().trim();

    Feedback f = feedback.findByBookingId(bookingId).orElse(null);
    if (f == null) {
        f = new Feedback();
        f.setBooking(b);
        f.setEvent(event);
        f.setUser(b.getUser());
    }
    else {
        f.setUpdatedAt(LocalDateTime.now());
    }
}
```

```

    }
    f.setRating(request.rating());
    f.setComment(comment);
    feedback.save(f);
    return new MyFeedbackResponse(b.getId(), event.getId(), event.getTitle(),
event.getStartTime(), f.getRating(), comment);
}

```

- Only the ticket's owner, only if scanned at the gate, only after the event ended.
- **PUT** means "set it to this": sending the same rating twice gives the same result, and a student can change their mind.
- Organizers see average, star counts and comments **without names**, so students can be honest.

A small real problem: the database column is **TINYINT**, the Java field **int**. With **ddl-auto: validate**, Hibernate refused to start ("wrong column type"). The fix was one annotation: **@JdbcTypeCode(SqlTypes.TINYINT)**.

7.12 Analytics with caching

The database does the counting with JPQL aggregates (**SUM**, **COUNT**, **AVG**, **GROUP BY**), so Java receives a few small rows instead of every booking:

AnalyticsQueries.java (backend/src/main/java/com/eventhub/analytics/AnalyticsQueries.java, lines 36-55)

```

/**
 * One row per published event starting on or after `from`. LEFT JOIN keeps events with no
 * bookings (0 sold).
 * The booking condition sits in the ON part, not in WHERE - in WHERE it would drop those
 * events again.
 */
@Query("""
    select new com.eventhub.analytics.AnalyticsQueries$EventSales(
        e.id, e.title, e.startTime, e.totalSeats,
        coalesce(sum(b.quantity), 0L),
        coalesce(sum(case when b.checkedInAt is not null then b.quantity else 0 end), 0L),
        coalesce(sum(b.amount), 0))
    from Event e
        left join Booking b on b.event = e and b.status =
com.eventhub.booking.BookingStatus.CONFIRMED
    where e.status = com.eventhub.event.EventStatus.PUBLISHED
        and e.startTime >= :from
        and (:clubId is null or e.club.id = :clubId)
    group by e.id, e.title, e.startTime, e.totalSeats
    order by e.startTime
    """)
List<EventSales> salesPerEvent(@Param("clubId") Long clubId, @Param("from") LocalDateTime
from);

```

AnalyticsService.java (backend/src/main/java/com/eventhub/analytics/AnalyticsService.java)

```

/**
 * @param clubId one club, or null = all clubs
 * @param days the last N days, today included
 */
@Cacheable(cacheNames = CACHE, key = "(#clubId ?: 'all') + ':' + #days")
@Transactional(readOnly = true)
public AnalyticsResponse dashboard(Long clubId, int days) {
    LocalDateTime now = LocalDateTime.now();
    LocalDate to = now.toLocalDate();
    LocalDate from = to.minusDays(days - 1L);

    List<DayPoint> perDay = everyDay(from, to, queries.salesPerDay(clubId,
from.atStartOfDay()));
}

```

```

    List<EventRow> events = withRatings(queries.salesPerEvent(clubId, from.atStartOfDay()),
now);

    return new AnalyticsResponse(from, to, days, now, totals(perDay, events), perDay, events);
}

```

- `@Cacheable(cacheNames = "analytics", key = "(#clubId ?: 'all') + ':' + #days")` stores the answer per club and period in a Caffeine cache for 60 seconds (`spring.cache.caffeine.spec: expireAfterWrite=60s`).
- `everyDay` fills days without sales with 0, so the chart has no gaps.
- The check-in rate uses only events that have already started (for a future event nobody could come yet).
- The average rating is **weighted**: an event rated by 10 people counts more than one rated by 1.

Real-life example: Caching is like a shop's daily sales board: the manager writes the totals once every few minutes instead of counting every receipt each time someone asks.

7.13 The audit log

AuditService.java (backend/src/main/java/com/eventhub/audit/AuditService.java)

```

/** Who did it comes from the logged-in user of this request (empty for system jobs and
service tests). */
public void record(AuditAction action, Event event, String details) {
    AuditEntry entry = new AuditEntry();
    entry.setAction(action);
    entry.setEventId(event.getId());
    entry.setEventTitle(event.getTitle());
    entry.setDetails(details);
    CurrentUser.get().ifPresent(user -> {
        entry.setUserId(user.id());
        entry.setUserName(user.fullName());
        entry.setUserEmail(user.email());
    });
    entries.save(entry);
}

```

It is called inside the same transaction as the change, so a change and its audit row are saved together or not at all. The user's name and email are copied into the row, so the history stays readable even if the account is later deleted.

8. REST API reference

8.1 What a REST API is

The backend offers its features as **URLs** that the frontend (or Postman, or a test) calls with an HTTP **method**:

Method	Meaning	Example
GET	read, changes nothing	GET /api/events/2 - one event
POST	create or trigger an action	POST /api/bookings - hold seats
PUT	replace / set to this value	PUT /api/events/2 - save the edited event
DELETE	remove	(not needed in EventHub; things are cancelled, not deleted)

Answers use HTTP **status codes**: 200 OK, 201 Created (with a **Location** header), 204 No Content, 400 Bad Request, 401 Unauthorized (not logged in), 403 Forbidden (not allowed), 404 Not Found, 409 Conflict (a business rule said no), 429 Too Many Requests, 500 Server Error.

Real-life example: A REST API is like the menu of a restaurant: every dish (URL) has a name and a way to order it (method). You do not walk into the kitchen; you order from the menu and the waiter brings the answer.

8.2 All endpoints

This table is generated from the controller classes when the document is built.

Method	URL	Who may call it	Controller
GET	/api/organizer/clubs/{clubId}/analytics	club rule: isOrganizer(#clubId)	AnalyticsController
GET	/api/admin/analytics	ADMIN	AnalyticsController
POST	/api/auth/register	anyone	AuthController
POST	/api/auth/login	anyone	AuthController
POST	/api/auth/logout	anyone	AuthController
GET	/api/auth/me	anyone	AuthController
POST	/api/bookings	logged in	BookingController
GET	/api/bookings/mine	logged in	BookingController
GET	/api/bookings/{id}	logged in	BookingController
GET	/api/bookings/{id}/qr.png	logged in	BookingController
POST	/api/bookings/{id}/pay	logged in	BookingController
POST	/api/bookings/{id}/cancel	logged in	BookingController
GET	/api/certificates/mine	logged in	CertificateController
GET	/api/certificates/{number}/pdf	logged in	CertificateController
GET	/api/certificates/verify/{number}	anyone	CertificateController
GET	/api/clubs	anyone	ClubController
GET	/api/clubs/{slug}	anyone	ClubController

Method	URL	Who may call it	Controller
GET	/api/admin/events/pending	ADMIN	AdminEventControll
GET	/api/admin/stats/clubs	ADMIN	AdminEventControll
GET	/api/admin/audit	ADMIN	AdminEventControll
GET	/api/events	anyone	EventController
GET	/api/events/{id}	anyone	EventController
POST	/api/events	club rule: isOrganizer(#request.clubId())	EventController
PUT	/api/events/{id}	club rule: canManageEvent(#id) and @clubAccess.isOrganizer(#request.clubId())	EventController
POST	/api/events/{id}/submit	club rule: canManageEvent(#id)	EventController
POST	/api/events/{id}/approve	ADMIN	EventController
POST	/api/events/{id}/reject	ADMIN	EventController
GET	/api/organizer/clubs/{clubId}/events	club rule: isOrganizer(#clubId)	OrganizerEventCont
GET	/api/organizer/events/{id}	club rule: canManageEvent(#id)	OrganizerEventCont
GET	/api/feedback/mine	logged in	FeedbackController
PUT	/api/bookings/{bookingId}/feedback	logged in	FeedbackController
GET	/api/organizer/events/{eventId}/feedback	club rule: canManageEvent(#eventId)	FeedbackController
GET	/api/gate/events	logged in	GateController
POST	/api/gate/events/{eventId}/check-in	club rule: canScanEvent(#eventId)	GateController
GET	/api/gate/events/{eventId}/stats	club rule: canScanEvent(#eventId)	GateController
GET	/api/health	anyone	HealthController
GET	/api/notifications	logged in	NotificationControll
POST	/api/notifications/{id}/read	logged in	NotificationControll
POST	/api/notifications/read-all	logged in	NotificationControll
POST	/api/admin/reminders/run	ADMIN	NotificationControll
POST	/api/payments/stripe/webhook	anyone	PaymentController
POST	/api/payments/{sessionId}/verify	logged in	PaymentController
GET	/api/payments/fake/{sessionId}	logged in	PaymentController
POST	/api/payments/fake/{sessionId}/complete	logged in	PaymentController
GET	/api/tags	anyone	TagController
POST	/api/waitlist	logged in	WaitlistController
GET	/api/waitlist/mine	logged in	WaitlistController
GET	/api/waitlist/{id}	logged in	WaitlistController
POST	/api/waitlist/{id}/leave	logged in	WaitlistController
POST	/api/waitlist/{id}/accept	logged in	WaitlistController

50 endpoints in total. "logged in" = any logged-in user; club rules are checked by the ClubAccess bean on every call.

8.3 Examples with request and response

Search events (public)

```
GET /api/events?tag=robotics&page=0&size=9&sort=date&dir=asc

200 OK
{
  "content": [
    { "id": 16, "title": "Arduino Basics Workshop", "clubName": "Robotics Club",
      "clubSlug": "robotics-club", "venue": "ECE Lab 3", "startTime": "2026-10-14T10:00:00",
      "availableSeats": 12, "totalSeats": 40, "soldOut": false, "price": 50.00,
      "status": "PUBLISHED", "tags": ["robotics", "workshop"] },
    { "id": 15, "title": "Robo Race", ... }
  ],
  "page": 0, "size": 9, "totalElements": 2, "totalPages": 1
}
```

Log in

```
POST /api/auth/login
X-XSRF-TOKEN: 3f1c... (copied from the XSRF-TOKEN cookie)
{ "email": "ravi@eventhub.test", "password": "*****" }

200 OK
Set-Cookie: EVENTHUB_TOKEN=eyJhbGciOiJIUzI1NiJ9...; Path=/; HttpOnly; SameSite=Lax; Max-Age=28800
{ "id": 5, "fullName": "Ravi Kumar", "email": "ravi@eventhub.test", "roles": ["STUDENT"],
  "clubs": [] }
```

Wrong password: **401** {"detail": "Wrong email or password."}. Sixth try within 15 minutes: **429** with **Retry-After: 900**.

Hold seats, then pay

```
POST /api/bookings
{ "eventId": 2, "quantity": 1 }

201 Created
Location: /api/bookings/41
{ "id": 41, "status": "HELD", "quantity": 1, "amount": 150.00, "ticketCode": "EVH-97Z38-F9Y5Q",
  "secondsLeft": 600, "event": { "id": 2, "title": "24-Hour Hackathon", ... }, "canCancel": true }

POST /api/bookings/41/pay
200 OK
{ "provider": "FAKE", "sessionId": "fake_cs_8f2...", "redirectUrl":
  "http://localhost:5173/test-payment/fake_cs_8f2..." }
```

Scan a ticket at the gate

```
POST /api/gate/events/2/check-in
{ "code": "EVH-97Z38-F9Y5Q" }

200 OK
{ "result": "VALID", "message": "Let in 1 person.", "holder": "Ravi Kumar", "quantity": 1,
  "code": "EVH-97Z38-F9Y5Q", "stats": { "bookedPeople": 111, "checkedInPeople": 1 } }

(the same request again)
200 OK
{ "result": "ALREADY_USED", "message": "Already used at 8:41 pm by Priya Raman.", ... }
```

Validation error

```
POST /api/events
```

```
{ "clubId": 1, "title": "", "totalSeats": 0, ... }
```

400 Bad Request

```
{ "title": "Bad Request", "status": 400, "detail": "Validation failed",  
  "errors": { "title": "title is required", "totalSeats": "totalSeats must be at least 1" } }
```

8.4 Swagger UI

The full, always-current API description is generated by springdoc-openapi from the controllers. Log in as the admin in the React app, then open <http://localhost:8080/swagger-ui.html>. Each endpoint can be tried directly from the page ("Try it out"). It is admin-only because a complete map of the API is valuable information for an attacker.

8.5 The Postman collection

`postman/EventHub.postman_collection.json` contains 25 requests in folders (catalogue, create and edit, approval workflow, error cases) with 43 automatic checks such as "status is 201" and "the Location header points to the new event". It is a quick way to see that the API works after a change.

9. Frontend explained

9.1 How the React app starts

`main.jsx` puts three things around the whole app: the Redux **store** (shared data), the **router** (which page to show) and React's StrictMode (extra checks in development).

main.jsx (frontend/src/main.jsx, lines 1-19)

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import { Provider } from 'react-redux'
import { BrowserRouter } from 'react-router-dom'
import './index.css'
import App from './App.jsx'
import { store } from './store'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <Provider store={store}>
      <BrowserRouter>
        <App />
      </BrowserRouter>
    </Provider>
  </StrictMode>,
)
```

`App.jsx` lists every route. Pages that only some users need are loaded **lazily** (code splitting) - see 9.7.

9.2 Folder by folder

Folder	What is inside	Example
<code>api/</code>	one file per backend area; each function is one HTTP call	<code>bookings.js</code> : <code>holdSeats</code> , <code>payBooking</code> , <code>fetchMyBookings</code>
<code>components/ui/</code>	the design system: Button, Card, Badge, Modal, StatCard, FormField, Loader, EmptyState	<code><Button variant="secondary"></code>
<code>components/layout/</code>	Navbar, Footer, NotificationBell, Toaster, Layout	the bell polls every 30 s
<code>components/...</code>	feature pieces: events, scanner, tickets, organizer, admin, analytics, auth	<code>EventCard</code> , <code>QrCamera</code> , <code>ResultPanel</code>
<code>pages/</code>	one file per screen	<code>CheckoutPage.jsx</code> , <code>ScannerPage.jsx</code>
<code>store/</code>	Redux slices	<code>authSlice.js</code> , <code>cartSlice.js</code>
<code>hooks/</code>	reusable logic	<code>useForm</code> , <code>useAsync</code> , <code>useCountdown</code> , <code>useTheme</code>
<code>utils/</code>	helpers	<code>format.js</code> (dates, ₹ prices), <code>validation.js</code> (form rules)

9.3 Talking to the backend

client.js (frontend/src/api/client.js, lines 1-12)

```
import axios from 'axios'

// All backend calls go through this one "messenger".
```

```
// "/api" is forwarded to Spring Boot (localhost:8080) by vite.config.js.
// Errors: every page turns a failed call into a friendly message with describeError()
(hooks/useAsync.js),
// which reads the backend's problem-details JSON ({ detail, errors }).
const api = axios.create({
  baseURL: '/api',
  timeout: 10_000, // give up after 10 seconds instead of spinning forever
})

export default api
```

All calls go through this one Axios instance. The base address `/api` goes to the Vite dev server, which forwards it to Spring Boot. Axios automatically sends cookies (the login) and copies the `XSRF-TOKEN` cookie into the `X-XSRF-TOKEN` header, which is exactly what the backend's CSRF protection expects.

When a call fails, `describeError` turns the backend's Problem Details into `{ status, message, fieldErrors }` that every page can show:

useAsync.js (frontend/src/hooks/useAsync.js)

```
/**
 * Turns an Axios error into { status, message, fieldErrors } using the backend's problem-
 * details JSON.
 * fieldErrors is the "errors" map from GlobalExceptionHandler, e.g. { title: 'title is
 * required' }.
 */
export function describeError(error) {
  const status = error?.response?.status
  const data = error?.response?.data
  if (!status) return { status: 0, message: 'Cannot reach the server. Is the backend
  running?', fieldErrors: {} }
  return { status, message: data?.detail || 'Something went wrong.', fieldErrors: data?.errors
  ?? {} }
}
```

The store also installs an **interceptor**: if any call answers 401 while the app thinks someone is logged in, the login has expired (after 8 hours) - the app forgets the user and says "Your login has expired".

index.js (frontend/src/store/index.js, lines 32-41)

```
// If any call answers 401 while we think someone is logged in, their login ran out (8 hours):
// forget the user and say so. Protected pages then send them to the login page.
api.interceptors.response.use(undefined, (error) => {
  const isAuthCall = error.config?.url?.startsWith('/auth/')
  if (error.response?.status === 401 && !isAuthCall && store.getState().auth.user) {
    store.dispatch(sessionExpired())
    store.dispatch(notify('Your login has expired. Please log in again.', 'error'))
  }
  return Promise.reject(error)
})
```

9.4 Redux slices

Slice	Holds	Main actions
auth	the logged-in user, loading status	<code>loadSession</code> , <code>login</code> , <code>register</code> , <code>logout</code> , <code>sessionExpired</code>
cart	the seats being held and when the hold ends	<code>seatsHeld</code> , <code>cartCleared</code>
notifications	toast messages (bottom corner)	<code>notify(text, tone)</code> , <code>dismissed</code>
student	the student's waitlist entries	<code>loadWaitlist</code>
organizer	the chosen club and its events	<code>loadClubEvents</code> , <code>saveEvent</code> , <code>submitEvent</code>
admin	approval queue, club stats, audit log	<code>approveEvent</code> , <code>rejectEvent</code> , <code>loadClubStats</code>

An **async thunk** is an action that calls the API and has three outcomes: pending, fulfilled or rejected.

Example:

organizerSlice.js (frontend/src/store/organizerSlice.js)

```
/**
 * The club + all its events (drafts too). Which club: the one picked in the sidebar,
 * else the first club the logged-in user organizes (from GET /api/auth/me).
 */
export const loadClubEvents = createAsyncThunk('organizer/loadClubEvents', async (_, {
  getState, rejectWithValue }) => {
  const mine = clubsWithRole(getState().auth.user, 'ORGANIZER')
  const picked = getState().organizer.clubId
  const clubId = mine.some((c) => c.clubId === picked) ? picked : mine[0]?.clubId
  if (!clubId) return rejectWithValue({ message: 'You are not an organizer of any club.' })
  try {
    const [clubs, events] = await Promise.all([fetchClubs(), fetchClubEvents(clubId)])
    return { clubId, club: clubs.find((c) => c.id === clubId) ?? null, events }
  } catch (e) {
    return rejectWithValue(describeError(e))
  }
})
```

Real-life example: A thunk is like sending a friend to the shop: you note "gone to the shop" (pending), then either "came back with milk" (fulfilled) or "shop was closed" (rejected). The page shows a spinner, the data, or an error for each case.

9.5 Protecting pages: RequireAuth

RequireAuth.jsx (frontend/src/components/auth/RequireAuth.jsx)

```
/**
 * Guards a group of routes:
 * <Route element={<RequireAuth allow={isAdmin} />} ...admin pages... </Route>
 * - still asking the server who you are -> spinner
 * - not logged in -> login page, then back here (?next=)
 * - logged in but not allowed -> a friendly "no permission" page
 * This only hides pages. The real protection is the backend, which checks every request
 * again.
 */
export default function RequireAuth({ allow = () => true, what = 'this page' }) {
  const user = useSelector(selectUser)
  const status = useSelector(selectAuthStatus)
  const location = useLocation()

  if (status === 'checking') {
    return <div className="flex justify-center py-24"><Spinner label="Checking your login"
  /></div>
  }
  if (!user) {
    const next = encodeURIComponent(location.pathname + location.search)
    return <Navigate to={`/login?next=${next}`} replace />
  }
  if (!allow(user)) {
    return (
      <EmptyState
        title="You cannot open this page"
        message={`Your account does not have permission for ${what}. Ask the admin if you
think this is a mistake.`}
        action={<Button to="/" variant="secondary">Go to Home</Button>}
      />
    )
  }
  return <Outlet />
}
```

Used in `App.jsx` around groups of routes, e.g. `<Route element={<RequireAuth allow={isAdmin} what="the admin area" />}>`. As the comment in the code says: this only **hides** pages. The backend checks every request again.

9.6 Forms and validation

`useForm` keeps the values of a form, validates them with small rules from `utils/validation.js`, shows an error only after the user leaves a field (or presses submit), puts the cursor on the first wrong field, and can show errors that came back from the server.

useForm.js (frontend/src/hooks/useForm.js)

```
/**
 * Form state + validation.
 * - errors appear after you leave a field (blur), or for every field when you press submit
 * - once shown, an error updates live while you type the fix
 * - onValidSubmit(values, { setServerErrors }): setServerErrors({ title: '...' }) shows the
    backend's
    * answer under a field until that field is edited
 */
export default function useForm(initialValues, rules, onValidSubmit) {
  const [values, setValues] = useState(initialValues)
  const [touched, setTouched] = useState({})
  const [submitting, setSubmitting] = useState(false)
  const [serverErrors, setServerErrors] = useState({})

  const allErrors = validate(values, rules)
  const errors = { ...serverErrors,
    ...Object.fromEntries(Object.entries(allErrors).filter(([f]) => touched[f])) }

  const field = (name) => ({
    name,
    id: name,
    value: values[name],
    error: errors[name],
    onChange: (e) => {
      const { type, checked, value } = e.target
      setValues((v) => ({ ...v, [name]: type === 'checkbox' ? checked : value }))
      setServerErrors(({ [name]: _fixed, ...rest }) => rest)
    },
    onBlur: () => setTouched((t) => ({ ...t, [name]: true })),
  })

  const handleSubmit = async (e) => {
    e.preventDefault()
    setTouched(Object.fromEntries(Object.keys(rules).map((f) => [f, true])))
    if (Object.keys(allErrors).length > 0) {
      // move the cursor to the first field that needs fixing
      document.getElementById(Object.keys(allErrors)[0])?.focus()
      return
    }
    setSubmitting(true)
    try {
      await onValidSubmit(values, { setServerErrors })
    } finally {
      setSubmitting(false)
    }
  }

  /** For inputs that are not a plain <input>, e.g. a list of tag chips. */
  const setValue = (name, value) => {
    setValues((v) => ({ ...v, [name]: value }))
    setTouched((t) => ({ ...t, [name]: true }))
  }
}
```

```
return { values, errors, field, setValue, handleSubmit, submitting }
}
```

Example rules for the register form: email must look like an email, password at least 8 characters with letters and numbers (and at most 72 - BCrypt only reads 72 bytes), "confirm password" must match, and the rules checkbox must be ticked. The backend checks the same rules again with Bean Validation, because anybody can call the API without the React app.

9.7 Timers, QR codes, charts and code splitting

- **Seat-hold timer** - `useCountdown(endsAt)` recalculates from the real clock every second, so it stays right even if the phone put the tab to sleep. The end time comes from the server (`secondsLeft`).

useCountdown.js (frontend/src/hooks/useCountdown.js)

```
/**
 * Counts down to a fixed moment. Returns the seconds left (never below 0).
 * Uses the real clock, so it stays correct even if the browser tab was in the background.
 */
export default function useCountdown(endsAt) {
  const [now, setNow] = useState(() => Date.now())

  useEffect(() => {
    const timer = setInterval(() => setNow(Date.now()), 1000)
    return () => clearInterval(timer)
  }, [])

  return Math.max(0, Math.ceil((endsAt - now) / 1000))
}
```

- **QR camera** - `QrCamera` uses `html5-qrcode` to read the code from the camera; on devices without a camera the volunteer types the code.
- **Charts** - `AnalyticsDashboard` draws two bar charts with `Recharts` (tickets per day, money per day - never on one axis, because they have different scales), plus a table view for screen readers.
- **Code splitting** - organizer, admin, scanner and analytics pages are loaded with `React.lazy`. The first download fell from 1,213 kB to 329 kB; the chart and camera libraries load only when those pages open.

App.jsx (frontend/src/App.jsx, lines 28-40)

```
// Loaded only when someone opens them (React.lazy = code splitting): a student browsing
// events never
// downloads the charts (Recharts), the camera scanner or the organizer/admin screens.
// <Suspense> in Layout / OrganizerLayout / AdminLayout shows a spinner while a page's file
// loads.
const AdminAnalyticsPage = lazy(() => import('./pages/admin/AdminAnalyticsPage'))
const AdminOverviewPage = lazy(() => import('./pages/admin/AdminOverviewPage'))
const ApprovalQueuePage = lazy(() => import('./pages/admin/ApprovalQueuePage'))
const EventFeedbackPage = lazy(() => import('./pages/organizer/EventFeedbackPage'))
const EventFormPage = lazy(() => import('./pages/organizer/EventFormPage'))
const OrganizerAnalyticsPage = lazy(() => import('./pages/organizer/OrganizerAnalyticsPage'))
const OrganizerDashboardPage = lazy(() => import('./pages/organizer/OrganizerDashboardPage'))
const VolunteersPage = lazy(() => import('./pages/organizer/VolunteersPage'))
const ScannerPage = lazy(() => import('./pages/ScannerPage'))
const StyleGuidePage = lazy(() => import('./pages/StyleGuidePage'))
```

9.8 Design system, dark mode and accessibility

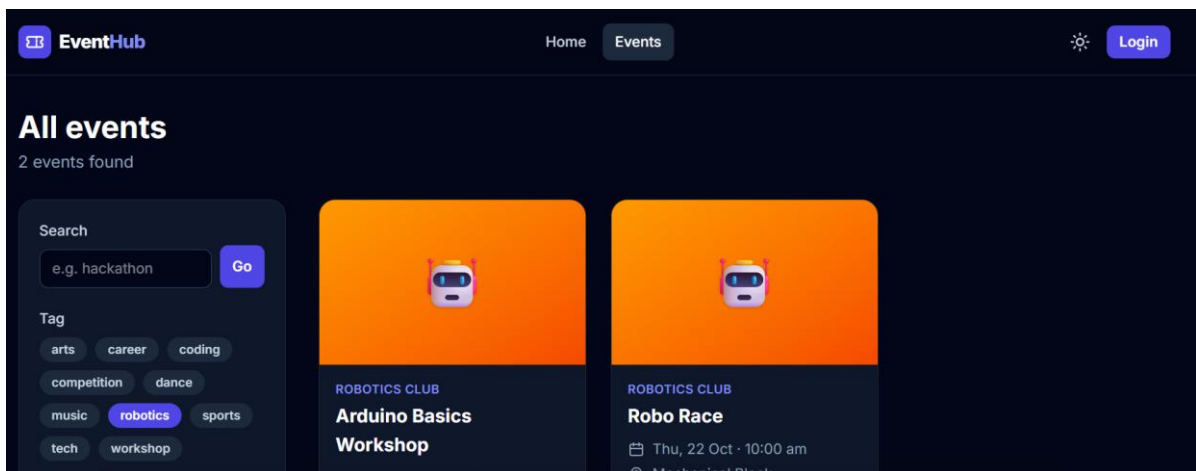
- Colours are **design tokens** in `index.css` (`--color-brand-50 ... --color-brand-900`, indigo), used as Tailwind classes such as `bg-brand-600`.
- Dark mode: a `dark` class on `<html>`; `useTheme` remembers the choice in `localStorage`; the first visit follows the device setting.
- Every input has a `<label>`; errors are linked with `aria-describedby`; scan results use `role="alert"` so screen readers announce them.
- Phase 8 measured every public page at 375 px (phone) in light and dark mode: no sideways scrolling, tap targets at least 24 px, and text contrast at least 4.5:1. Sixty-two grey hint texts, the dark-mode buttons and the green "Let in" panel were darkened or lightened to pass.

10. User guide (with screenshots)

This chapter shows how each kind of user works with EventHub. The screenshots come from the real application (demo data, dark mode).

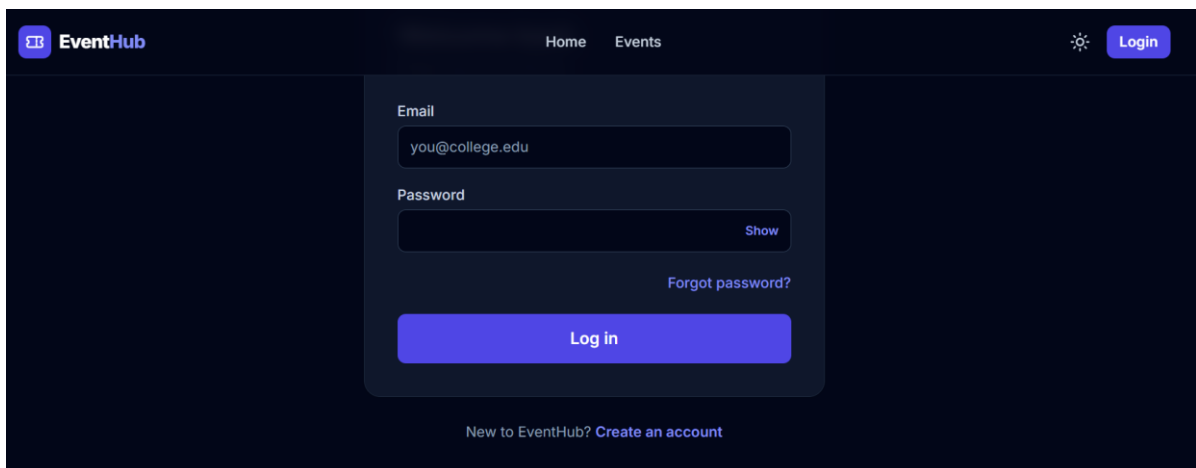
10.1 Visitor: finding events

1. Open the home page. Type in the search box or click a tag chip such as **robotics**.
2. The events page shows matching events. Use the filters on the left: tag, club, dates, sort order. The address bar keeps the filters, so the link can be shared.



All events filtered by the tag "robotics"

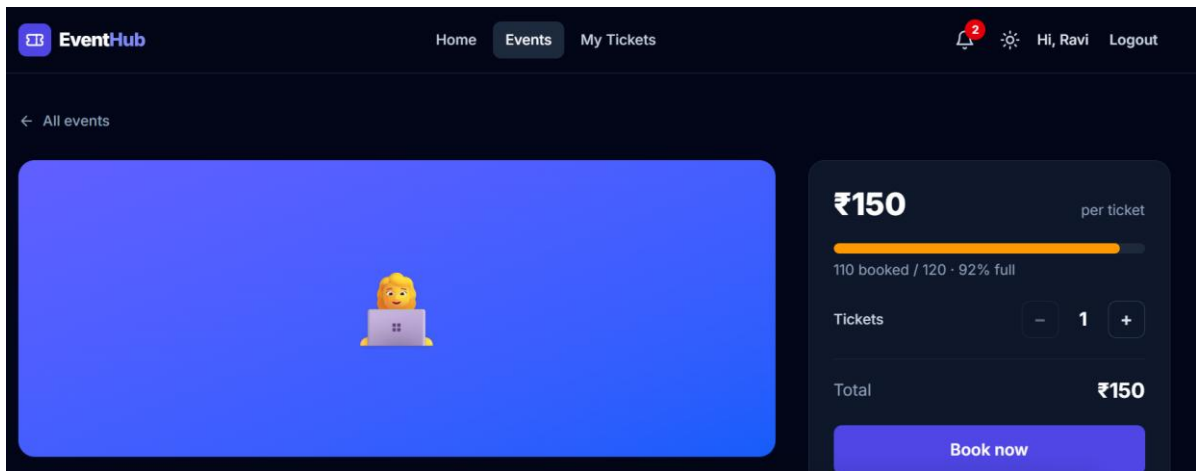
3. Click an event to see its details and the live seat bar. Pressing **Book now** without being logged in opens the login page, which brings you back to the event afterwards.



Login page - after login EventHub returns to the event you were looking at

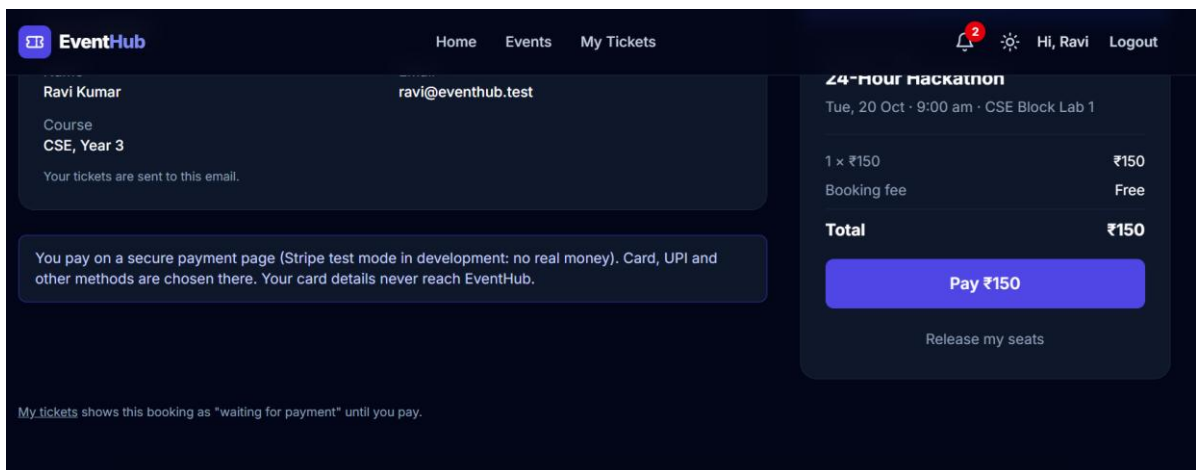
10.2 Student: booking and paying

1. On the event page, choose the number of tickets (1-10) and press **Book now**.



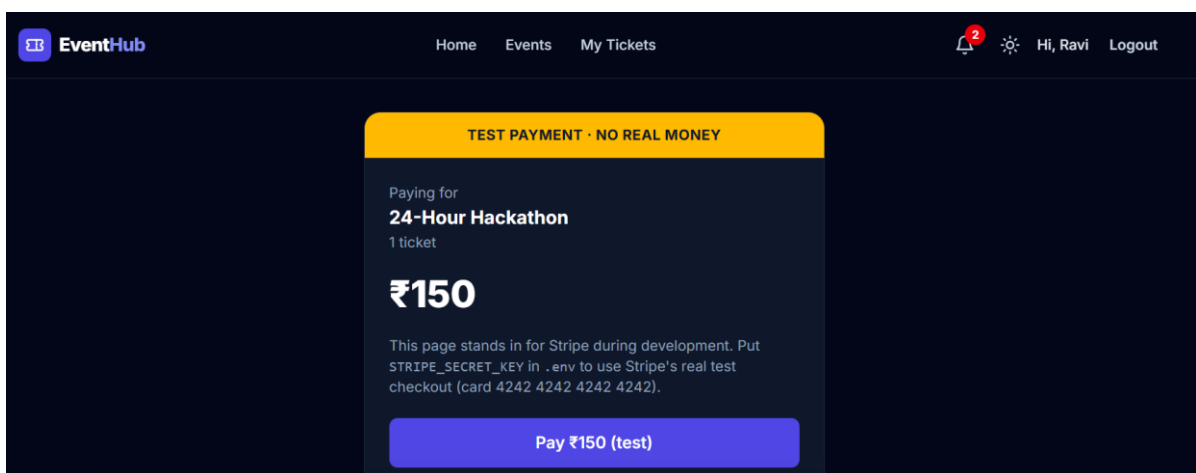
Event details: price, live "booked / total" bar, ticket stepper

- The checkout page shows your details and a **10-minute timer**. The seats are yours while it runs. Press **Pay**. (Press "Release my seats" if you changed your mind.)



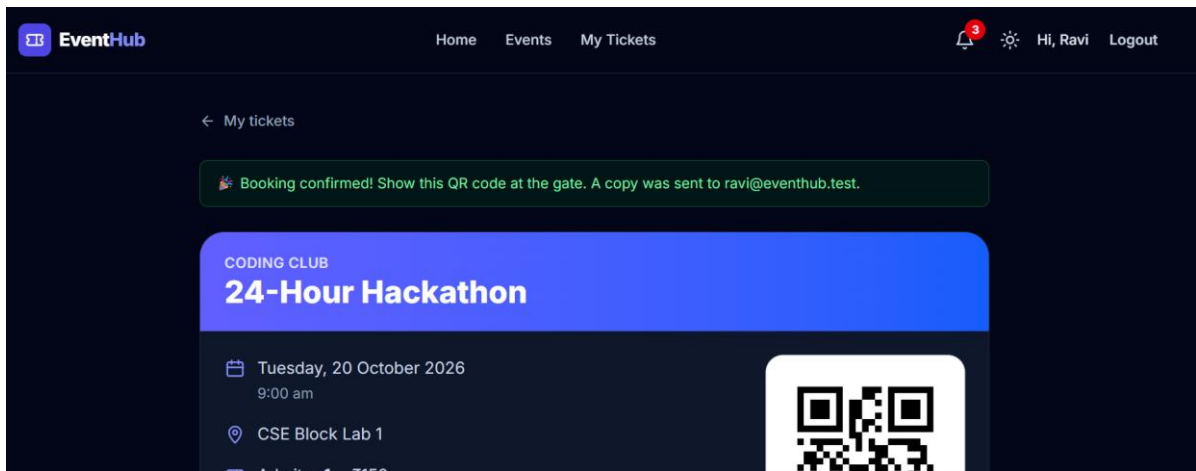
Checkout: seats held, total, Pay button

- In development you see EventHub's **test payment page** (no real money). With a Stripe test key you would see Stripe's own page instead. Press **Pay**.



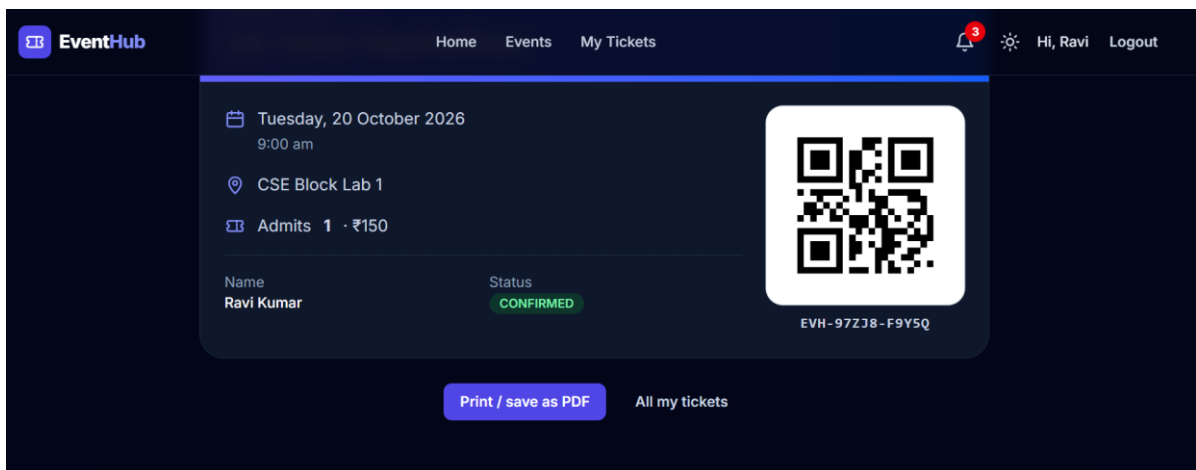
The built-in test payment page

4. "Payment received. You are going!" Press **Show my ticket**.



Booking confirmed with the QR code

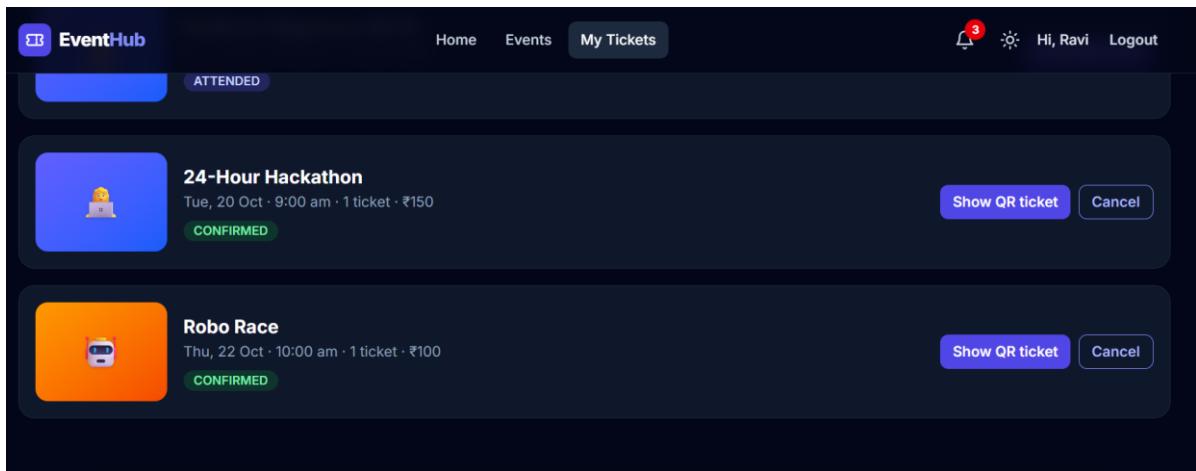
5. The ticket page shows the QR code and the ticket code. Print it or save it as PDF - or just show it on the phone at the gate.



The QR ticket

10.3 Student: my tickets, waitlist, notifications

- **My tickets** lists upcoming, past and cancelled bookings. A paid ticket can be cancelled before the event (the money is refunded); a ticket still waiting for payment shows "Pay now".

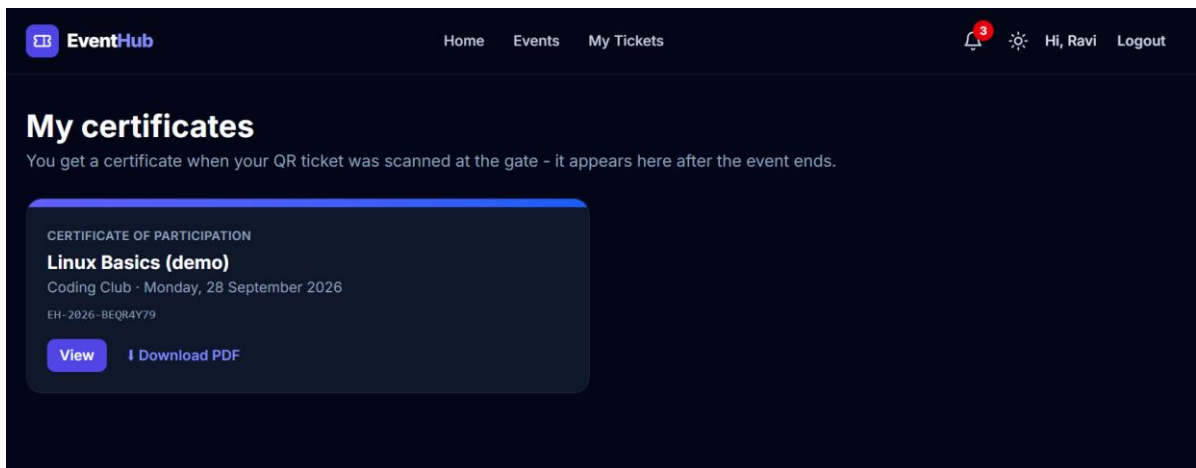


My tickets

- **Waitlist**: on a sold-out event press "Join the waitlist" and choose how many seats you need. The waitlist page shows your place ("You are #2"). When seats are offered, a card with a 30-minute countdown appears: press **Accept** to go to checkout.
- **Bell**: the bell in the top bar shows unread notifications (booking confirmed, seat offered, event approved, reminders). The same messages arrive by email (in development: the Mailpit inbox at <http://localhost:8025>).

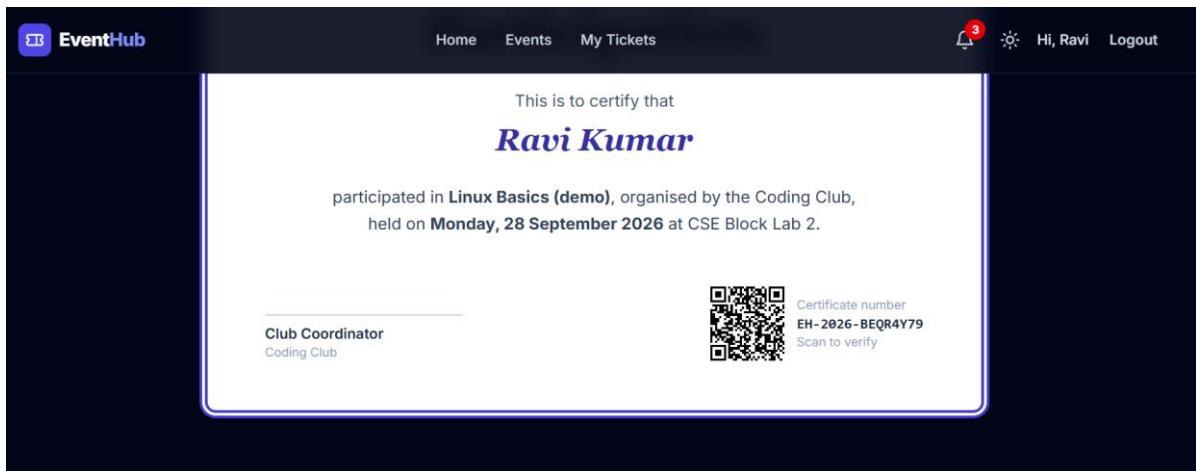
10.4 Student: certificates and feedback

After an event you attended (your ticket was scanned at the gate) is over, it appears under **Certificates**.



My certificates

Open it to see the certificate and download the PDF. The QR code in the corner opens the public verify page.

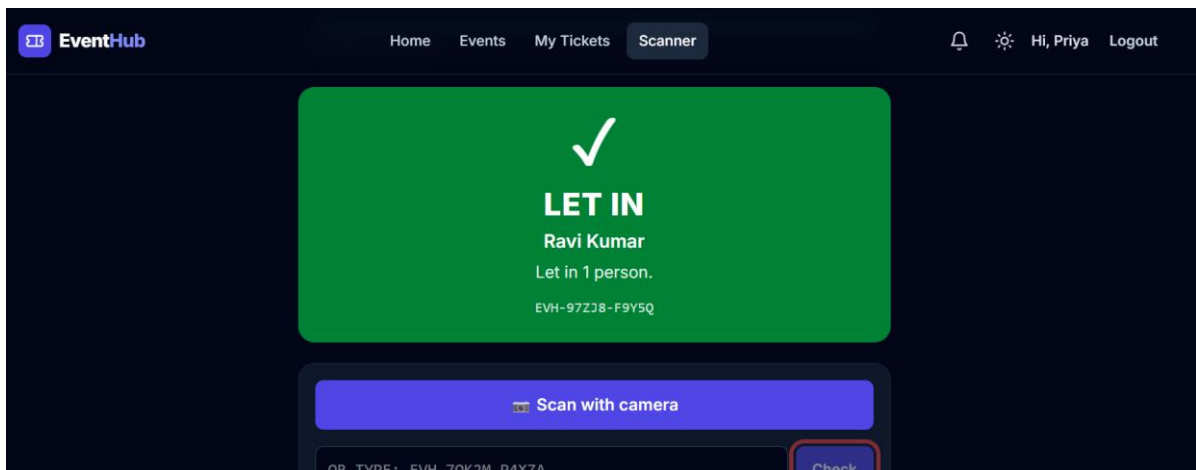


The certificate

On **My tickets** → **Past**, press **Rate** to give 1-5 stars and an optional comment. You can change your rating later.

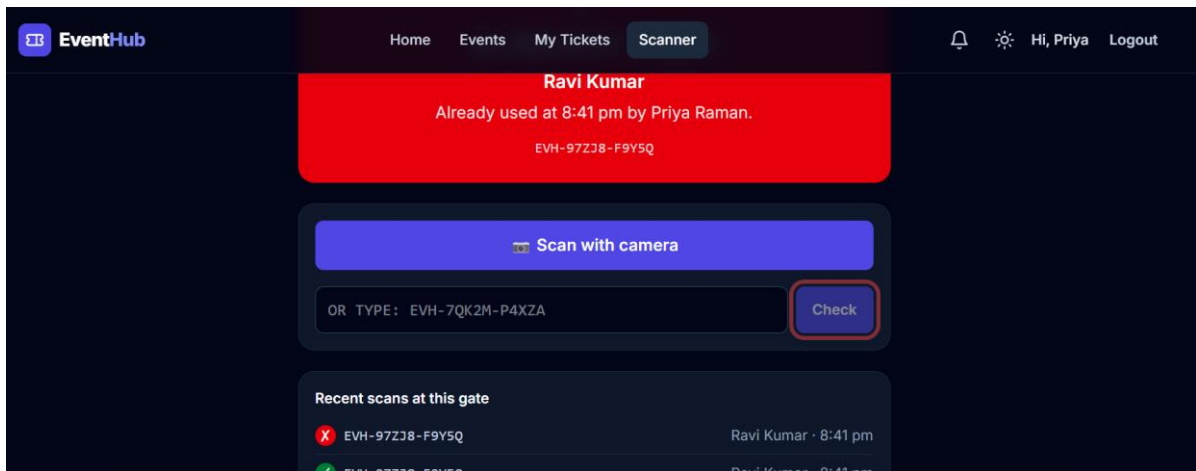
10.5 Volunteer: the gate scanner

1. Log in as a volunteer (for example Priya). Open **Scanner** in the top bar.
2. Choose the event you are on duty for. The live counter shows "checked in / booked" for all gates together.
3. Press **Scan with camera** and point it at the QR code - or type the code and press **Check**.



First scan: LET IN, with the name and number of people

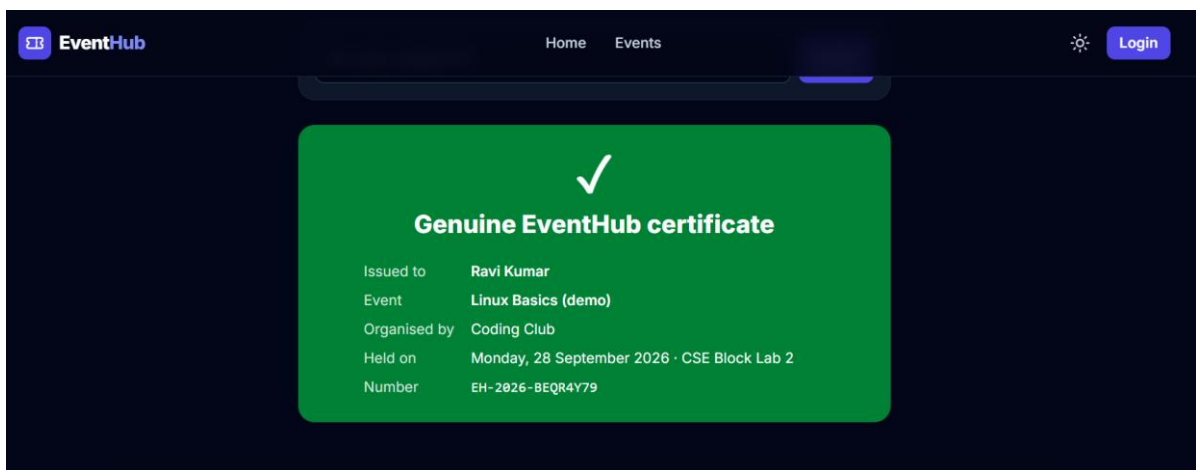
4. The answer is big and coloured: green **LET IN**, or red **ALREADY USED / STOP**. Phones vibrate on red.



The same ticket again: ALREADY USED, with the time and the volunteer's name

10.6 Anyone: verifying a certificate

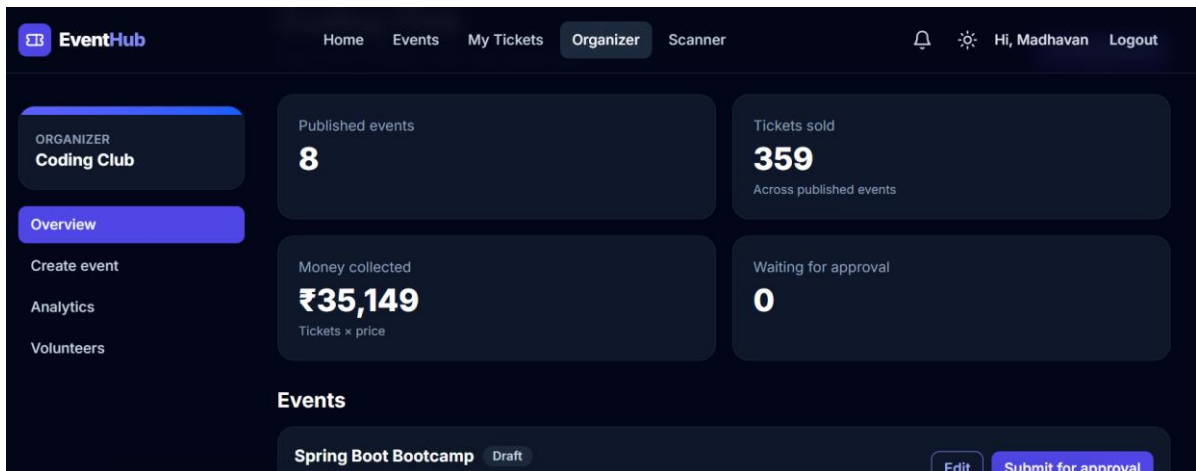
Open [/verify](#) (link "Verify a certificate" in the footer) and type the number, or scan the QR code on the certificate. No login is needed.



A genuine certificate on the public verify page

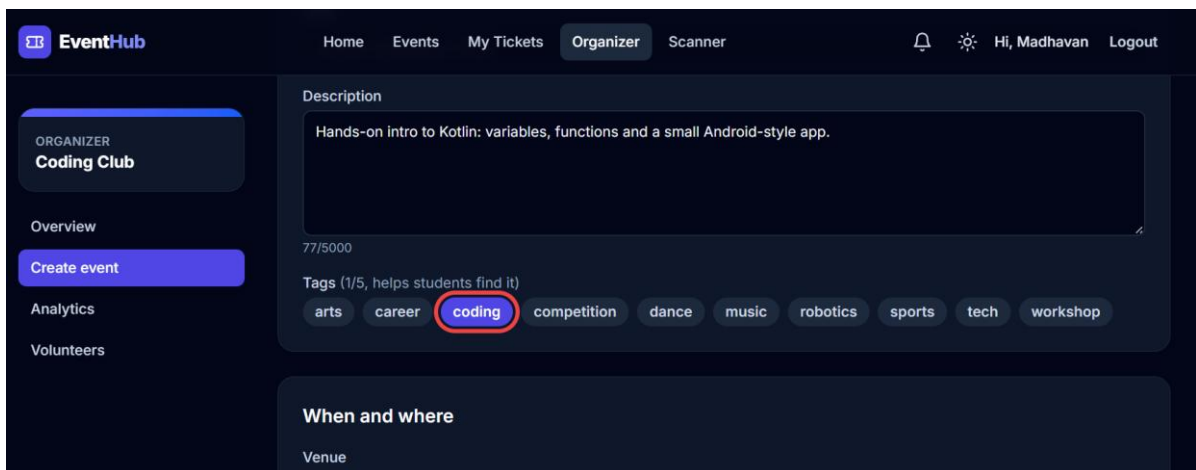
10.7 Organizer: creating and managing events

1. Log in as an organizer (Madhavan). The **Organizer** area shows the club's dashboard: published events, tickets sold, money, waiting approvals, and every event with its status.



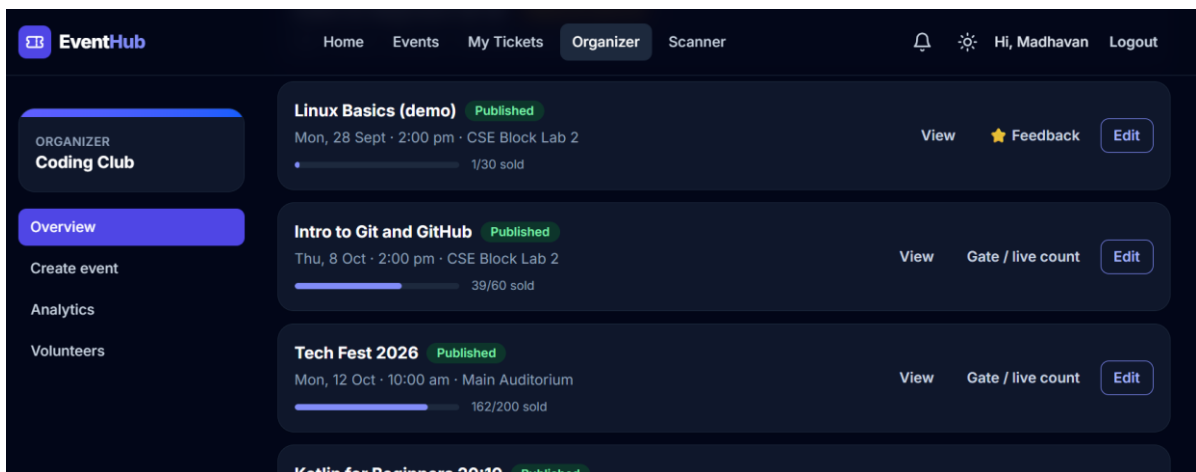
Organizer dashboard of the Coding Club

2. **Create event:** fill in title, description, venue, start and end, seats, price and up to 5 tags. The form checks each field as you go.



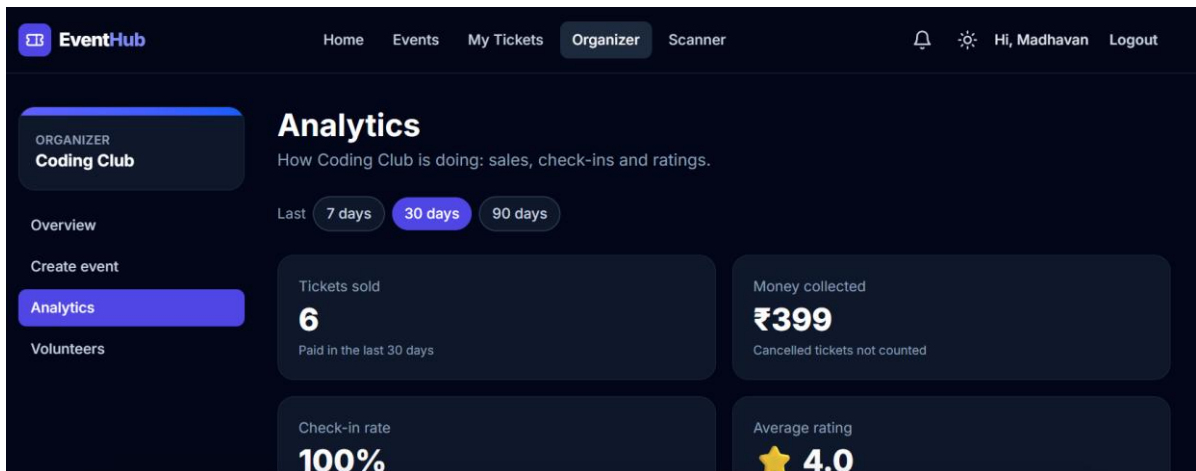
Create event form

3. Press **Save and submit** (or save as a draft first). The event waits for the admin. If the admin sends it back, the reason appears in red under the event.



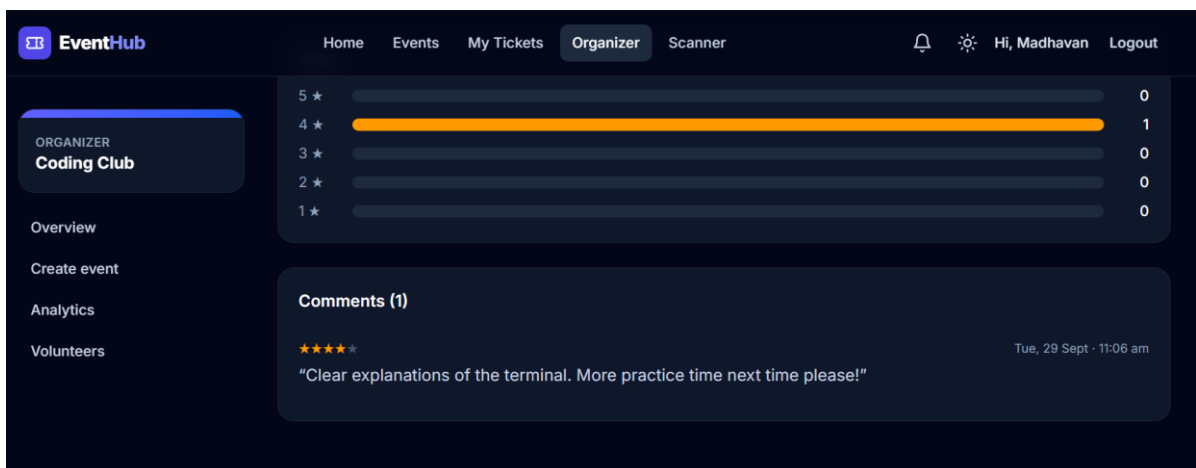
The organizer's event list with statuses

4. **Analytics:** tickets and money per day for the last 7/30/90 days, check-in rate and average rating.



Organizer analytics

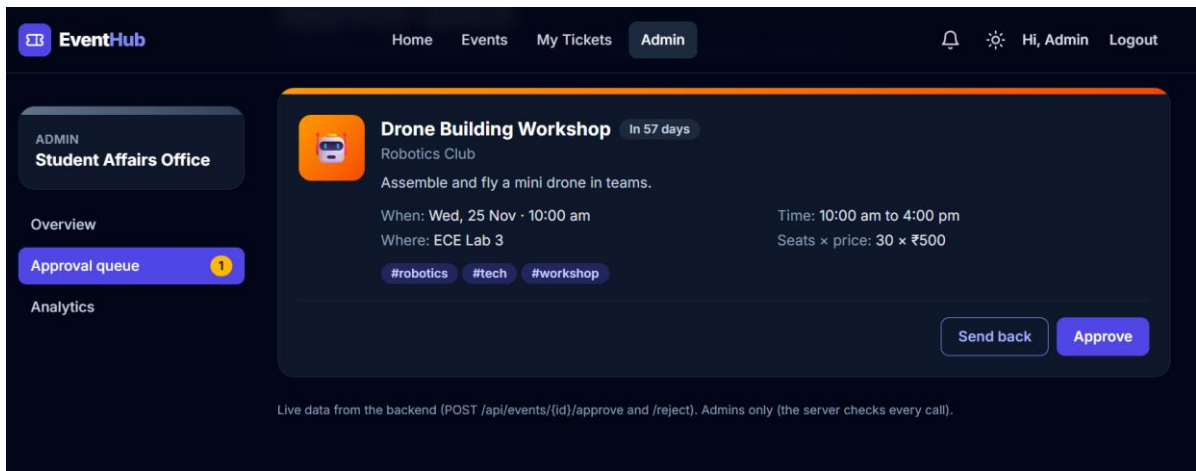
5. **Feedback** (on past events): average stars, how many gave each star, and the comments without names.



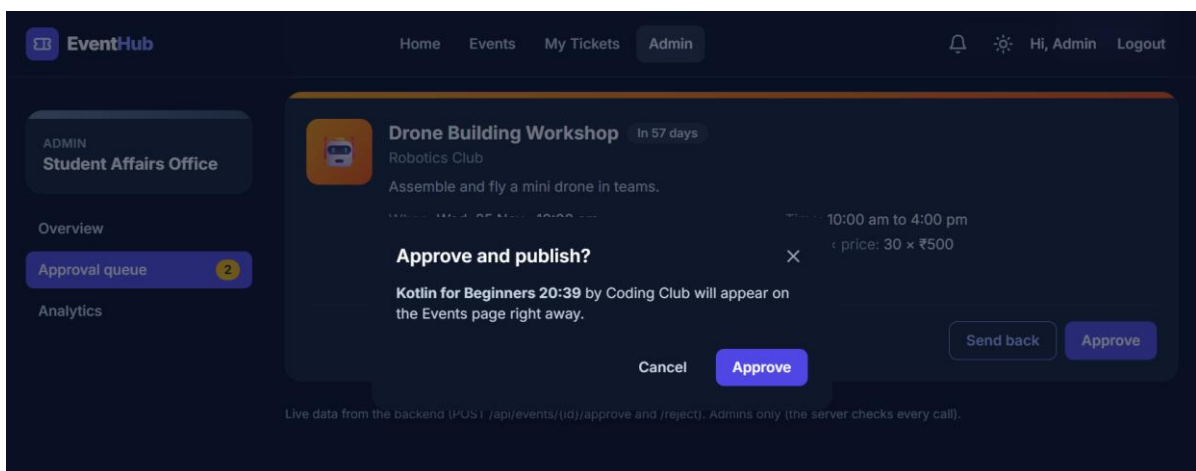
Feedback of Linux Basics (demo)

10.8 Admin: approvals, overview, analytics

1. Log in as the admin. **Approval queue** lists waiting events, soonest first. Press **Approve** (and confirm), or **Send back** with a reason.

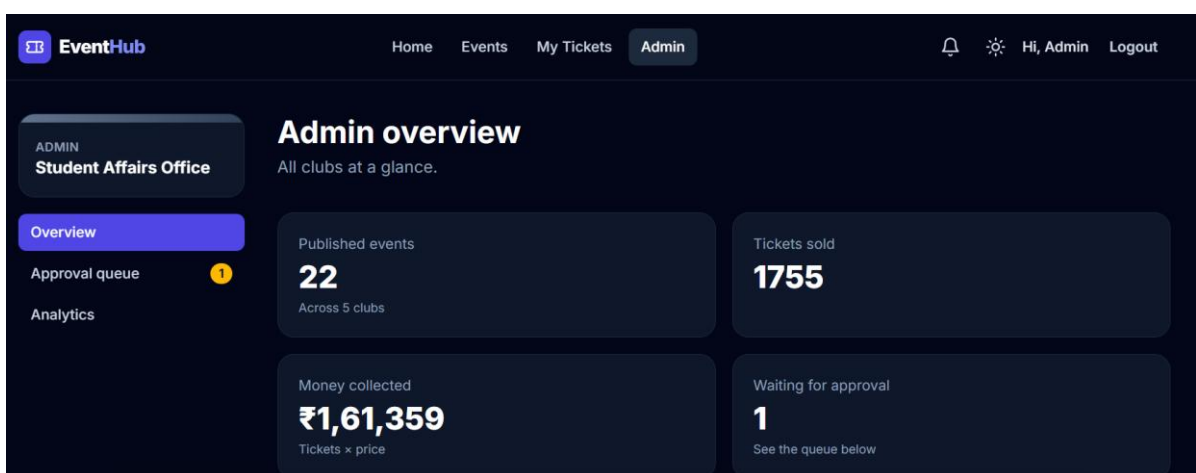


Approval queue



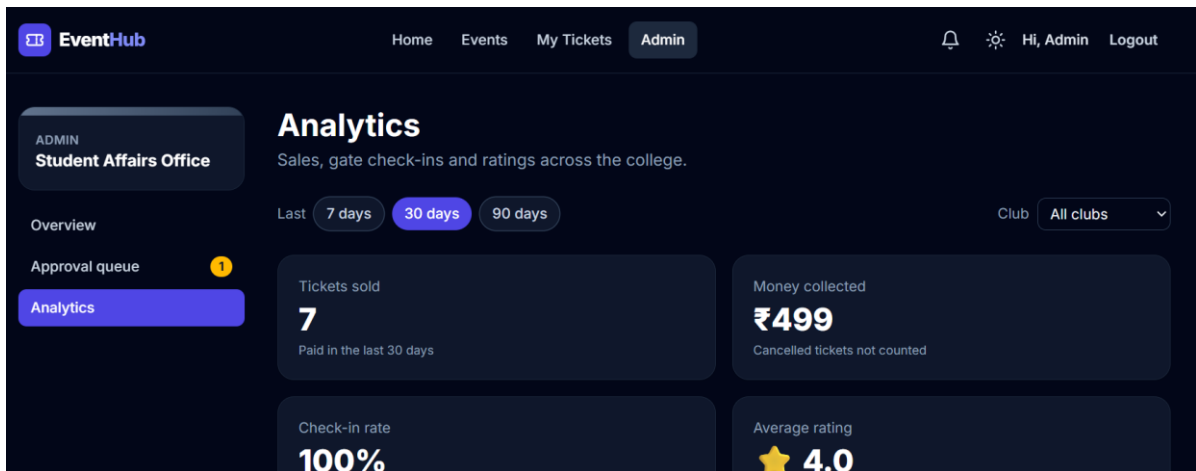
Confirming an approval

2. **Overview** shows totals per club and the activity log (who created, submitted, approved or sent back each event).



Admin overview

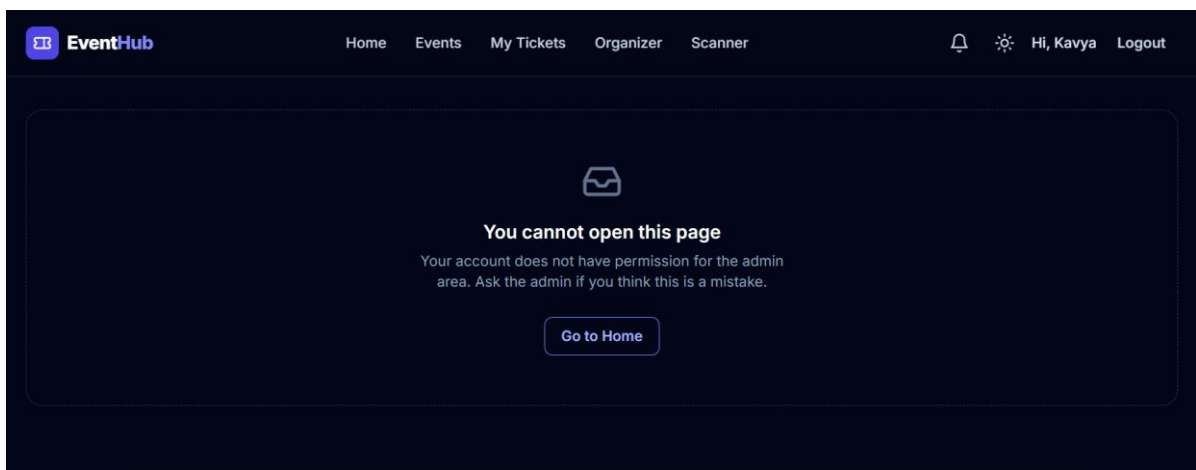
3. **Analytics** shows the same dashboard as organizers, for all clubs or one club.



Admin analytics for all clubs

10.9 What happens when someone is not allowed

Kavya organizes the Cultural Club. If she opens the admin area, the page says she has no permission - and even if she called the admin API directly, the server would answer 403 Forbidden.



A non-admin opening the admin area

11. Testing

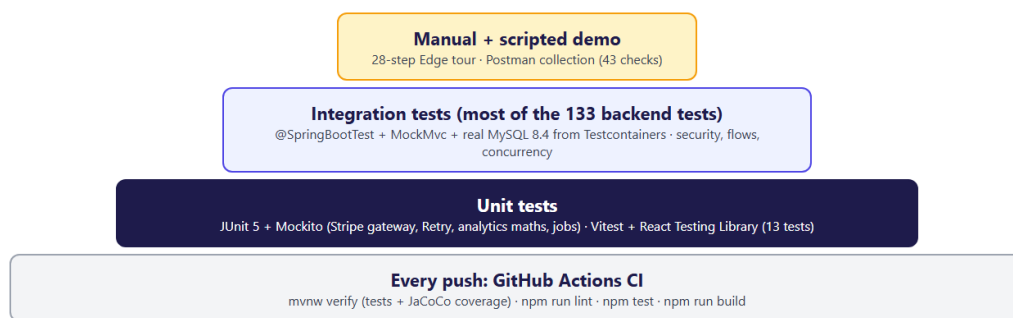
11.1 Why test automatically?

Every time the code changes, something that used to work can break. Clicking through 29 pages by hand after every change is slow and people forget steps. Automated tests are small programs that use the application and check the answers - in about two minutes, every time, on every push.

Real-life example: Automated tests are like the quality-control machine at a biscuit factory: every biscuit is weighed automatically; a person only looks when the machine rings the bell.

11.2 The testing pyramid

The testing pyramid used in EventHub



The testing pyramid used in EventHub

Level	What it checks	Speed	EventHub examples
Unit test	One class alone; everything around it is replaced by mocks	milliseconds	<code>StripePaymentGatewayTest</code> , <code>RetryTest</code> , <code>AnalyticsServiceTest</code> , <code>EventStatusTest</code>
Integration test	Several layers together, with a real database	seconds	<code>BookingFlowTest</code> , <code>GateFlowTest</code> , <code>SecurityRulesTest</code> , <code>WaitlistFlowTest</code>
Concurrency test	Many threads at the same moment	seconds	<code>BookingConcurrencyTest</code> (100 threads), <code>GateConcurrencyTest</code> (20 gates)
Component test (React)	One page or component in a fake browser	milliseconds	<code>LoginPage.test.jsx</code> , <code>ScannerPage.test.jsx</code> , <code>useCountdown.test.jsx</code>
End-to-end demo	The whole running system in a real browser	minutes	<code>tools/edge-tour.mjs</code> (28 steps)

11.3 The numbers

Measure	Value
Backend tests	133 (26 test classes)
Frontend tests	13 (4 test files)
Line coverage of the service classes	91%

Measure	Value
Line coverage of the whole backend	92%
Time for all backend tests	about 2 minutes (including starting MySQL)
CI	GitHub Actions, on every push, green

11.4 Unit tests with Mockito

A **mock** is a fake object that answers what the test tells it to. It lets us test the maths of **AnalyticsService** without a database: the test decides what the "database" returns and checks what the service calculates.

AnalyticsServiceTest.java (backend/src/test/java/com/eventhub/analytics/AnalyticsServiceTest.java)

```
@Test
void checkInRateCountsOnlyStartedEventsAndStarsAreWeightedByHowManyRated() {
    LocalDateTime yesterday = LocalDateTime.now().minusDays(1);
    LocalDateTime nextWeek = LocalDateTime.now().plusDays(7);
    when(queries.salesPerDay(any(), any())).thenReturn(List.of());
    when(queries.salesPerEvent(any(), any())).thenReturn(List.of(
        new EventSales(10L, "Git workshop", yesterday, 60, 40, 30, BigDecimal.ZERO),
        new EventSales(11L, "Robo Race", yesterday, 100, 10, 10, new BigDecimal("1000")),
        new EventSales(12L, "Tech Fest", nextWeek, 200, 150, 0, new
BigDecimal("15000"))));
    when(queries.ratings(List.of(10L, 11L, 12L))).thenReturn(List.of(
        new EventRating(10L, 4.0, 3), // 3 people gave 4 stars
        new EventRating(11L, 2.0, 1))); // 1 person gave 2 stars

    AnalyticsResponse r = service.dashboard(null, 30);

    // started events only: (30 + 10) came of (40 + 10) sold = 80 %; Tech Fest (not started)
    // is left out
    assertThat(r.totals().checkedIn()).isEqualTo(40);
    assertThat(r.totals().expectedAtGate()).isEqualTo(50);
    assertThat(r.totals().checkInRate()).isEqualTo(0.8);
    // (4+4+4+2) / 4 ratings = 3.5, NOT (4.0 + 2.0) / 2 events = 3.0
    assertThat(r.totals().averageRating()).isEqualTo(3.5);
    assertThat(r.totals().ratings()).isEqualTo(4);

    assertThat(r.events().extracting(AnalyticsResponse.EventRow::started).containsExactly(true,
true, false);
    assertThat(r.events().get(2).averageRating()).isNull(); // Tech Fest: no ratings yet
}
```

Line by line:

1. **@Mock AnalyticsQueries queries** - a fake repository.
2. **when(queries.salesPerEvent(any(), any())).thenReturn(...)** - "if the service asks for sales per event, give it these three rows".
3. **service.dashboard(null, 30)** - run the real code.
4. **assertThat(...).isEqualTo(0.8)** - (30 + 10) people came out of (40 + 10) tickets for the started events = 80%. The future event is correctly left out.
5. The weighted average: (4 + 4 + 4 + 2) / 4 ratings = 3.5, not (4.0 + 2.0) / 2 events = 3.0.

The Stripe gateway was never run against real Stripe in this project, so its unit test is especially valuable. Mockito's `mockStatic` replaces Stripe's static methods for one test and captures exactly what EventHub would send:

StripePaymentGatewayTest.java (backend/src/test/java/com/eventhub/payment/StripePaymentGatewayTest.java)

```
@Test
void checkoutAsksStripeForTheRightAmountInPaiseAndTheRightReturnPages() {
    ArgumentCaptor<SessionCreateParams> sent =
    ArgumentCaptor.forClass(SessionCreateParams.class);
    ArgumentCaptor<RequestOptions> options = ArgumentCaptor.forClass(RequestOptions.class);
    // make the fake session FIRST: creating a mock in the middle of another when(...)
    confuses Mockito
    Session created = session("cs_test_1", "https://checkout.stripe.com/c/pay/cs_test_1",
    "unpaid");
    sessions.when(() -> Session.create(sent.capture(),
    options.capture())).thenReturn(created);

    CheckoutSession checkout = gateway.createCheckout(booking());

    assertThat(checkout.sessionId()).isEqualTo("cs_test_1");

    assertThat(checkout.redirectUrl()).isEqualTo("https://checkout.stripe.com/c/pay/cs_test_1");

    SessionCreateParams p = sent.getValue();
    LineItem item = p.getLineItems().getFirst();
    assertThat(item.getQuantity()).isEqualTo(3L);
    assertThat(item.getPriceData().getUnitAmount()).isEqualTo(10050L); // ₹100.50 = 10050
    paise
    assertThat(item.getPriceData().getCurrency()).isEqualTo("inr");
    assertThat(item.getPriceData().getProductData().getName()).isEqualTo("Robo Race");
    assertThat(p.getMode()).isEqualTo(SessionCreateParams.Mode.PAYMENT);
    assertThat(p.getMetadata()).containsEntry("booking_id", "42"); // the webhook finds the
    booking with it
    assertThat(p.getClientReferenceId()).isEqualTo("42");
    assertThat(p.getCustomerEmail()).isEqualTo("ravi@eventhub.test");
    assertThat(p.getSuccessUrl())

    .isEqualTo("http://localhost:5173/payment/success?booking=42&session={CHECKOUT_SESSION_ID}");

    assertThat(p.getCancelUrl()).isEqualTo("http://localhost:5173/payment/cancelled?booking=42");
    // Stripe refuses pages that close sooner than 30 minutes
    assertThat(Instant.ofEpochSecond(p.getExpiresAt()))
        .isBetween(Instant.now().plus(30, ChronoUnit.MINUTES), Instant.now().plus(32,
    ChronoUnit.MINUTES));
    assertThat(options.getValue().getApiKey()).isEqualTo(KEY);
}
```

11.5 Integration tests with Testcontainers

Integration tests start the whole Spring application and call it through **MockMvc**, exactly like the browser would, including the login cookie and the CSRF header. They need a real MySQL - an in-memory database behaves differently (locks, isolation, SQL details).

Testcontainers starts a brand-new MySQL 8.4 container in Docker once per test run and removes it at the end:

MySQLTestDatabase.java (backend/src/test/java/com/eventhub/MySQLTestDatabase.java)

```
/**
 * Phase 8 step 2 - Testcontainers: the tests start their OWN MySQL 8.4 in Docker (same
 * version as
 * docker-compose and production), fresh and empty; Flyway then builds the tables from V1..V9.
 *
 * before: tests used the database "eventhub_test" inside our dev container - it had to be
 * created
```

```

*      by hand, leftovers from old runs stayed, and CI needed its own MySQL service.
*      now: `mvnw test` only needs Docker running. The container starts ONCE for the whole
test run
*      (about 10 s) and Testcontainers removes it when the run ends.
*
* Registered in src/test/resources/META-INF/spring.factories, so every Spring test context
* (@SpringBootTest, @DataJpaTest ...) gets it without any change in the test classes.
*/
public class MySqlTestDatabase implements
ApplicationContextInitializer<ConfigurableApplicationContext> {

    /** static = one container per test run, shared by every test context (like a singleton).
*/
    private static final MySQLContainer MYSQL = new MySQLContainer("mysql:8.4")
        .withDatabaseName("eventhub_test")
        .withUsername("eventhub")
        .withPassword("test-only-password");

    @Override
    public void initialize(ConfigurableApplicationContext context) {
        synchronized (MYSQL) {
            if (!MYSQL.isRunning()) {
                MYSQL.start();
            }
        }
        // addFirst = these win over application.yml (which points at the dev database)
        context.getEnvironment().getPropertySources().addFirst(new
MapPropertySource("testcontainers-mysql", Map.of(
            "spring.datasource.url", MYSQL.getJdbcUrl(),
            "spring.datasource.username", MYSQL.getUsername(),
            "spring.datasource.password", MYSQL.getPassword())));
    }
}

```

It is registered in `src/test/resources/META-INF/spring.factories`, so every test class uses it automatically. Most integration tests are `@Transactional`: each test's changes are rolled back at the end, so tests never disturb each other.

Example of an integration test - the security rule "Swagger and Actuator are for the admin only":

SecurityRulesTest.java (backend/src/test/java/com/eventhub/auth/SecurityRulesTest.java)

```

@Test
void swaggerAndActuatorAreForTheAdminOnly() throws Exception {
    for (String url : new String[] { "/v3/api-docs", "/swagger-ui.html", "/actuator/info" }) {
        mvc.perform(get(url)).andExpect(status().isUnauthorized());
    }
    // visitor
    mvc.perform(get(url).with(accounts.as(accounts.student()))).andExpect(status().isForbidden());

    mvc.perform(get(url).with(accounts.as(accounts.organizerOf(1L)))).andExpect(status().isForbidden());
}
    mvc.perform(get("/v3/api-docs").with(accounts.as(accounts.admin())))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.info.title").value("EventHub API"));

    mvc.perform(get("/actuator/info").with(accounts.as(accounts.admin()))).andExpect(status().isOk());
}

```

11.6 Concurrency tests

BookingConcurrencyTest.java (backend/src/test/java/com/eventhub/booking/BookingConcurrencyTest.java)

```

@RepeatedTest(3)
void hundredStudentsTenSeatsExactlyTenBookings() throws Exception {

```

```

AtomicInteger booked = new AtomicInteger();
AtomicInteger soldOut = new AtomicInteger();
ConcurrentLinkedQueue<Throwable> unexpected = new ConcurrentLinkedQueue<>();
CountDownLatch startGun = new CountDownLatch(1);

ExecutorService pool = Executors.newFixedThreadPool(STUDENTS);
List<Future<?>> runs = new ArrayList<>();
for (Long userId : userIds) {
    runs.add(pool.submit(() -> {
        try {
            startGun.await(); // everybody waits here ...
            service.hold(userId, eventId, 1);
            booked.incrementAndGet();
        }
        catch (BusinessRuleException ex) {
            if (ex.getMessage().contains("sold out") || ex.getMessage().contains("Only"))
            {
                soldOut.incrementAndGet();
            }
            else {
                unexpected.add(ex);
            }
        }
        catch (Throwable ex) {
            unexpected.add(ex);
        }
        return null;
    }));
}
startGun.countDown(); // ... and all 100 press "Book" together
for (Future<?> run : runs) {
    run.get(60, TimeUnit.SECONDS);
}
pool.shutdown();

assertThat(unexpected).as("unexpected errors").isEmpty();
assertThat(booked.get()).as("successful bookings").isEqualTo(SEATS);
assertThat(soldOut.get()).as("told 'sold out'").isEqualTo(STUDENTS - SEATS);
assertThat(events.findById(eventId).orElseThrow().getAvailableSeats()).as("seats
left").isZero();
assertThat(jdbc.queryForObject("select count(*) from bookings where event_id = ?",
Integer.class, eventId))
    .as("rows in bookings").isEqualTo(SEATS);
}

```

- 100 threads wait at a **CountDownLatch** and are released at the same moment.
- The test is **not @Transactional** (each thread needs its own real transaction) and cleans up after itself.
- **@RepeatedTest(3)** runs it three times, because concurrency bugs do not always show up on the first try.
- **Mutation check:** when **@Version** was removed from **Event** on purpose, this test failed with 96-99 bookings. A test that can fail is a test that proves something.

11.7 React component tests

Vitest runs the tests in **jsdom**, a fake browser. React Testing Library finds elements the way a user would (by label, role or text) and **user-event** types and clicks like a person.

LoginPage.test.jsx (frontend/src/pages/LoginPage.test.jsx)

```

it('after login goes back to the page in ?next= (and trims the email)', async () => {
    vi.mocked(loginRequest).mockResolvedValue(ravi)

```

```
const user = renderLogin('/login?next=/events/15')
await fillAndSubmit(user, ' ravi@eventhub.test ', 'secret123')

expect(await screen.findByText('Event page')).toBeInTheDocument()
expect(loginRequest).toHaveBeenCalledWith({ email: 'ravi@eventhub.test', password:
'secret123' })
})
```

The API module is replaced by a mock (`vi.mock('../api/auth', ...)`), so the test controls the server's answer.

`useCountdown.test.jsx` uses **fake timers**: the test moves the clock by 65 seconds instantly and checks that "10:00" became "08:55".

11.8 Continuous integration (GitHub Actions)

ci.yml (github/workflows/ci.yml, lines 1-48)

```
# CI = GitHub builds and tests EventHub automatically on every push.
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  backend:
    name: Backend (Spring Boot)
    runs-on: ubuntu-latest
    # no MySQL service needed: the tests start their own MySQL with Testcontainers (Docker is
on the runner)
    defaults:
      run:
        working-directory: backend
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-java@v5
        with:
          distribution: temurin
          java-version: '25'
          cache: maven
      - run: ./mvnw -B verify
      # Phase 8: the JaCoCo coverage report, downloadable from the run's page (Artifacts)
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: coverage-report
          path: backend/target/site/jacoco/

  frontend:
    name: Frontend (React)
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: frontend
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-node@v5
        with:
          node-version: '24'
          cache: npm
          cache-dependency-path: frontend/package-lock.json
      - run: npm ci
      - run: npm run lint
      - run: npm test # Phase 8: Vitest + React Testing Library
      - run: npm run build
```

Every push to GitHub starts two jobs on fresh Ubuntu machines: the backend (Java 25, `mvnw verify` - all tests with Testcontainers - and the JaCoCo report as a downloadable artifact) and the frontend (Node 24: `npm ci`, lint, test, build). The green or red result appears next to each commit.

11.9 Bugs the tests found

Bug	Found by	Fix
Two bookings for the last seat	BookingConcurrencyTest	<code>@Version</code> + retry
MySQL deadlock between two bookings	BookingConcurrencyTest	update the event row before inserting the booking
A second gate re-read the ticket and still saw it unused	GateConcurrencyTest	decide from the UPDATE result
CSRF token changed on every request (second parallel request failed)	browser test	keep one token (<code>sessionAuthenticationStrategy</code>)
Jackson 3 refused a missing number	ErrorHandlingTest	<code>Integer</code> + <code>@NotNull</code> instead of <code>int</code>
Hibernate refused TINYINT	start-up validation	<code>@JdbcTypeCode(SqlTypes.TINYINT)</code>
JWT test failed 1 run in 16	CI	change the first signature letter, not the last (base64 padding bits)
Scanner page went blank after the first scan	Edge demo tour	import the moved colour table; new ScannerPage test; lint rule <code>no-undef</code>

11.10 How to run the tests

```

cd backend
mvnw test                all backend tests
mvnw test -Dtest=BookingConcurrencyTest  one test class
start target\site\jacoco\index.html      open the coverage report

cd frontend
npm test                  all React tests once
npm run test:watch        re-run on every save

```

12. Security in depth

12.1 The main threats and the answers

Threat	What an attacker tries	EventHub's defence
Stolen password database	read passwords from a leaked copy	BCrypt hashes with salt; passwords never stored or logged
Password guessing	try thousands of passwords	5 failures per email + address → 15-minute lock (429)
Stolen login token (XSS)	a script on the page reads the token	JWT in an httpOnly cookie - scripts cannot read it
Forged request (CSRF)	another website makes your browser post to EventHub	SameSite=Lax cookie + CSRF token header that other sites cannot read
Doing other people's work	an organizer edits another club's event; a student opens admin URLs	role and club checks on every request (SecurityConfig , @PreAuthorize , ClubAccess) → 403
Guessing ids	walk through /tickets/1, /2, /3 ...	ownership checks (404 for other people's bookings); random ticket and certificate codes
Fake payment notification	call the webhook pretending to be Stripe	Stripe-Signature (HMAC with the webhook secret) checked first
Replayed payment notification	send the same "paid" message again	idempotency table processed_payment_events
Fake ticket at the gate	a screenshot or a made-up code	server-side check; each code works once; codes are random
Fake certificate	edit a PDF	public verify page shows the truth for each number
Leaking secrets	passwords in GitHub	.env in .gitignore ; history scanned; only placeholders in .env.example
Learning the system	reading the API map and server details	Swagger and Actuator admin-only; 500 errors hide internal messages
Open redirect	/login?next=https://evil.com	safeNext only allows paths on this site

12.2 Authentication vs authorization

- **Authentication** = *who are you?* - proven by the password once, then by the signed JWT cookie on every request.
- **Authorization** = *what may you do?* - global roles (STUDENT, ADMIN) from the token; club roles (ORGANIZER, VOLUNTEER) from the database.

Real-life example: Authentication is showing your ID card at the college gate. Authorization is whether that ID card opens the chemistry lab.

12.3 Why the JWT is in a cookie

	localStorage + Authorization header	httpOnly cookie (EventHub)
Can a malicious script read it?	Yes	No
Sent automatically?	No (code must add it)	Yes (so CSRF protection is needed)
Needs CSRF protection?	No	Yes - done with the XSRF token

EventHub chose the cookie because stealing a token is worse than a forged request that CSRF protection already blocks.

12.4 The login lock in action

1. Ravi's password is typed wrongly 4 times from one laptop - each answer: 401 "Wrong email or password."
2. The 5th wrong try locks the pair (ravi@eventhub.test, that laptop's address) for 15 minutes.
3. Even the right password now gets 429 "Too many wrong passwords. Please wait 15 minutes and try again." with a **Retry-After** header (seconds to wait).
4. Ravi can still log in from his phone on another network, and other students on the same Wi-Fi are not affected.
5. A successful login resets the counter.

12.5 Security checklist (Phase 8)

- Swagger UI, `/v3/api-docs` and Actuator (except `/actuator/health`) require ADMIN - tested by `SecurityRulesTest`.
- `/actuator/health` shows only UP/DOWN to visitors; database and disk details only to the admin.
- A scan of the code **and the whole git history** found no secrets; `.env` was never committed.
- The JWT secret must be at least 32 characters, and in production there is no default: the application refuses to start without it.
- Production refuses the fake payment gateway.
- Cookies are **Secure** in production (HTTPS only).

13. Troubleshooting

Each entry: what you see, why it happens, and how to fix it. Most of them really happened while building EventHub.

13.1 Setup and start-up

What you see	Why	Fix
<code>docker compose up</code> → "error during connect" / "cannot find the file specified"	Docker Desktop is not running	Start Docker Desktop, wait for "Engine running", try again
Backend: "Communications link failure"	MySQL container not started or still starting	<code>docker compose up -d</code> , then <code>docker compose ps</code> until mysql is "healthy"
Backend: "Access denied for user 'eventhub'"	<code>.env</code> passwords differ from the ones the MySQL volume was created with	Use the original passwords, or reset: <code>docker compose down -v</code> (deletes the data!) and start again
Backend: "Could not resolve placeholder 'MYSQL_USER'"	No <code>.env</code> file, or the backend was started from a different folder	Create <code>.env</code> from <code>.env.example</code> ; start the backend from the <code>backend</code> folder (it imports <code>../.env</code>)
Backend: "app.jwt.secret must be at least 32 characters"	<code>JWT_SECRET</code> too short	Generate one: <code>python -c "import secrets; print(secrets.token_hex(32))"</code>
Backend: "Port 8080 was already in use"	Another backend is still running (IntelliJ + command line)	Stop the other one; on Windows find it with <code>netstat -ano</code> (look for :8080) and stop it with <code>taskkill /PID <pid> /F</code>
Backend: "Schema-validation: wrong column type"	An entity field does not match the table	Fix the entity (e.g. <code>@JdbcTypeCode(SqlTypes.TINYINT)</code>) or add a new migration - never edit an old one
Backend: "Migration checksum mismatch"	An already-applied migration file was edited	Undo the edit and put the change in a new V10 file
Frontend: blank page, "Backend: DOWN" in the footer	Spring Boot not running or crashed	Check the backend console; open <code>http://localhost:8080/api/health</code>
<code>npm run dev</code> → "Port 5173 is in use"	A second Vite is running	Close it, or accept the other port Vite suggests
Vite does not pick up a change	The file watcher missed a very fast second save	Save the file again or restart <code>npm run dev</code>
<code>mvnw</code> "Permission denied" (Linux CI)	The wrapper lost its executable bit	<code>git update-index --chmod=+x backend/mvnw</code> and commit
IntelliJ will not start (".port" lock error)	A crashed IntelliJ left a lock file	Close all IntelliJ processes, delete the <code>.port</code> file in the IntelliJ config folder, start again

13.2 Login and security

What you see	Why	Fix
"Wrong email or password." with the right password	A typo, an extra space from copy-paste, or the demo accounts were created with an older <code>DEMO_PASSWORD</code>	Copy exactly; the seeder only creates missing accounts, it never changes an existing password
429 "Too many wrong passwords"	5 wrong tries from this address	Wait 15 minutes (or restart the backend in development - the lock is kept in memory)
403 on every POST from Postman	No CSRF header	First call <code>GET /api/auth/me</code> , copy the <code>XSRF-TOKEN</code> cookie value into an <code>X-XSRF-TOKEN</code> header
403 "You do not have permission to do this."	Logged in, but not the right role or club	Log in as the right user (e.g. the organizer of that club)
Logged out after some hours	The JWT expires after 8 hours	Log in again (by design)
Swagger page gives 401	Swagger is admin-only	Log in as the admin in the React app, then open Swagger in the same browser

13.3 Booking, payment and gate

What you see	Why	Fix
"You already have seats for ..."	One active booking per student per event	Open My tickets; cancel the old one first if needed
The hold timer ran out	Not paid within 10 minutes	Book again; the seats were returned to other students
Payment done, page keeps waiting	The webhook has not arrived (Stripe CLI not running)	The page calls "verify" automatically; with Stripe, run <code>stripe listen --forward-to localhost:8080/api/payments/stripe/webhook</code>
Scanner: "INVALID - not a ticket for this event"	Wrong event chosen, or the ticket is for another event	Pick the right event in "Gate duty for"
Scanner camera does not start	No camera, or the browser blocked it	Allow camera access, or type the code
No certificate after the event	The ticket was never scanned at the gate, or the event is not over yet	Only attendees get certificates (by design)
No email arrived	Development sends to Mailpit, not real mail	Open <code>http://localhost:8025</code>

13.4 Tests

What you see	Why	Fix
"Could not find a valid Docker environment"	Testcontainers needs Docker	Start Docker Desktop
Tests fail on Windows with file-locked errors	The backend is running from the same <code>target</code> folder	Stop the backend while running <code>mvnw test</code>

What you see	Why	Fix
A test passes alone but fails with the others	Data left behind by a non-transactional test	Non-transactional tests must clean up (see <code>BookingConcurrencyTest.cleanup</code>)
Vitest: "unhandled rejection" although the page catches it	Vitest 5 quirk with <code>mockReset()</code> + <code>mockRejectedValue</code>	Use <code>vi.clearAllMocks()</code> in <code>beforeEach</code>
Lint passes but the page crashes with "X is not defined"	The undefined-variable rule was off	Now fixed: <code>.oxlintrc.json</code> has <code>"no-undef": "error"</code>

13.5 How to investigate any problem

1. **Read the message.** The backend's Problem Details `detail` usually says exactly what is wrong.
2. **Browser DevTools** (F12): Console for JavaScript errors, Network for the failing request and its answer.
3. **Backend console:** the full exception and, in development, every SQL statement (`show-sql: true`).
4. **Reproduce with a test.** Write a failing test first, then fix the code until it passes - the bug can never come back unnoticed.

14. Lessons learned

These are the most valuable lessons of the project - good material for interviews.

14.1 Technical lessons

1. **The server must own every rule.** The page can hide a button, but anyone can call the API with Postman. Every rule lives in a service and is tested there.
2. **Concurrency is real.** "Read, then write" breaks when two people do it at once. Optimistic locking (`@Version`) with a retry fixes it without slow locks - and a test with 100 threads proves it.
3. **Lock order prevents deadlocks.** Two transactions touching the same rows in a different order can wait for each other forever. Always update the event row first.
4. **One step is safer than two.** "Check if unused, then mark as used" is two steps with a gap; `UPDATE ... WHERE checked_in_at IS NULL` is one step with no gap.
5. **Know your isolation level.** Under REPEATABLE READ a transaction keeps seeing its first snapshot; decide from the result of your own UPDATE instead of reading again.
6. **Do side effects after commit.** Emails and other outside actions must wait until the database change is final (outbox idea).
7. **Idempotency.** Anything that can arrive twice (webhooks, double clicks) must give the same result the second time: use unique keys.
8. **Let the database protect itself.** UNIQUE, CHECK and foreign keys stop wrong data even when Java has a bug.
9. **Random, not sequential, public codes.** Ticket and certificate numbers must not be guessable.
10. **Measure before optimising.** Paging, `@EntityGraph`, `@BatchSize`, a 60-second cache and code splitting were each added for a measured reason.

14.2 Working lessons

1. **Small steps, each tested and committed.** 60+ commits, CI green nearly all the time.
2. **Checklists with "done when".** Each phase ended only when its checks passed.
3. **A test for every bug.** The flaky JWT test and the scanner crash each got a test that would have caught them.
4. **Automate the demo.** The Edge tour found a real bug that unit tests missed, and it produced every screenshot in this guide.
5. **Write for the reader.** Comments in simple English explain *why*, not only *what*.

15. Deployment plan and future scope

15.1 Deployment plan (Phase 9)

Part	Planned host	Settings (environment variables, never in code)
Backend (Docker image or JAR)	Render or Railway	<code>SPRING_PROFILES_ACTIVE=prod</code> , <code>DB_URL</code> , <code>DB_USERNAME</code> , <code>DB_PASSWORD</code> , <code>JWT_SECRET</code> , <code>CORS_ALLOWED_ORIGINS</code> , <code>FRONTEND_URL</code> , <code>MAIL_*</code> , <code>STRIPE_*</code>
Database	Managed MySQL (e.g. Aiven or Railway)	automatic backups
Frontend (static files from <code>npm run build</code>)	Vercel or Netlify	a rewrite that forwards <code>/api/</code> to the backend, so the cookie stays on one site
Email	a free SMTP service (optional)	<code>MAIL_HOST</code> , <code>MAIL_USERNAME</code> , <code>MAIL_PASSWORD</code>

The `prod` profile already switches on secure cookies, requires all secrets (no defaults), and refuses the fake payment gateway. The "done when" check of Phase 9: *a friend can open the link on their phone, register, book, get a QR ticket and download a certificate.*

15.2 Future scope

Idea	Why it matters	How
Password reset by email	Students forget passwords	one-time token table, email link, expiry
Real volunteer management page	Organizers add and remove gate volunteers	endpoints on <code>club_members</code> with role <code>VOLUNTEER</code>
Installable app / offline scanner	Weak Wi-Fi at the gate	Progressive Web App, queue scans and sync later
UPI payments	Most Indian students pay with UPI	Razorpay behind the same <code>PaymentGateway</code> interface
Several servers	More users	Redis for the login lock and the cache; sticky-free design already (stateless JWT)
Several colleges	One platform for a whole university	a <code>college_id</code> on clubs and users, filtered everywhere
Recommendations	Students discover more events	suggest events by tags the student booked before

16. Glossary

Term	Meaning in simple words
API	A set of URLs a program can call to use another program's features
Audit log	A diary of who changed what and when
BCrypt	A slow, salted way of hashing passwords so they cannot be turned back
Bean	An object created and managed by Spring
Cache	A short-term memory of an answer, to avoid computing it again
CI (continuous integration)	Automatic build and tests on every push
Component (React)	A reusable piece of the screen, written as a function
Container (Docker)	A small isolated box that runs one program with everything it needs
CORS	Browser rule about which websites may call an API
CSRF	An attack where another website makes your browser send a request; blocked with a secret header
Deadlock	Two transactions waiting for each other forever
Dependency injection	Spring hands a class the objects it needs, instead of the class creating them
DTO	A small class or record describing the shape of JSON sent or received
Entity	A Java class that represents one database table
Flyway migration	A numbered SQL file that changes the database in a controlled order
Foreign key	A column that must point to an existing row in another table
Hash	A one-way fingerprint of data
HttpOnly cookie	A cookie JavaScript cannot read
Idempotent	Doing it twice gives the same result as doing it once
Integration test	A test of several parts working together (here with a real database)
JPA / Hibernate	The standard and the library that map Java objects to tables
JPQL	A query language like SQL, but written with entity and field names
JWT	A signed token that says who you are, valid until it expires
Mock	A fake object used in a test
Optimistic locking	Save only if nobody else changed the row since you read it (version check)
Outbox pattern	Save the message with the change; deliver it only after the change is committed
Race condition	A bug that happens only when two things run at the same moment
Redux slice	One area of the shared frontend state, with its actions
Repository	An interface that reads and writes one kind of entity
REST	A style of API built on URLs and HTTP methods
Specification	A reusable piece of a WHERE clause that can be combined with others
Testcontainers	A library that starts real databases in Docker for tests
Thunk	A Redux action that does async work (like an API call)
Transaction	A group of database changes that succeed or fail together

Term	Meaning in simple words
Webhook	A call from another company's server to ours when something happens (e.g. "paid")

17. Viva and presentation questions with answers

Short, honest answers you can say in your own words. Grouped by topic; the most common questions come first in each group.

17.1 About the project

1. What is EventHub in one sentence? A college event platform where students find, book and pay for events, enter with a one-time QR ticket, and get a verifiable certificate, while clubs create events, the admin approves them and volunteers check tickets.

2. Why did you build it? Colleges run events on WhatsApp and Google Forms, which causes overbooking, shared ticket screenshots, late and fake-able certificates and no numbers for organizers. EventHub solves each of these with a rule in the software.

3. What is special compared with a simple CRUD project? It stays correct when many people act at once (no overselling, no double entry), it handles money (holds, payments, refunds, idempotent webhooks), and it has real-world features: waitlist offers, emails after commit, QR check-in, PDF certificates and cached analytics - all covered by automated tests.

4. Who are the users? Visitor, student, organizer (per club), volunteer (per club) and admin.

5. How long did it take and how did you organise it? Ten phases, from Java basics to launch, each with a checklist and "done when" checks; small commits, every push tested by GitHub Actions.

17.2 Architecture

6. Explain the architecture. Three tiers: a React single-page app in the browser, a Spring Boot REST API, and MySQL. Inside the backend: security filters, controllers, services, repositories.

7. Why separate controller, service and repository? Each has one job: the controller handles HTTP, the service holds the business rules and transactions, the repository talks to the database. Rules live in one place and each layer can be tested alone.

8. Why package by feature? Everything about one feature (for example booking) is in one folder, so it is easy to find, change and test.

9. How do the frontend and backend communicate? JSON over HTTP. The browser calls `/api/...`; the Vite dev server forwards it to port 8080. The login travels in an `httpOnly` cookie and the CSRF token in a header.

10. What is a REST API? A set of URLs with HTTP methods (GET to read, POST to create or act, PUT to replace) that return data, usually JSON, with status codes such as 200, 201, 400, 404 and 409.

17.3 Backend and Spring

11. What is Spring Boot? A Java framework that sets up a web server, database access, security and more automatically, so we only write the business logic.

12. What is dependency injection? Spring creates the objects (beans) and passes each class the beans it needs through its constructor, instead of the class creating them itself. It makes code loosely coupled and easy to test with mocks.

13. What does @Transactional do? It runs a method in a database transaction: all its changes are saved together, or none of them if an exception happens.

14. What is JPA / Hibernate? JPA is the standard for mapping Java classes to tables; Hibernate is the library that implements it and writes the SQL.

15. What is the N+1 problem and how did you avoid it? Loading a list (1 query) and then the details of each item separately (N queries). I used `@EntityGraph` to load the club with the events and `@BatchSize` to load all tags in one extra query.

16. How does search with filters work? With JPA Specifications: small WHERE-clause pieces (text, tag, club, dates) combined only when the filter is filled, plus server-side paging and sorting.

17. How do you handle errors? A global `@RestControllerAdvice` turns exceptions into Problem Details JSON with the right status: 400 validation, 404 not found, 409 business rule, 401/403 security, 429 login lock, 500 unexpected (details only in the log).

18. What is Flyway and why use it? It runs numbered SQL migration files in order and remembers which ran, so every database is identical. Hibernate only validates the schema.

19. What are scheduled jobs in your project? Every minute: expire unpaid holds, expire unanswered waitlist offers, retry failed emails. Every day at 18:00: reminders for tomorrow's events. They use `@Scheduled` and are switched off in tests.

17.4 Concurrency and correctness

20. How do you prevent overbooking? Optimistic locking: the event has a `@Version` column. An UPDATE only succeeds if the version is unchanged; the loser gets an exception and retries in a new transaction with fresh data, then sees fewer seats.

21. Why optimistic and not pessimistic locking? Optimistic locking does not hold database locks while the user thinks, scales better and conflicts are rare. Pessimistic `SELECT ... FOR UPDATE` would make everyone wait.

22. How did you prove it works? A test starts 100 threads for 10 seats at the same moment, three times: exactly 10 bookings each time. Without `@Version` the same test sold 96-99.

23. What deadlock did you meet? Two booking transactions locked the event and booking rows in different orders and waited for each other. The fix was to always update the event row first (`saveAndFlush`) before inserting the booking.

24. How does the gate stop a ticket being used twice? One conditional UPDATE: set `checked_in_at` only WHERE `checked_in_at` IS NULL. If it changed 1 row, let them in; if 0 rows, it was already used. A test with 20 gates proves exactly one VALID.

25. What is REPEATABLE READ and how did it affect you? MySQL's default isolation: a transaction keeps seeing the snapshot from its first read. The losing gate re-read the ticket and still saw "unused", so I decide from the UPDATE's row count instead of re-reading.

26. What is idempotency and where do you use it? Doing something twice gives the same result as once. Stripe may send the same webhook twice; each message id is inserted into a table with a primary key, so the second one is ignored. PUT for feedback is also idempotent.

17.5 Security

27. How is the password stored? As a BCrypt hash with a random salt; never in plain text.

28. What is a JWT and what is inside yours? A signed token with the user id, email, name, roles and expiry (8 hours), signed with HMAC-SHA256 using a secret key.

29. Why store the JWT in an httpOnly cookie? JavaScript cannot read it, so a malicious script cannot steal the login. localStorage would be readable.

30. Then how do you stop CSRF? SameSite=Lax cookies plus a CSRF token: the server sets a readable XSRF-TOKEN cookie, Axios copies it into a header, and other websites cannot read our cookie to forge the header.

31. Authentication vs authorization? Authentication proves who you are (password, then JWT). Authorization decides what you may do (roles and club rules, checked on every request).

32. Why are club roles not in the JWT? So that removing an organizer works immediately; a token would keep the old role until it expires.

33. How do you stop password guessing? Five wrong passwords for the same email from the same address lock that pair for 15 minutes (429 with Retry-After).

34. Can a student open admin pages by typing the URL? The page shows "no permission", and more importantly every admin API call returns 403, because the server checks the role.

35. How do you keep secrets out of GitHub? They live in `.env`, which is git-ignored; `.env.example` has placeholders; I scanned the code and the whole git history.

17.6 Payments, waitlist, notifications, certificates

36. How does payment work? Seats are held for 10 minutes, the server creates a Stripe Checkout session (or the built-in test page in development), the student pays on Stripe's page, and Stripe's signed webhook confirms the booking. A verify-on-return call is the backup.

37. Why the PaymentGateway interface? The booking code does not depend on Stripe. The same code works with Stripe or the fake test gateway, and a new provider (Razorpay) only needs a new class.

38. What if the payment arrives after the hold expired? The code tries to take the seats again; if they are gone, the payment is refunded automatically.

39. Explain the waitlist. When an event is sold out, students join a queue. Freed seats are offered to the first students that fit and kept for them 30 minutes; if not accepted, they go to the next.

40. Why send emails after commit? If the transaction rolled back after sending, the student would get an email about something that did not happen. So the notification is saved in the transaction and the email is sent after the commit, with retries.

41. How are certificates protected against faking? Only checked-in students of ended events get one; the number is random; anyone can check a number on the public verify page.

42. How are QR codes made? The server draws a PNG with ZXing. The QR contains only the random ticket code, nothing private.

17.7 Frontend

43. Why React? Component-based UI, a huge ecosystem, and fast updates of only the changed parts.

44. What is Redux used for? Data many pages need at once: the logged-in user, held seats, notifications, the organizer's events, the admin's queue. Async thunks call the API with pending/fulfilled/rejected states.

45. How do you protect frontend routes? A `RequireAuth` wrapper: spinner while checking, login page (remembering `?next=`) when not logged in, a friendly message when not allowed. The backend still checks every call.

46. How does form validation work? `useForm` validates with small rule functions, shows errors after leaving a field or on submit, focuses the first error and can show errors returned by the server.

47. What is code splitting? Loading some pages only when they are opened (React.lazy). It cut the first download from 1,213 kB to 329 kB.

48. How did you make it accessible and mobile-friendly? Labels on all inputs, `role="alert"` for scan results, a table view for charts, a 375 px phone check of every page, and a contrast check (4.5:1) in light and dark mode.

17.8 Testing and quality

49. What kinds of tests do you have? Unit tests with Mockito, integration tests with MockMvc and a real MySQL from Testcontainers, concurrency tests, React component tests with Vitest and Testing Library, and a scripted end-to-end tour.

50. What is a mock? A fake object that returns what the test says, so one class can be tested alone.

51. Why Testcontainers instead of H2? H2 behaves differently from MySQL (locking, SQL details, isolation). Testcontainers runs the same MySQL 8.4 as development.

52. What is your coverage? 91% of the service code, 92% of the whole backend, measured with JaCoCo.

53. What is CI? GitHub Actions runs all backend and frontend checks on every push, so a broken change is noticed at once.

54. Tell me about a bug you found. The gate scanner went blank after the first scan. My automated Edge tour found it: while refactoring for tests I had moved a colour table to another file but one list still used it. I fixed the import, added a page test that fails without the fix, and switched on the lint rule for undefined variables.

55. Tell me about a flaky test. A JWT tamper test failed about 1 in 16 runs. It changed the last base64 letter of the signature, but that letter has two unused padding bits, so sometimes the bytes did not change and the token was still valid. Changing the first letter fixed it.

17.9 Database

56. How many tables and which are the most important?

17. The core are users, clubs, events, bookings and payments; then waitlist_entries, notifications, certificates, feedback and audit_log.

57. Why VARCHAR + CHECK instead of ENUM? Easier to add a value later, readable, and maps naturally to Java enums stored as strings.

58. What constraints protect the data? Primary and foreign keys, UNIQUE (email, ticket code, one certificate and one rating per booking, one processed webhook id), CHECK (statuses, quantity 1-10, rating 1-5) and NOT NULL.

59. Why keep available_seats instead of counting bookings? One number is fast to read and, with the version column, easy to keep correct; counting bookings on every request would be slow and race-prone.

17.10 Future and reflection

60. What would you improve? Deploy it, add password reset, a real volunteer management page, UPI payments, an offline-capable scanner app, and Redis for running several servers.

61. What did you learn most? That correctness under concurrency and security must be designed and tested on the server, and that small, tested steps make a big project manageable.

62. What was the hardest part? The booking engine: holds, payments that arrive late, refunds, deadlocks and proving with 100 parallel threads that it never oversells.

18. How to present EventHub (demo script)

This chapter is a ready-made script for a 10-12 minute presentation with a live demo. The same flow is used in the narrated demo video (Tamil audio, English subtitles).

18.1 Before you start (checklist)

1. Start Docker Desktop, then `docker compose up -d`.
2. Start the backend (IntelliJ or `mvnw spring-boot:run`) and check `http://localhost:8080/api/health`.
3. Start the frontend (`npm run dev`) and open `http://localhost:5173`.
4. Open Mailpit (`http://localhost:8025`) in a second tab.
5. Keep the demo password ready (from `.env`), and the PowerPoint open on slide 1.
6. Zoom the browser to 110-125% so the audience can read it.
7. Plan B: if Wi-Fi or a laptop fails, use the screenshots in the slides or play the demo video.

18.2 The flow (about 11 minutes)

Time	Show	Say (short version)
0:00	Slides 1-3	"EventHub runs college events end to end. Today colleges use WhatsApp and Forms, which causes overbooking, fake tickets and late certificates."
1:30	Slides 4-7	"Five kinds of users. React frontend, Spring Boot backend, MySQL. Every request passes security, controller, service, repository."
3:00	Live: home → search "robotics" → event page	"Search and filters run on the server; visitors see only published events."
4:00	Live: log in as Ravi → Book now → checkout timer → Pay → QR ticket	"Seats are held 10 minutes. Optimistic locking guarantees nobody gets the same last seat - proven with 100 threads."
5:30	Live: log in as Priya → Scanner → type the code twice	"One conditional UPDATE: the first scan lets in, the second says ALREADY USED with time and name."
6:30	Live: Ravi's certificate → public verify page	"Only attendees of finished events get one; anyone can verify the number."
7:30	Live: Madhavan → create event → submit; admin → approve	"Draft, pending approval, published - a small state machine; every step in the audit log."
8:30	Live: organizer analytics	"The database counts with GROUP BY; the answer is cached 60 seconds."
9:00	Live: Kavya opens /admin	"The page hides it, and the server answers 403 anyway."
9:30	Slides 16-19	"BCrypt, httpOnly JWT, CSRF, login lock. 146 tests, 91% coverage, CI on every push. Hard problems: deadlock, snapshot, flaky test."
10:30	Slides 20-21	"Next: deployment, password reset, UPI. Thank you - questions?"

18.3 Tips for presenting

- **Tell a story, not a feature list:** follow Ravi from "I want to go to the hackathon" to "I have a certificate".
- **Show one hard thing well** instead of ten things quickly - the gate double-scan is the best "wow" moment.
- **Say numbers:** 100 threads, 10 seats, exactly 10 bookings; 146 tests; 91% coverage.
- **Admit limits honestly** ("Stripe was tested with mocks; the live card test is planned for my main project").
- If something fails live, stay calm, explain what should happen, and show the screenshot in the slides.

19. How EventHub was built: the phase-by-phase story

Each phase had its own checklist document (in the `Phase_*` folders) with steps and "done when" checks. A phase was finished only when every check passed.

Phase	Name	What was built	"Done when" (examples)
0	Foundations	Java console version (classes Event, Student, Booking; a Queue waitlist; a custom SoldOutException), a small JavaScript page, SQL practice	Booking, sold out, cancel and waitlist work in the console
1	Creating the project	Git repository, Spring Boot 4.1 backend, React + Vite frontend, Docker Compose (MySQL + Mailpit), health check, CI, first page sketches	React shows "Backend: UP"; <code>docker compose up</code> works; CI green
2	Backend	9 tables (Flyway V1-V2), entities, repositories, DTOs, EventService rules, search with filters and paging, approval state machine, error handling, Postman collection	44 tests; Postman 43 checks green
3	Frontend	Design tokens and dark mode, UI components, all 25 pages with sample data where needed, Redux store, mobile check at 375 px	Every page works on a phone, light and dark
4	Connecting	CORS; organizer and admin pages on the real API; server field errors shown in forms	Create → submit → approve works through the UI
5	Security	Register/login, BCrypt, JWT in httpOnly cookie, CSRF, club roles with @PreAuthorize, login lock, audit log, demo accounts	Student gets 403 on admin APIs; an organizer cannot edit another club's events
6	Booking and payments	Seat holds, optimistic locking + retry, expiry job, payment gateway (Stripe / test page), idempotent webhook, late-payment refunds, My tickets	100 threads / 10 seats → exactly 10 bookings
7	Signature features	Smart waitlist, notifications + emails + reminders, QR tickets, gate scanner, PDF certificates + verify, feedback, analytics + cache	Cancel → offer email to #1 waitlisted; same ticket twice → ALREADY USED; only checked-in students get a certificate
8	Testing and polish	JaCoCo, Mockito unit tests, Testcontainers, React tests, admin-only Swagger/Actuator, secret scan, contrast and mobile fixes, code splitting, cleanup	CI green; service coverage ≥ 70% (reached 91%)
9	Launch	Deployment, README, demo video, v1.0 tag	A friend can register, book, get a QR ticket and a certificate on their phone

Real-life example: Building in phases is like building a house: foundation, walls, wiring, plumbing, painting, then the house-warming. Each stage is inspected before the next one starts, so a problem is found while it is still small.

20. How to change EventHub (developer how-to)

Four common tasks, step by step, so a new developer can extend the project safely.

20.1 Add a new column (example: a "meeting link" for online events)

1. **Migration** - create `V10__add_meeting_link.sql` (never edit V1-V9):

```
ALTER TABLE events ADD COLUMN meeting_link VARCHAR(300);
```

2. **Entity** - add the field to `Event`:

```
@Column(name = "meeting_link", length = 300)
private String meetingLink;
```

3. **DTOs** - add `meetingLink` to `EventRequest` (with `@Size(max = 300)` and maybe `@URL`) and to `EventDetailResponse`; map it in `EventMapper` and `EventService.applyRequest`.
4. **Frontend** - add a `TextField` to `EventFormPage` and show the link on `EventDetailsPage` for ticket holders.
5. **Test** - extend `EventServiceTest`: create an event with a link, read it back.
6. Run `mvnw test`, `npm test`, commit, push; CI must be green.

20.2 Add a new endpoint (example: "my upcoming events count")

1. Service method with the rule: `public long upcomingCount(Long userId)` in `BookingService`, `@Transactional(readOnly = true)`.
2. Repository query: a `@Query("select count(b) from Booking b where ...")` method.
3. Controller: `@GetMapping("/api/bookings/mine/upcoming-count")` returning `Map.of("count", n)`.
4. Security: it is under `/api/**` and not public, so it already needs login - check `SecurityConfig` if you want different rules.
5. Test with `MockMvc`: log in with `TestAccounts`, call the URL, check the number; also check that a visitor gets 401.
6. Frontend: add `fetchUpcomingCount` in `api/bookings.js` and use it with `useAsync` on a page.

20.3 Add a new page

1. Create `src/pages/MyPage.jsx` (a function component that returns JSX).
2. Add a route in `App.jsx`; wrap it in `RequireAuth` if it needs login; use `lazy(() => import(...))` if it is large or used by few people.
3. Add a link in `Navbar.jsx` or a sidebar.
4. Use the UI components (`Card`, `Button`, `EmptyState`, `Skeleton`) so it matches the design and dark mode.

5. Write a component test with React Testing Library if it has logic.
6. Check it at 375 px width and in dark mode.

20.4 Add a new business rule (example: max 2 active bookings per student per day)

1. Write a **failing test** first in `BookingFlowTest`: book three times on the same day, expect the third to answer 409 with a clear message.
2. Add the check in `BookingService.holdOnce`, throwing `BusinessRuleException` with a friendly sentence.
3. Run the test until it passes; run all tests.
4. Show the message in the UI - it already appears automatically, because pages display the `detail` of any 409.

Note: Golden rule for every change: the **server** enforces the rule, a **test** proves it, and **CI** is green before the change counts as done.

Appendix A. Commit history

Every change of the project, in order, as recorded by Git.

#	Commit	When	What changed
1	dc97e9e	27 Sep 22:15	Initial commit: EventHub project structure and planning docs
2	e94ee0e	27 Sep 22:20	Phase 1 step 3: generate Spring Boot 4.1.1 backend (Java 25)
3	153b962	28 Sep 11:17	Phase 1 step 3: ignore IntelliJ out/ folder
4	6f582ed	28 Sep 11:24	Phase 1 step 4: React frontend with Vite, Tailwind, Router, Redux Toolkit, Axios
5	572c2a1	28 Sep 11:40	Phase 1 step 5: docker-compose with MySQL 8.4 and Mailpit
6	f880f76	28 Sep 11:40	Phase 1 step 6: connect Spring Boot to MySQL with dev and prod profiles
7	88cfbe0	28 Sep 11:40	Phase 1 step 7: /api/health, Swagger UI, security rules; React shows backend status
8	79dbb12	28 Sep 11:40	Phase 1 step 8: GitHub Actions CI for backend and frontend
9	737c6b1	28 Sep 11:40	Phase 1 step 8: make mvnw executable for CI
10	6711b5f	28 Sep 11:48	Phase 1 docs: Phase1_Explained (Word + PDF)
11	404f20e	28 Sep 12:03	Phase 1 step 9: first-draft sketches (4 pages + ER diagram) for review
12	f4c4496	28 Sep 12:21	Phase 1 complete: sketches approved
13	34118f7	28 Sep 12:24	Phase 2 step 1: Flyway V1 core tables (users, roles, profiles, clubs, club_members, events, tags, event_tags)
14	e5e9c68	28 Sep 12:24	Phase 2 step 2: Flyway V2 sample data (5 clubs, 10 tags, 20 events)
15	b1e0f01	28 Sep 12:34	Phase 2 step 3: JPA entities with One-to-One, One-to-Many and Many-to-Many relationships
16	2978105	28 Sep 12:34	Phase 2 step 4: Spring Data JPA repositories + tests on sample data
17	163860a	28 Sep 12:37	Phase 2 step 5: DTOs (event summary/detail/request, review, club, page) + mapper
18	1a7ed0d	28 Sep 12:40	Phase 2 step 6: EventService with business rules + exceptions + tests
19	93488b0	28 Sep 12:44	Phase 2 step 7: event APIs with search, tag/club/date filters, sort and pagination; clubs and tags lists
20	5262daa	28 Sep 12:48	Phase 2 step 8: approval workflow (submit, approve, reject with reason) as a state machine
21	ac58479	28 Sep 12:51	Phase 2 step 9: @Valid input checks and global exception handler (400/404/409 problem-details JSON)
22	532c113	28 Sep 12:55	Phase 2 step 10: separate eventhub_test database for automated tests
23	7ceac4f	28 Sep 12:55	Phase 2 step 10: Postman collection (25 requests, 43 checks) and Swagger API info; Phase 2 complete
24	2705979	28 Sep 13:13	Phase 3 steps 1-3: design tokens + dark mode, shared components, all routes with 404 page
25	739a47c	28 Sep 13:20	Phase 3 step 4: Home, Events (filters, sort, pagination), Event details and Club pages on the real API
26	68bafd3	28 Sep 13:26	Phase 3 step 5: Login and Register pages with live form validation and password strength meter

#	Commit	When	What changed
27	dd0f8b1	28 Sep 13:47	Phase 0: console app (OOP, collections, SoldOutException, Queue waitlist), JS event list page, SQL practice
28	4a7628f	28 Sep 13:48	Phase 0 complete: reviewed and approved
29	72b2f09	28 Sep 13:54	Phase 3 step 6: student pages - checkout with 10-minute seat-hold countdown, QR ticket, My tickets, waitlist, certificates (sample data)
30	d0cd020	28 Sep 13:59	Phase 3 step 7: organizer area - dashboard layout, club overview with stats, create/edit event form, submit for approval, volunteers (sample data)
31	be0fb18	28 Sep 14:10	Phase 3 step 8: volunteer gate scanner - camera QR (BarcodeDetector) or typed code, big green/red result, no double check-in, recent scans (sample data)
32	0f2ed4c	28 Sep 17:12	Docs: Project Guide v1.1 - main project named TriVoKo, Java 25 / Spring Boot 4
33	a441491	28 Sep 17:18	Phase 3 step 9: admin area - approval queue (approve / send back with reason, quick reasons, 500-char limit) and overview (totals, per-club table, recent decisions); orga
34	85d1511	28 Sep 17:28	Phase 3 step 10: Redux Toolkit store - auth, cart (seat hold survives page changes, navbar timer), notifications (toasts), student / organizer / admin slices replace the
35	bac91b5	28 Sep 17:34	Phase 3 step 11: mobile check of all 25 pages at 375px (light + dark) - fix sideways scroll on Volunteers (sr-only label escaped the table scroller), 24px+ tap height for
36	3624223	28 Sep 17:39	Phase 4 step 2 + backend for steps 6-7: CORS (app.cors.allowed-origins), GET /api/organizer/clubs/{id}/events and /events/{id} (any status), GET /api/admin/events/pending
37	5325c59	28 Sep 17:45	Phase 4 steps 1, 5-7: organizer + admin pages on the real API (createAsyncThunk: loadClubEvents, saveEvent, submitEvent, loadApprovalQueue, loadClubStats, approveEvent, r
38	bdf61a1	28 Sep 18:10	Phase 5 steps 1-6 (backend): register (BCrypt), login/logout with JWT in httpOnly cookie, JwtCookieFilter, CSRF (spa + cookie filter), roles STUDENT/ADMIN + club roles vi
39	0635a07	28 Sep 18:12	Phase 5 step 8: audit log (V3 audit_log table) - CREATE/UPDATE/SUBMIT/APPROVE/REJECT recorded with the user in the same transaction; GET /api/admin/audit; 70 tests
40	26eb862	28 Sep 18:13	Phase 5 step 9: login rate limit - 5 wrong passwords per email+address locks login for 15 min (429 + Retry-After); 73 tests
41	653e437	28 Sep 18:26	Phase 5 step 7 (frontend): real login/register/logout (auth slice + /api/auth/me on start), RequireAuth routes by role, menu by role, organizer club from the account (swi
42	6d33f34	28 Sep 18:46	Phase 6 steps 1-10 (backend): V4 bookings/payments/processed_payment_events, hold API (HELD 10 min / free CONFIRMED), optimistic locking + retry, hold-expiry job, Payment
43	1b1c1e6	28 Sep 19:14	Phase 6 step 11 (frontend): Book button holds seats on the server, checkout with server timer + Pay, test payment page (dev), payment success/cancelled pages, My tickets
44	b8cb5ac	28 Sep 19:14	Tools: edge-demo.mjs - scripted EventHub booking demo in a separate Edge window (DevTools protocol)
45	a1a81c8	28 Sep 20:14	Tools: edge-tour.mjs - 20-step guided tour in Edge with explanations (student, organizer, admin, security)

#	Commit	When	What changed
46	548a66b	29 Sep 09:45	Phase 7 step 1: smart waitlist - join (position "You are #N"), freed seats kept for the next student for 30 minutes, accept -> normal booking, leave / offer expiry -> nex
47	5802ef6	29 Sep 10:01	Phase 7 step 2: notifications - bell with unread count + emails (Mailpit) for seat offers, confirmed bookings, organizer approval decisions, and a day-before reminder job
48	497cc32	29 Sep 10:10	Phase 7 steps 3-4: QR tickets drawn by the server (ZXing) and the gate scanner on the real API - VALID / ALREADY USED / INVALID, live check-in counter
49	dca270c	29 Sep 10:53	Phase 7 step 5: PDF certificates (OpenPDF) only for tickets scanned at the gate, after the event; public /verify page
50	ba3f082	29 Sep 11:06	Phase 7 step 6: feedback - 1-5 stars + optional comment, only from students scanned at the gate, after the event; organizers see average, stars and anonymous comments
51	df4cbfd	29 Sep 19:38	Phase 7 step 7: analytics dashboard - tickets and money per day, check-in rate, ratings (JPQL GROUP BY); @Cacheable with Caffeine (60 s); Recharts pages for organizers an
52	8c8619c	29 Sep 19:38	tools: the Edge tour picks a paid event Ravi has not booked yet (was always Robo Race)
53	29ee449	29 Sep 19:44	Phase 8 step 1: JaCoCo coverage + Mockito unit tests (Stripe gateway with mockStatic, Retry, analytics maths, timer jobs); fix a flaky JWT test (last base64 letter has pa
54	5c7e8ff	29 Sep 19:47	Phase 8 step 2: integration tests on Testcontainers (MySQL 8.4 started once per test run); CI no longer needs a MySQL service; remove the old eventhub_test init script
55	fc145b8	29 Sep 19:53	Phase 8 step 3: Vitest + React Testing Library - countdown timer (fake timers), gate scanner result panel, login form (validation, server error, safe ?next=); npm test in
56	b770b68	29 Sep 19:56	Phase 8 step 4: Swagger and Actuator for ADMIN only (health stays public, details for admin); secret scan of code + history clean
57	13842f5	29 Sep 20:00	Phase 8 step 5 (public pages): contrast fixes - dark-mode grey text one step lighter (62 places), primary button brand-600 in dark, green panels green-700; footer links 2
58	badf298	29 Sep 20:14	Phase 8 step 7: cleanup - React.lazy code splitting (first download 1213 kB -> 329 kB; charts and scanner load on demand), remove 3 unused repository methods, update stal
59	ef97bbe	29 Sep 20:18	README: Phases 3-8 complete, known limitations of the practice project
60	9cbb031	29 Sep 20:39	Fix: gate scanner crashed after the first scan (colour table moved to ResultPanel in step 3); scanLook.js shared, ScannerPage test, oxlint no-undef on; Edge tour: waits f